

Programming Guide



Programming Guide

The LithTech Jupiter system programming guide contains information relating to coding games using the Jupiter system. Previous sections have described how Level Designers and Graphic Artists can create levels, models, and brushes using the tools provided with Jupiter. This section focuses on how to create your game code to interact using the levels and objects you have created.

The following sections contain information specific to programming Jupiter games:

Section	Description
Programming Overview	Introduction to programming the Jupiter system, and information about the sample base provided with Jupiter.
Interface Manager	Describes a consistent method for the game engine code to define, implement, instantiate, and retrieve interfaces.
Client Shell	Describes client-side game code operations and logic.
Server Shell	Describes server-side game code operations and logic.
Simulation Objects	Describes how to create, define, publish and use objects that represent items in the game environment.
Audio	Describes how to use sound and music in your game.
Models and Animation	Describes how to create and animate models for use as props in your game.
Rendering	Describes how to render primitives, objects, and world geometry.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

Programming Guide



Programming Overview

Jupiter contains dozens of interfaces, scores of structures and enumerated types, and hundreds of functions. This section provides Jupiter-specific advice and guidance for common game coding tasks. Of course, review of the source code is an invaluable and irreplaceable requirement to understanding Jupiter and successfully developing your own game. Jupiter includes the full source code for the sequel to No One Lives Forever. You can use this game code to help you understand the engine code.

There are many ways to code a game using Jupiter. This document explains issues and subjects that you should keep in mind when developing your game. Your game code may radically depart from the simple examples given here and still operate normally.

This section contains the following topics:

Topic	Description
Getting Started	Explains how to build and compile your Jupiter game.
System Overviews	Describes the numerous systems used to engineer games using Jupiter.
Common Tasks	Provides guidance on completing tasks common to most games using the Jupiter system.
Collecting Input	Describes how to collect information from hardware input devices.
File Structure	There are several files that all Jupiter games must include.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

Programming Guide



Getting Started

Jupiter includes both engine code and the game code for the No One Lives Forever sequel. You can use the latter as a starting point for your own game and as a guide to understanding how various coding tasks are accomplished using Jupiter. This section provides to following directions for getting started developing your own game using Jupiter:

- [Component Locations](#)
- [Building the Jupiter Engine](#)
 - [Building the Tools](#)
 - [Building the Game Code](#)
 - [Running the Game](#)

Component Locations

The various Jupiter components are located in the following directories:

\Engine\	Engine source code.
\Game\	Game source code.
\Development\TO2\Game\	Game resources.
\Development\Tools\	Pre-built tools.

Building the Jupiter Engine

To build the engine (source code licensees only)

1. Open a DOS window.
2. Switch to the \LithTech\LT_Jupiter_Src\Engine\Runtime\ directory.
3. Run the build.bat file

You can also open the build.bat file from an explorer window, but you won't be able to review the output after the build process has completed.

[Top](#)

Building the Tools

To build the engine tools (source code licensees only)

1. Open a DOS window.
2. Switch to the \LithTech\LT_Jupiter_Src\Engine\Tools\ directory.
3. Run the build.bat file.

In order for the exporter plugins to build, you will need to copy over the Max and/or Maya SDK files. For source licensees, see the readme files in the \Development\Tools\Plugins\ sub-folders for more information.

[Top](#)

Building the Game Code

Before building the Jupiter Engine you must have installed Microsoft's DirectX 8.1 SDK. This can be obtained from Microsoft's website.

To build the game code

1. Launch Microsoft Visual Studio v6.0.
2. Select Open Workspace from the File menu.
3. Browse to the \LithTech\LT_Jupiter_Bin\Game\ folder (or \LithTech\LT_Jupiter_Src\Game\ for source licensees).
4. Open the TO2.dsw project file.
5. If any Source Control dialogs appear, click No to dismiss them.
6. Select Build | Batch Build from the menu.
7. Check all project targets except for Final and Demo.
8. Select the Rebuild All button.

You may only want to build all Debug or Release targets.

[Top](#)

Running the Game

To run the game, you can run the game launcher from one of the following two directories:

\LithTech\LT_Jupiter_Bin\Development\TO2\NOLF2.exe

-or for source licensees-

\LithTech\LT_Jupiter_Src\Development\TO2\NOLF2.exe

Command Line Parameters

Here is a list of command line parameters available for launching LithTech. All parameters should be passed using `-<command>`. In addition to the commands listed below, console variables may also be passed by using `+<command and parameters>`. So for example you can run LithTech using the following syntax at the command line:

```
LithTech -rez resource.rez +windowed 1 -WorkingDir c:\somedir\
```

For a list of important console variables, see [Appendix C: Console Variables](#).

Breakonerror—Determines if LithTech should break when an error occurs. This is primarily used for debugging purposes.

config <config filename>—Specifies the configuration file that will be used to configure LithTech. If none is specified, it defaults to `autoexec.cfg`. The `autoexec.cfg` file is always processed, even if an alternate configuration file is specified with this command.

DebugStructBanks—Used primarily for debugging memory leaks, this causes the memory banks to be allocated and released normally.

Host—Forces the host flag to be set so that LithTech will act as though it is hosting the current game.

Launch—This command allows the launch to be handled by the `launch.dll` file, which exports a function `GetLithTechCommandLine` that is called to parse the command line.

noinput—Disables the use of input to LithTech.

rez <rez file>—Specifies the resource that will be used. This can be either a `.rez` file or a resource directory. Multiple files can be specified, each with its own `rez` command. Subsequent files override duplicate resource information in previous `.rez` files.

Windowtitle <title>—Specifies the title of the window that LithTech creates.

Workingdir <directory>—Changes the current working directory to that specified.

World <world name>—Specifies the world that will be loaded upon initialization.

Windowed—This is a console variable and is added to the command line prepended with a `+`. It runs the game in a window as opposed to full screen.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Programming Guide



System Overviews

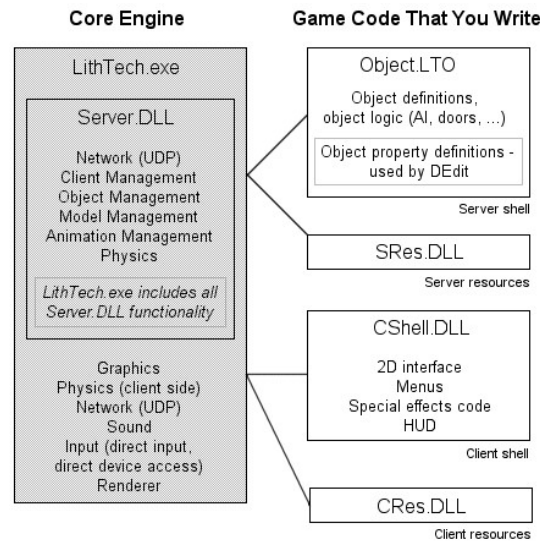
There are many areas of Jupiter code that you must familiarize yourself with in order to successfully complete a game. This section provides short overviews of the following primary Jupiter systems:

- [System Architecture Diagram](#)
 - [Client Shell](#)
 - [Server Shell](#)
- [Interface Manager](#)
 - [Input](#)
 - [Lighting](#)
 - [Physics](#)
 - [Rendering](#)
- [Simulation Objects](#)

Since Jupiter is based on a client-server model, the Client Shell and Server Shell systems are of utmost importance and are discussed first.

System Architecture Diagram

The following image illustrates the system architecture of the LithTech Game Development System:



The Core Engine section is the primary component supplied by LithTech for the creation of your game. The Game Code That You Write is the portion of your game that you must create. The game code supplied with your build only serves as a reference for you to research when determining how to design your game. Actual implementation of Game Code varies greatly from game to game, and depends on many aspects of how you want your game to operate. Read the following sections for descriptions of the components in the System Architecture Diagram.

Client Shell

Client Shell is a term used to encompass all client-side game code operations and logic. The client shell game code provided with Jupiter is located in the `\Game\ClientShellDLL\` folder. You can use this code as a starting place for your own game's client shell.

The client shell communicates with the Jupiter engine code to initiate a variety of operations. Jupiter, in return, notifies the client shell of various events, such as engine initialization, messages from the server, input status changes, and so on.

The client shell performs the following required tasks:

- It calls the **SETUP_CLIENTSHELL** macro to set up required .DLL exports and some global variables. This macro is defined in the `\Engine\sdk\inc\iclientshell.h` header file.
- It creates an **IClientShell**-derived class instance. Jupiter sends notifications to this derived class to inform your game of various events. There are a variety of notification-handling functions that may be implemented. The two most important of these are **OnEngineInitialized** and **Update**. This class is a good starting point for your game code and is defined in the `\Engine\sdk\inc\iclientshell.h` header file.
- The **ILTClient** class is instantiated by Jupiter and provides significant functionality to your game code, such as the **ILTClient::PauseMusic** function. You can obtain a pointer to the instance using the Interface Manager's **define_holder_to_instance** macro, defined in the `\Engine\sdk\inc\ltmodule.h` header file.

[Top](#)

Server Shell

Server Shell is a term used to encompass all server-side game code operations and logic. The server shell game code provided with Jupiter is located in the `\Game\ObjectDLL\` folder. You can use this code as a starting place for your own game's server shell.

The server shell communicates with the Jupiter engine to initiate a variety of operations. The Jupiter engine code, in return, notifies the server shell of various events.

The server shell performs the following required tasks:

- It calls the **SETUP_SERVERSHELL** macro to set up required .DLL exports and some global variables. This macro is defined in the `\Engine\sdk\inc\iservershell.h` header file.
- It creates an instance of the **IServerShell** class. Jupiter sends notifications to this class to inform your game of various events. There are a variety of notification-handling functions that you may implement. Two important functions are **OnClientEnterWorld** and **OnClientExitWorld**. The **IServerShell** class is defined in the `\Engine\sdk\inc\iservershell.h` header file.
- The **ILTServer** class is instantiated by Jupiter and provides significant functionality for your game. You can obtain a pointer to the instance using the Interface Manager's **define_holder_to_instance** macro.

[Top](#)

Interface Manager

The Interface Manager provides a consistent, modular method by which game and engine code can define, implement, instantiate, and retrieve interfaces. Jupiter instantiates a variety of interfaces that offer a wide assortment of operations: client (**ILTClient**) and server (**ILTServer**) procedures, physics (**ILTPhysics**), sound (**ILTSoundMgr** and **ILTClientSoundMgr**), text and user interface (**ILTFontManager** and **ILTWidgetManager**), custom rendering (**ILTDrawPrim**), and more.

To retrieve engine interface instances, the game code uses the Interface Manager. The Interface Manager provides a set of macros, such as **define_holder** or **define_holder_to_instance**, that game code can use to acquire global pointers to instances of various engine interfaces. The Interface Manager macros are defined in the `\Engine\sdk\inc\ltmodule.h` header file.

Notification Calls from Jupiter

Jupiter often notifies game code of events by calling a variety of functions on your implementations of various class instances. The most common notifications from Jupiter are directed at your **IClientShell** and **IServerShell**-derived interfaces. You will also instantiate other classes that receive notifications from Jupiter, such as **BaseClass** and **BaseClassClient**.

Before any of your interfaces can receive notifications from Jupiter, Jupiter must know about them. You can register your interface instances with Jupiter using the **declare_interface** macro.

[Top](#)

Input

Jupiter can support input from a variety of devices. When input is detected from such a device, Jupiter checks to see if you have bound any actions to that input event. If you have, Jupiter notifies your game code of the input event. Your game code can then take appropriate action.

To bind an input event to an action, you must define it in the autoexec.cfg file. The game source autoexec.cfg file is located in the \Development\T02\ folder. This is a text file that lists the various input events that initiate notifications from Jupiter to your game code. It uses the **addaction**, **enableddevice**, and **rangebind** commands to define desired events.

Jupiter notifies game code of input events through the **IClientShell::OnCommandOn** and **IClientShell::OnCommandOff** functions. Each of these notification calls will pass in an integer value identifying the command associated with the input event. Your game code must know what the integer value means so that the appropriate action can be taken.

[Top](#)

Lighting

There are two kinds of lighting in Jupiter: static and dynamic.

Static lighting is set by level designers in the world .LTA file using DEdit.

Dynamic lighting is a simple process. Merely create an object using the **ILTServer::CreateObject** function and the **ObjectCreateStruct** structure with the **m_Type** member variable set to **OT_LIGHT**. Then you can use **ILTClient::SetLightColor** and **ILTClient::SetLightRadius** to alter the lighting effects. You can change the size and position of the light as you would any other object. Dynamic lights have a performance impact, so consider their use carefully.

[Top](#)

Physics

The object physics in Jupiter are dictated by setting the acceleration and velocity of an object. Over time, the acceleration changes the velocity, and the velocity changes the position of an object.

The default model for collision is the axis aligned bounding box. If an object is a world model, the object is treated as a BSP unless the **FLAG_BOXPHYSICS** flag is set for the object. To use cylinder collision detection on objects that are not world models, set the **FLAG2_PLAYERCOLLIDE** bit or the **FLAG2_CYLINDER_CD** bit of the **LTOBJECT::m_Flag2** member. Cylinder collision detection is intended to be used with the player object.

The **ILTPhysics** interface provides a method of controlling the attributes of the objects in relation to the physics, both locally to the object, and globally for the entire world. Each of the object attribute accessor functions has a Get and Set version for retrieving and storing the attribute. This interface is defined in the \Engine\sdk\inc\iltphysics.h header file.

The **ILTClientPhysics** interface defines a way of simulating physics using only the client-side version of the world.

[Top](#)

Rendering

Rendering in Jupiter is handled by engine code. A variety of exposed engine interfaces allow the game code to render primitives, objects, and world geometry.

Use the **ILTClient::SetRenderMode** function and the rendering console variables to initialize the renderer.

The **ILTDDrawPrim** and **ILTTexInterface** interfaces expose functions that enable you to render custom effects, such as low-level primitives and textures.

[Top](#)

Simulation Objects

Simulation objects represent items in a Jupiter application environment. Players, monsters, guns, ammunition packs, lights, key framers, and more are all created as simulation objects. Simulation objects are represented in code by game objects and an associated engine object.

To create a simulation object, you must define a class based on the **BaseClass** class in the Server Shell, or **BaseClassClient** in the Client Shell. Objects receive event notifications from Jupiter through the **EngineMessageFn** and **ObjectMessageFn** functions. You must override these notification functions to handle events of your choice.

Level designers need access to your simulation objects so that they can place them in their levels using the Jupiter DEdit tool. Simulation objects are published to DEdit using the **BEGIN_CLASS** and **END_CLASS** macros.

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Programming Guide



Common Tasks

When you develop an application there are certain common tasks that need to be conducted. This section discusses the following common tasks:

- [Camera Creation](#)
- [Player Creation](#)
- [World Loading](#)

Camera Creation

If you want to see anything other than flat menus in Jupiter, you must define a camera class with which you can instantiate a camera object. Camera objects provide views into your worlds and generally reside on the client.

Note: Jupiter game code defines a camera class in the `\Game\ClientShellDLL\PlayerCamera.h` header file. You can use and modify this class as a starting point for your own camera.

Usually, you will attach the camera object to a player object. There probably is no reason the server needs to know about the camera object. Instead, the server will monitor and be aware of the player object. Therefore, you do not need to derive the camera class from the **BaseClass** class, which is the common practice for creating simulation objects that are monitored by the server. This means you need to create a client-side object for the camera object.

Your client-side camera object still needs an associated engine object to keep track of position and rotation and other information. You can create a client-side camera object using the **ObjectCreateStruct** structure and the **ILTClient::CreateObject** function to retrieve a handle to an associated engine object.

When defining camera classes or using the camera class provided with Jupiter, you should consider the following:

- Since you are only creating a client-side object, you need only define the camera object in the Client Shell project.
 - You must create an **ObjectCreateStruct** and set the **m_ObjectType** member variable to **OT_CAMERA**.
- You must call **ILTClient::CreateObject** and pass in the address of your **ObjectCreateStruct** structure. This function returns a handle to the associated engine object.
- Your camera class should include a member variable to store the handle (**HLOCALOBJ**) to the associated engine object. You will use this handle to conduct various engine operations on the camera, such as setting rotation, position, and field of view.
 - A good place to instantiate the camera object is in constructor of your implementation of the **IClientShell** interface.
 - A good place to initialize your camera object with starting position and rotation data is in the **IClientShell::OnEngineInitialized** function.
- You may want to include a camera state member variable. An enumerated type with elements such as **CAMERASTATE_FIRST_PERSON** and **CAMERASTATE_THIRD_PERSON** could prove useful in keeping track of the camera view.
- Your camera object may need to know about various events. Therefore, you should consider implementing some event notification functions, such as **Update**, **OnCommandOn**, and **OnCommandOff**.
- The camera class's **Update** function is a good place to reposition the camera for player movement, if the camera is attached to a player object. You should retrieve a pointer to the player object and convert that to a handle to the associated engine object using the **ILTClient::ObjectToHandle** function. You can then use the **ILTPysics::GetPosition** parameter to retrieve the player's position and then reposition the camera object using the **ILTPysics::SetPosition** function. You can repeat this process with **ILTPysics::GetRotation** and **SetRotation**.
- You can use the **OnCommandOn** event to reset or center the camera view, or enable other view options such as zoom.

Player Creation

A player is the focal point of the game. A player object represents the application user in the world. Input from the user is often applied to the player object. A camera is often attached to the player object to give the user a view of the world.

A player object must obviously exist on the Client Shell, so the client can move it around and be aware of its existence. A player object usually must also exist on the Server Shell, so that the server can monitor it and distribute movement and other information to all clients. You must define both the client player object and the server player object.

To create player objects for a single-player game

1. Game code sends a **StartGameRequest** instance to Jupiter using the **ILTClient::StartGame** function.
2. Jupiter calls your implementation of the **IServerShell::OnClientEnterWorld** function.
3. Inside the implementation of the **IServerShell::OnClientEnterWorld** function, a server player object is instantiated.
4. When the server player object is instantiated by the game code, Jupiter also instantiates an associated client player object.
5. The client player object notifies the Client Shell that it exists.

Note: For examples and starting points for player objects see the `\Game\ObjectDLL\PlayerObj.h` header file.

When defining player classes or using the player classes provided with Jupiter, you should consider the following issues concerning the Client Shell and Server Shell player classes.

Server Shell Player Class

- The Server Shell player class must derive from **BaseClass**. This class is defined in the `iltengineobjects.h` header file.

- You must define a **BaseClass::EngineMessageFn** function for the player class. This engine sends messages to the game object using this function. Your implementation of **EngineMessageFn** should handle the **MID_** messages and pass them along to the parent **BaseClass::EngineMessageFn** function. Player objects should support at least **MID_PRECREATE**, **MID_OBJECTCREATED**, and **MID_UPDATE**.
- You must define a **BaseClass::ObjectMessageFn** function for the player class. This function receives messages from other objects. You should pass any messages along to the parent **BaseClass::ObjectMessageFn** function.
 - You must register the player object class with the distributed object system using the **distr_class** macro in the class definition.
- The Server Shell player object must be able to identify the client connection. Therefore, your Server Shell player class should include a member variable containing an **HCLIENT** handle to the client.
 - You should instantiate the Server Shell player object in the **OnClientEnterWorld** function of your implementation of the **IServerShell** interface.

Client Shell Player Class

- The Client Shell player class must derive from the **BaseClassClient** class. Among other things, this parent class provides your player object class with distributed object system functionality. This class is defined in the **ltengineobjects.h** header file.
- Deriving your client-side player class from **BaseClassClient** also gives your player class a member variable, **m_hObject**. This is an **HOBJECT** handle to the associated engine object which you can use to access and modify position, rotation, and other aspects of the player object.
 - You must register the player object class with the distributed object system using the **distr_class** macro in the class definition.
- Your player class should include a member variable to store a pointer to the camera object attached to the player object. You can use this pointer to adjust the camera as needed.
 - When Jupiter instantiates the client player object, it does not automatically inform your **IClientShell** implementation of the existence of the new object. Therefore, in your client player class's constructor, you should give a pointer to the new player object to your **IClientShell** instance.
- You may want to include a player state member variable. An enumerated type with elements such as **PLAYERSTATE_WALK**, **PLAYERSTATE_RUN**, **PLAYERSTATE_HOOSIER**, and **PLAYERSTATE_DEAD** could prove useful in keeping track of the player object.

Your player object may need to know about various events. Therefore, you should consider implementing some event notification functions, such as **Update**, **OnCommandOn**, and **OnCommandOff**.

Projected Shadows

To enable projected shadows for a model requires the following tasks be completed by the programmer and the model artist:

- In ModelEdit, set the **User Dim** for the model animations.
- In ModelEdit, set **ShadowEnable** in the **Model Command String**. To override this, set **DrawAllModelShadows** = 1.
 - In ModelEdit, the animation dims on the object must be set to a value that surrounds the object.
 - In DEdit, the lights must be enabled to cast shadows.
 - In the game code, set the console variable **ModelShadow_Proj_Enable** = 1.
- The **FLAG_SHADOW** flag must be set in the model's **m_Flag** member variable.

[Top](#)

World Loading

Jupiter provides world loading code in the **\Game\ClientShellDLL\GameClientShell.cpp** file. Worlds are loaded in the following manner:

1. Create a **StartGameRequest** class instance.
2. Populate the **StartGameRequest** members.
3. Call the **ILTClient::StartGame** function.

Create a StartGameRequest Class Instance

The **StartGameRequest** class is defined in the **\Engine\sdk\inc\LTBaseDefs.h** header file. You must create an instance of this class for use with the **ILTClient::StartGame** function.

Populate the StartGameRequest Member Variables

The **StartGameRequest** class contains several member variables that describe the type of game to be started. Only the **m_Type** and **m_WorldName** member variables need to be set for simple applications that do not involve hosting or networking operations.

m_Type—The type of game to start. Set this variable to **STARTGAME_NORMAL** to start a game without networking capability.

m_WorldName—A string identifying the relative path (from the application's resource directory) to the file containing the world.

Call ILTClient::StartGame

To start the game and load the world, call the **ILTClient::StartGame** function and pass in the address of the **StartGameRequest** instance you just created and populated. The **StartGame** function is defined in the **ILTClient.h** header file.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.



File Structure

Jupiter games require several different files to run. Some of Jupiter's files contain engine code, and other files contain game code that you must create. Example files can be found in the game code, located in the following directory:

<installation directory>\Development\TO2\

Engine Code Files

These files contain engine code.

LITHTECH.EXE—This is the core of the Jupiter system. It functions as the client, and is the program that actually displays what the player sees on the screen, handles input from the player and communicates it to the game, and so on. It also contains all of the server code for Jupiter. When running a single player game or hosting a network game, this functions as the server. LithTech.exe loads cshell.dll and cres.dll. It also loads object.lto and sres.dll when functioning as a server.

SERVER.DLL—This is the server module for Jupiter. The server-only parts of Jupiter are separated into server.dll. This allows the developer to write a program to load the server portion of Jupiter and function as a standalone application. The server.dll file loads object.lto and sres.dll when run as a standalone server.

Game Code Files

These files contain game-specific code. The game code versions of these are located in the \Development\TO2\Game\ directory.

CSHELL.DLL—This file must implement a client shell. IClientShell is a Jupiter class that has certain functions Jupiter knows it can call into, such as the main update function, or the initialization function.

CRES.DLL—This is a standard Windows resource DLL, containing strings and other resource information. Instead of hard-coding dialog and text into your game, it is a good idea to use this DLL for things like a string-table to facilitate localization. This file should also contain graphics for hardware 2D cursor support.

CRESL.DLL—This file is optional. If cresl.dll is present, any resources in cres.dll that are duplicated in cresl.dll will be used from cresl.dll instead of cres.dll. You can put data that has been localized into other languages into this file and thus not have to change any of your game to port it to another language.

OBJECT.LTO—You must include all the server logic code in this file, such as artificial intelligence routines for enemies and scorekeeping for multiplayer games. This file should also define all the objects in your game. The DEdit tool can read them from this file directly. This allows a programmer to make a change to what properties a level designer can set on an object and have the changes immediately reflected in the level-editing package.

SRES.DLL—This file is rarely used, but is the server equivalent of cres.dll.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

Content Guide



Working With Attributes

TO2 Game code uses attributes files to control a wide variety of functions. Attributes files are located in the following directory:

LithTech\LT_Jupiter_Src\Development\TO2\Game\Attributes

If you are using the TO2 game code as a base, or as a reference when creating your own game, then the attributes files are particularly useful in setting the basic functionality of components in the game. Attributes are applied to many components of the game from animations to surface textures. By editing the attributes files you can redefine attributes or add entirely new attributes to your game.

To redefine an attribute, or to add a new attribute, simply open the appropriate attributes file in a text editor, edit the file following the commented instructions, save the file, process your game, and test the results.

The following table describes each of the files contained in the Attributes directory:

File Name	Description
ActivateTypes.txt	Defines activation types such as OpenClose, TipOver, and SoundAlarm. Associates string IDs from Cres.DLL to display when an object is in an On or Off state.
aibutes.txt	Defines the "brains" of the AIs determining how they act and react in the game.
aigoals.txt	Defines goals for AIs used and their requirements.
animationshuman.txt	Defines animation sets used by human models.
animationsmime.txt	Defines animation sets for mimes.
animationsninja.txt	Defines animation sets for ninjas.
animationspierre.txt	Defines animation sets for Pierre.
animationsrat.txt	Defines animation sets for rats.
animationsrobot.txt	Defines animation sets for robots.
animationssupersoldier.txt	Defines animation sets for super soldiers.
animationsunicycle.txt	Defines animation sets for unicycles.
attachments.txt	Defines game objects that can attach to models.

clientbutes.txt	Defines client effects attributes and debuggin information for AIs.
clientsnd.txt	Defines sound path attributes.
commands.txt	Defines groups of commands and messages that can be called by game objects.
coopmissions.txt	Defines multi-player missions that specify the levels in a game. Sets mission goals, sets default weapons, and associates level files with missions.
damagetx.txt	Defines all of the attributes associated with a particular DamageFX
damagetypes.txt	Defines the attributes of the various damage types in the game.
debris.txt	Defines debris used in the game.
fx.txt	Defines the special effects used in the game. The majority of effects are weapons related, but a small minority of effects used for other purposes are also included in this file.
gadgettargets.txt	Defines all of the attributes associated with a particular Gadget Target
intelitems.txt	Defines all of the attributes associated with a particular intelligence item.
inventory.txt	Defines inventory items. Not currently used.
KeyItems.txt	Defines all of the attributes associated with a particular key item type. Key items are required to unlock or perform certain actions in the game.
layout.txt	Specifies positions and sizes for common screen components.
missions.txt	Defines single-player missions that specify the levels in a game. Sets mission goals, sets default weapons, and associates level files with missions.
modelbutes.txt	Defines attributes of model types used in the game.
mp_weapons.txt	Defines WeaponOrder, Ammo, Weapon, Mod, Gear, and Weapon animations for weapons used in the game.
music.txt	Controls the dynamic music. there are two elements of the dynamic music which are controllable: the mood of the music, and the events. This concept maps into direct music's intensities and motifs, respectively.
PopupItems.txt	Defines popup sprites and textures used to display items to the character when they are looking closely at an object.
proptypes.txt	Defines the attributes of props used in the game.
RadarTypes.txt	Defines radar points that appear on the characters radar.
RelationData.txt	Defines relationships between the characters in the game. Relationships are used by the AI to help determine actions.
searchitems.txt	Defines all the attributes associated with a particular item that is found while searching.
serverbutes.txt	Defines attributes for the player and for security cameras. Use the file to adjust how the player can interact with elements of the game.
serveroptions.txt	Defines server options such as control types and game types.
serversnd.txt	Defines server sound attributes such as sound paths, body sounds, and AI sounds.
skills.txt	Defines skills that the player can acquire.
soundbutes.txt	Defines all of the attributes associated with a particular Sound Bute. Defining a SoundBute allows you to make global changes to all the Sounds.
sountfilters.txt	Defines sound filters used in the game.
surface.txt	Defines properties of surface textures.
TriggerTypes.txt	Defines trigger types used in the game.
weapons.txt	Defines weapons used in the game.

[Top](#)

Programming Guide

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.



Sample Base

The samplebase sample is an introductory programming sample that provides a basic framework for Jupiter game code. Licensees can use it to begin building their projects and it is the base project for other samples and tutorials. This documentation presents an overview of the functions and classes in samplebase and the basic flow of a Jupiter game.

The samplebase sample is located in your Jupiter installation directory under `\samples\base\samplebase\`. The `run.bat` file in that directory can be used to launch samplebase. The Escape key can be used to exit the sample.

This section contains the following topics related to the program flow of the sample base:

- [The ClientShell](#)
- [The ServerShell](#)

The Client Shell

A LithTech game begins its life in the game implementation of the **IClientShell** class. The interface for the **IClientShell** class is defined in `\engine\src\inc\iclientshell.h`. The game code must define its own class that inherits from **IClientShell** and implements certain methods. In samplebase, this class is called **CLTClientShell** and is defined and implemented in `\samples\base\samplebase\cshell\src\ltclientshell.h` and `ltclientshell.cpp`. For more information about the Client Shell, see [Client Shell](#).

OnEngineInitialized

The engine first transfers control to your game code in the **IClientShell::OnEngineInitialized** method. This function verifies that various systems are functioning properly, and then attempts to initialize the renderer and sound subsystems of the engine. It also makes a call to **CLTClientShell::InitGame** that will initialize our small game. Lastly, it creates a few classes that we will use on the client.

At this point in the execution of the game, only the `cshell.dll` and `cres.dll` files have been loaded. Usually, the game would display a menu at this point and ask the user to choose what she would like to do next. For this sample, we will be loading a world and jumping into it directly. This is only to reduce the amount of code in the sample.

InitGame

The **CLTClientShell::InitGame** method asks the engine to begin a game. Due to the client/server architecture of the LithTech engine, a game must have a client and a server part, each of which control certain behaviors. Creating or connecting to a server is necessary in order to load a world.

Once connected to the server, the client will load and unload worlds based on what the server instructs it to do. To do this, we create a **StartGameRequest** structure and fill in its members appropriately. We are asking for a **NORMAL** game which means the entire game will run on this computer, and it will not accept outside connections. This will create a standard, single-player game. Also, we tell the server which world to load. The call to **ILTClient::StartGame** initiates all of this. If this method returns failure, then there was a problem creating the server.

At this point, the object `lto` is loaded, and several methods in the **IServerShell** class are called to initiate program flow on the server. These will be covered in a separate section below.

OnEnterWorld

Once the server is initialized, then the **IClientShell::OnEnterWorld** method is called. This method is a good place to initialize any variables or classes on the client that should be reset when you enter a new world. Similarly, when you exit a world, the **IClientShell::OnExitWorld** method is called.

Updates

After your client has entered a world, a regular update loop will be called every frame until the world is exited. This loop consists of one call each to **IClientShell::PreUpdate**, **IClientShell::Update** and **IClientShell::PostUpdate**, in that order. These calls have no major functional difference, but you can place different code in each one to logically structure your program. In samplebase all of the code is in the **Update** method for simplicity.

The **CLTClientShell::Update** method is the main loop for the client game code. It first checks to see if this is the first update it has received. If it is, then it sets up some state on the player object regarding dimensions and mass. The **ILTClient::GetClientObject** method returns the single object which acts as a link between the client and the server. Usually, this object represents your player character or your avatar on the server.

The **Update** method next synchronizes the position of the camera to the player object. The camera is created as a separate object so that it is easier to implement a 3rd person camera or a cinematic cut scene. The call to **PollInput** uses the input system in LithTech to determine if certain input commands are being activated and adjusts the player's position and rotation accordingly. For more information on how to setup input in your game, see [Collecting Input](#). Finally, this method calls **Render** and gives the **CWorldPropsCInt** class a chance to update itself.

Render

The **CLTClientShell::Render** method handles the rendering of the scene each frame. This follows a basic pattern but can be modified to suite your needs. In this simple case, there is only one allowed camera and no heads up display or menu support. In addition to the **ILTClient** methods called here, you can use the CUI windowing toolkit and **DrawPrim** to render any custom effects that you have. For additional information about rendering, see [Rendering](#).

Other Functions

Several other methods in **CLTClientShell** perform useful functions.

The **OnMessage** method accepts messages sent from the server and reads the appropriate data or routes them to other places. Currently, this is used to get the appropriate starting position and rotation, and to set the world properties information.

The **OnCommandOn** method is called whenever a user presses a key or button that is mapped to a command using the input system. Most of our input processing is in **PollInput**, but you should handle commands that should be triggered on keypress in this method.

There are several methods that send the player's current position and rotation to the server. These are **SendPosition**, **SendRotation** and **SendPosAndRot**.

Another useful method is the **SpecialEffectNotify** method. Despite its name, this method is very useful for objects that are not special effects. The engine calls this method when the client receives a message from the server to create an object that has a Special Effects message attached to it.

Any object on the server can have its own message attached to it. This is especially useful for objects that will be largely controlled on the client. Objects that will be controlled on the server typically do not need extra data sent to the client. However, a particle system should be animated on the client and not the server, to save bandwidth, so the server attaches a Special Effects message to the object with the appropriate object data. The client will then read this message and use the data to properly animate the particle system.

There are many other useful functions in the `clientshell` that are called in response to certain events. For further information, see the [IClientShell](#) class in the

[ClientShell](#) section.

[Top](#)

The Server Shell

The server-side part of a LithTech game begins its life in the game implementation of the **IServerShell** class. The interface for the **IServerShell** class is defined in `\engine\src\inc\iservershell.h`. The game code must define its own class that inherits from **IServerShell** and implements certain methods. In **samplebase**, this class is called **CLTServerShell** and is defined and implemented in `\samples\base\samplebase\sshell\src\ltservershell.h` and `ltservershell.cpp`. For more information about the Server Shell, see [Server Shell](#).

OnServerInitialized

Remember from the clientshell section that the servershell is created in response to the call to **ILTClient::StartGame**. Similar to the clientshell, the life of the servershell begins in **IServerShell::OnServerInitialized**. The **CLTServerShell** only verifies that the necessary interfaces to the LithTech engine exist and then returns success. Since we told the engine to load a world in our **StartGameRequest**, we do not have to repeat that on the server. However, you can control the level through the **ILTServer::LoadWorld** and **ILTServer::RunWorld** methods.

Connect to the Server

The engine will then call the **IServerShell::OnAddClient** and **IServerShell::OnClientEnterWorld** methods because our local client is attempting to connect to our server. The **OnClientEnterWorld** method must create an object that represents the client. This special object is called the client object. It may be a model or any other type of object, even an invisible one. This method creates the object and attaches a Special Effects message to it. This message is currently empty, but allows the client shell to receive the callback and process the new object. The method also sends the starting position and rotation to the client.

OnMessage

The other important method in the servershell is the **IServerShell::OnMessage** method. This method will receive messages from the attached clients. This is critical for updating the client's positions, for processing their commands and for having an interactive game. The **CLTServerShell::OnMessage** method only handles the movement of the client object in this case.

Objects

The server shell also includes the code for all of the objects in your game. By placing the code for an object on the server, the engine will automatically update all connected clients about that object's built-in behaviors and properties. You can also send any custom data for each object using the engine's messaging system. These objects also allow level designers to place them in the world and to customize each object's properties. For more information on creating objects, see [Simulation Objects](#).

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

Programming Guide



Attachments Sample

The attachments sample builds upon the foundation of samplebase, and is located in the following directory:

```
\Samples\base\attachments.
```

The Attachments sample demonstrates the use of model attachments.

Server-side Changes

class PropModel

The attachments sample adds only one class, the PropModel class, to the samplebase foundation. The PropModel class contains a fully working example of how to make an attachment, break an attachment, and attach different objects to one socket.

Three "helper" functions exist to allow easy reuse of resources within PropModel. This section describes and shows the source code for each of the helper functions.

The CreateAttachmentObject Function

The first of helper function is the CreateAttachmentObject function. This function simply instantiates a model from the specified input parameters. The following source code shows the implementation of the CreateAttachmentObject function.

```
void Propmodel::CreateAttachmentObject(HOBJECT &hObj, const char* sFilename, const char* sTexture)
{
    ObjectCreateStruct objCreateStruct;
    HCLASS hClass = g_pLTServer->GetClass("BaseClass");

    if (!hClass)
    {
        assert(!"ERROR: Couldn't get BaseClass!");
        return;
    }

    objCreateStruct.Clear();
    objCreateStruct.m_ObjectType = OT_MODEL;
    objCreateStruct.m_Flags = FLAG_VISIBLE;
```

```

        strncpy(objCreateStruct.m_Filename, sFilename, 127);
        strncpy(objCreateStruct.m_SkinNames[0], sTexture, 127);
        BaseClass* m_pObj = (BaseClass*)g_pLTServer->CreateObject(hClass, &objCreateStruct);

        if (m_pObj)
        {
            hObj = m_pObj->m_hObject;
            g_pLTServer->CPrint("Attachment Object Created!");
        }
    }
}

```

The CreateAttachment Function

The next helper function is the CreateAttachment function. This function is responsible for making the actual attachment between the two input models. The following source code shows the implementation of the CreateAttachment function.

```

void Propmodel::CreateAttachment(HATTACHMENT &hAttachment,
                                HOBJECT hChildObject,
                                const char* sSocket,
                                LTVector &vRotOffset,
                                LTVector &vPosOffset)
{
    LTVector vOffset;
    LTRotation rOffset;

    vOffset.Init();
    rOffset.Init();
    hAttachment = NULL;

    HMODELSOCKET hSocket;
    LTRESULT SocketResult = g_pLTModel->GetSocket(m_hObject, sSocket, hSocket);
    if(LT_OK == SocketResult)
    {
        g_pLTServer->CPrint("Got the socket");
    }else
    {
        g_pLTServer->CPrint("GetSocket failed: %s", LTRESULT_TO_STRING(SocketResult));
    }

    LTransform tSocketTransform;
    LTRESULT SocketTrResult = g_pLTModel->GetSocketTransform(m_hObject, hSocket, tSocketTransform, LTTRUE);
    if(LT_OK == SocketTrResult)
    {
        g_pLTServer->CPrint("Got the transform");
    }else
    {
        g_pLTServer->CPrint("GetTrasform failed: %s", LTRESULT_TO_STRING(SocketTrResult));
    }

    // We need to adjust the rotation of our weapon before we make the attachment
    rOffset.Rotate(rOffset.Right(), MATH_DEGREES_TO_RADIANS(vRotOffset.x));
    rOffset.Rotate(rOffset.Up(), MATH_DEGREES_TO_RADIANS(vRotOffset.y));
    rOffset.Rotate(rOffset.Forward(), MATH_DEGREES_TO_RADIANS(vRotOffset.z));
    LTRESULT resultAttachment = g_pLTServer->CreateAttachment(m_hObject,
                                                            hChildObject,
                                                            sSocket,
                                                            &vOffset,
                                                            &rOffset,
                                                            &hAttachment);

    if(LT_OK != resultAttachment)
    {
        g_pLTServer->CPrint("Error creating attachment!!");
    }
}

```

The actual attachment gets created with the following engine call.

```

LTRESULT resultAttachment = g_pLTServer->CreateAttachment(m_hObject,
                                                         hChildObject,
                                                         sSocket,
                                                         &vOffset,
                                                         &rOffset,
                                                         &hAttachment);

```

To remove an attachment, the following call is made.

```

g_pLTServer->RemoveAttachment(m_hAttachment);

```

The Update Function

The PropModel::Update() function is responsible for controlling the playing of animations and which attachments are made. The following source code shows the implementation of the Update function.

```

uint32 Propmodel::Update()
{
    // Animation sequence
    switch(m_nAnimStep)
    {
        case 0:
            PlayAnimation("LStR");
            ++m_nAnimStep;
            break;

        case 1:
            PlayAnimation("LStL");
            ++m_nAnimStep;
            break;

        case 2:
            PlayAnimation("LRF");
            ++m_nAnimStep;
            // Let's run for 3 seconds instead of 2
    }
}

```

```

        g_pLTServer->SetNextUpdate(m_hObject, 3.0f);
        return 1;
        break;
        case 3:
            PlayAnimation("LSt");
            ++m_nAnimStep;
            break;

        case 4:
        {
            PlayAnimation("LSt");
            g_pLTServer->RemoveAttachment(m_hAttachment);
            g_pLTServer->RemoveObject(m_hCrossbow);
            CreateAttachmentObject(m_hBillyclub, "Models\\billyclub.ltb", "ModelTextures\\club.dtx");
            LTVector vRotOffset(25.0f, 25.0f, 0.0f);
            LTVector vPosOffset(0.0f, 0.0f, 0.0f);
            CreateAttachment(m_hAttachment, m_hBillyclub, "RightHand", vRotOffset, vPosOffset);
            ++m_nAnimStep;
        }
        break;

        case 5:
        {
            PlayAnimation("LC");
            g_pLTServer->SetNextUpdate(m_hObject, 2.0f);
            ++m_nAnimStep;
            return 1;
        }
        break;
        default:
        {
            m_nAnimStep = 0;
            PlayAnimation("LSt");
            g_pLTServer->RemoveAttachment(m_hAttachment);
            g_pLTServer->RemoveObject(m_hBillyclub);
            CreateAttachmentObject(m_hCrossbow, "Models\\crossbow.ltb", "ModelTextures\\crossbow.dtx");
            LTVector vRotOffset(25.0f, 25.0f, 0.0f);
            LTVector vPosOffset(0.0f, 0.0f, 0.0f);
            CreateAttachment(m_hAttachment, m_hCrossbow, "RightHand", vRotOffset, vPosOffset);
        }
        break;
    }
    g_pLTServer->SetNextUpdate(m_hObject, 2.0f);
    return 1;
}

```

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Programming Guide



Multiplayer Flying Camera Tutorial

The Multiplayer Flying Camera Tutorial adds multiplayer capabilities to the [samplebase](#) sample. This tutorial describes how to add multiplayer functionality to the the sample base, and serves as an example of how you can add multiplayer functionality to your own game.

You can find the flying camera sample code in your Jupiter installation under:

```
\\samples\tutorials\mpflycam
```

The `run.bat` file in that directory can be opened and runs the sample.

When the sample begins, you need to bring down the console by pressing the ~ (tilde) key. Then you should type normal, serve, or join and hit enter.

Typing `normal` creates a regular, single-player game.

Typing `server` creates a multiplayer server that other clients can attach to.

Type `join <IP address>` (where `<IP address>` is the IP of a computer that is running the sample in server mode) to join a server game. The IP should be in the form 127.0.0.1.

Client Shell

We will begin this tutorial looking at the changes to the `clientshell`. There are several changes in `\\cshell\\src\\ltclientshell.h` and `.cpp`. The two major changes are in the way the game is created and in the data that is sent to the server.

The **InitGame** method now takes a few parameters to specify the type of game you are creating, and an optional IP address to connect to.

There are also several functions near the top of `ltclientshell.cpp`. These functions are console programs. They are registered with the engine as console programs in **OnEngineInitialized**. Once they are registered, they will be called whenever the user brings down the console and types in the program name. This is how the game type is set for this sample. In a full game, you should build a UI that will take care of these choices.

The second major change to the client is that we are now using velocity to move the player instead of simply setting a new position. This creates a much smoother networked game because the engine can use the velocity to predict the

future location of an object.

Some other changes include the setting of the client object to non-visible and a new method called **SetService**. This method selects the TCP/IP service as the intended service for networking.

Server Shell

There are only a few changes to the Server Shell.

First, in the **OnClientEnterWorld** method, the object created is a model. This model represents the player and is visible on all clients (except the controlling client has overridden this). The model's filename and skinname (texture) are set appropriately. Also, after creation the model is set to play its flying animation.

The other change to the Server Shell is to read the data from the client as a velocity update instead of a position update. This is a simple change that only affects the **OnMessage** method.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Programming Guide



Simple Physics Tutorial

The Simple Physics tutorial is an extension of the foundation created by [MPFlyCam](#). In this tutorial we look at some of the problems and solutions that arise with physics in a multiplayer environment. We introduce several concepts such as gravity and stair stepping. This tutorial can be found in the following directory:

- \Samples\tutorials\simplephys

As with MPFlyCam, when you startup SimplePhys, the only thing visible is text from the console. You need to specify the game type that you want to start. The options are listed below:

- **Normal**—normal single player game. type: **normal**
- **Server**—start this as a multiplayer server. This is NOT a standalone server. type: **server**
 - **Join**—connect to a server. type: **join <ip.ip.ip>** or **join <computername>**

Common Problems

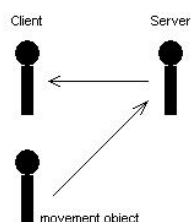
A few typical problems that surface with designing multiplayer games is dealing with slow packet speeds, poor connection quality, and latency. While there is no way to "solve" these problems, as network routing is out of the control of the programmer. You want to make these problems transparent to the player. Players want fluid motion on objects in the gaming world, including other players. Players also demand fast response time. When they press "left", they want to go left **now**, not in 100 milliseconds.

Fluid Motion

You can solve fluid motion by using the velocity of the object to "predict" where it should be the next frame. To use prediction, set a velocity on all objects you wish to have the clients "predict" and you specify "prediction" "1" in the autoexec.cfg. This tells the engine to turn prediction on. You must have prediction enabled for all multiplayer games.

Input Response Time

Input response time is a slightly less trivial matter. In the LithTech client/server architecture, each client has a client-side and a server-side, client object. When you "move" the server-side client object it automatically moves the client object on the client-side. So, if you were to move the client-side client object, then send the moves to the server, the server would then send the moves back to the client. This results in moving the client object 3 times, and will certainly be disruptive to the fluid input response. To get around this you can use the client object as an object seen only by other players. Using this technique a player's client object is invisible and they never see it. When the player moves he is actually moving another object created on the client-side only. The moves are then sent to the server, which get propagated to all of the other connected players able to see this player. In the case of a 3rd person camera, the movement object is visible. This is the case of the SimplePhys tutorial. The image below displays a graphical representation of the process.



Physics

This section provides a few notes about gravity and stair-stepping physics in the SimplePhys tutorial.

Gravity

There are a few important pieces needed in order to implement gravity using the Jupiter Engine. First you must set the actual amount of gravity you want to affect your player object.

```
ltclientshell.cpp
#define DEFAULT_GRAVITY    -2000.0f
...
void CLTClientShell::Update()
{
    ...
    LTVector vNewGlobalForce(0, DEFAULT_GRAVITY, 0);
    g_pLTCPHysics->SetGlobalForce(vNewGlobalForce);
    ...
}
```

Next you must manually update the physics system to take gravity into account. The **ILTClientPhysics::UpdateMovement** call does this for us.

Note: You must call either **SetVelocity** or **SetAcceleration** for the object prior to calling **UpdateMovement**. These two functions set a variable that tells the engine it is okay to affect this object's position.

```
playerclnt.cpp
void CPlayerClnt::RecalculatePosition()
{
    ILTClientPhysics* pCPhysics = (ILTClientPhysics*)g_pLTCPHysics;
    MoveInfo info;
    info.m_Offset.Init();
    info.m_hObject = m_hObject;
    info.m_dt = g_pLTClient->GetFrameTime();
    pCPhysics->UpdateMovement(&info);
    ...
}
```

Stair Stepping

On the object you must specify the **FLAG_GRAVITY** flag.

```
playerclnt.cpp
void CPlayerClnt::CreatePlayer()
{
    ...
    objCreateStruct.m_Flags = FLAG_SOLID | FLAG_GRAVITY | FLAG_STAIRSTEP | FLAG_VISIBLE;
    ...
}
```

Next, you must set the stair stepping height. This is the maximum increment at which the object is able to step up.

```
ltclientshell.cpp
void CLTClientShell::Update()
{
    ...
    if( m_bFirstUpdate )
    {
        ...
        g_pLTCPHysics->SetStairHeight(16.0f);
        ...
    }
    ...
}
```

If you have any questions or comments, please send them to support@touchdownentertainment.com

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Programming Guide



Creating Pickups Tutorial

The Pickups tutorial is an extension of the foundation created by SimplePhys. In this tutorial we add a common game object known as a pickup. This object is generally used for powerup items such as health or armor upgrades. The code for the Pickups tutorial is located in the following directory:

\\Samples\tutorials\pickups.

Most games have interactive game objects with which when touched give you a bigger and better weapon, or increase the amount of armor your character is currently wearing. This is referred to as a pickup or powerup. The normal actions for these objects is to disappear when touched. Pickups can "re-spawn" or "re-appear" after a certain amount of time, and they can play sounds when the pickup is touched to let players know they picked something up.

This section contains the following topics related to creating components to implement pickups:

- [Starting the Tutorial](#)
- [Touch Notify Messages](#)
- [Simple Sound Effects](#)
 - [Object Messages](#)
- [The PF_STATICLIST Property](#)
 - [Prehook Plugins](#)

Starting the Tutorial

As with SimplePhys, when you startup Pickups, the only thing visible is text from the console. You need to specify the game type that you want to start. Select from one of the following game types:

- **Normal**—normal single player game. type: normal
- **Server**—start this as a multiplayer server. This is NOT a standalone server. type: server
 - **Join**—connect to a server. type: join <ip.ip.ip> or join <computername>

[Top](#)

Touch Notify Messages

Touch Notify messages are sent to an object when a collision between itself and another object has occurred. The engine does not pass this message to the object by default.

You will notice in the **PreCreate** method of pickup.cpp that the **FLAG_TOUCH_NOTIFY** flag is specified.

```
pStruct->m_Flags = FLAG_VISIBLE | FLAG_TOUCH_NOTIFY;
```

If **m_Flags** is set, then the engine sends then **MID_TOUCHNOTIFY** messages if the object is touched. The following BaseClass function, **EngineMessageFn**, then gets called.

```
uint32 Pickup::EngineMessageFn(uint32 messageID, void *pData, float fData)
{
    switch (messageID)
    {
        case MID_PRECREATE:
            return PreCreate(pData, fData);
        case MID_TOUCHNOTIFY:
            return TouchNotify(pData, fData);
        case MID_UPDATE:
            return Update();

        default:
            break;
    }

    // Pass the message along to parent class.
    return BaseClass::EngineMessageFn(messageID, pData, fData);
}
```

[Top](#)

Simple Sound Effects

This topic explains how to play a simple sound associated with grabbing a pickup.

The first step in playing a sound is including the header file for the sound manager interface.

```
#include <iltsoundmgr.h>
```

Next create a **PlaySoundInfo** structure and fill it with the proper information.

```
void Pickup::PlaySound()
{
    PlaySoundInfo playSoundInfo;
    PLAYSOUNDINFO_INIT(playSoundInfo);
    HLTSOUND hSnd = LTNUL;

    playSoundInfo.m_dwFlags = PLAYSOUND_ATTACHED | PLAYSOUND_3D;
    playSoundInfo.m_hObject = m_hObject;
    playSoundInfo.m_fInnerRadius = 10.0f;
    playSoundInfo.m_fOuterRadius = 250.0f;
    playSoundInfo.m_nVolume = 100;

    SAFE_STRCPY(playSoundInfo.m_szSoundName, "Snd/pickup.wav");

    LTRESULT hResult = g_pLTServer->SoundMgr()->PlaySound(&playSoundInfo, hSnd);

    if(hResult != LT_OK)
    {
        g_pLTServer->CPrint("(PickUp) Error playing sound");
    }
}
```

In the previous syntax, we are telling the sound that we want it played in 3D space and attached to an object. If the object were moving, the sound would follow the object. We also specify the volume and distance for the sound.

[Top](#)

Object Messages

There are many situations within a game, that you might want the objects to pass information to other objects. One option is to get a handle to the object and get the instance of the class to which it is a member. But there are times where you want to pass information to a generic object without wanting to have direct access to the object's members. To do this, you can pass messages to other objects using the following method:

```
// Send this message to the player object
ILTMessage_Write *pMsg;
g_pLTSCCommon->CreateMessage(pMsg);
```



```

pMsg->Writeuint32(OBJ_MID_PICKUP);
pMsg->Writeuint8(m_nPickupType);
pMsg->Writefloat(m_fPickupValue);
g_pLTServer->SendToObject(pMsg->Read(), m_hObject, hObj, 0);

```

As you may notice, you need to create a message in the same manner that you create a message for any other purpose. You also need an **HOBJECT** handle to the object which is sending the message, and one to the object to which you wish to send the message.

You will also notice above that the **OBJ_MID_PICKUP** is getting pushed onto the message.

```

(in shared/msgids.h)
enum EObjMessageID
{
    OBJ_MID_PICKUP = 0
};

```

This is a value that we have defined in game code which gets referenced by the target object to determine if it has a way to deal with the incoming message. This is done below.

```

uint32 CPlayerSrvr::ObjectMessageFn(HOBJECT hSender, ILTMessage_Read *pMsg)
{
    pMsg->SeekTo(0);
    uint32 messageID = pMsg->Readuint32();
    switch(messageID)
    {
        case OBJ_MID_PICKUP:
        {
            uint8 nPickupType = pMsg->Readuint8();
            float fPickupValue = pMsg->Readfloat();
            g_pLTServer->CPrint("PlayerSrvr -> OBJ_MID_PICKUP %d %f", nPickupType, fPickupValue);
        }
        break;
        default:
        break;
    }
    return BaseClass::ObjectMessageFn(hSender, pMsg);
}

```

[Top](#)

The PF_STATICLIST Property

The **PF_STATICLIST** property is a flag which tells DEdit to create a "dropdown list" component for this object property.

```
ADD_STRINGPROP_FLAG(PickupType, "Health", PF_STATICLIST)
```

In the case of our pickup, you may want to support several types such as Health and Armor pickups. Using a static list will allow the level designers to easily change between our predefined pickup types.

[Top](#)

Prehook Plugins

DEdit has several very helpful features, among them is the ability to dynamically change lists at DEdit's runtime. You may want to do this if you are planning to store the pickup types in text file instead of hard coding them. This gives you a "hook" to read these values and fill the proper information.

The first part to this is setting up the macros that DEdit uses to find information about the objects. You will notice the **_PLUGIN** suffix to the standard end class macro. This allows you to specify a plugin class for DEdit to use. In the following example this is **CPickupPlugin**.

```
END_CLASS_DEFAULT_FLAGS_PLUGIN(PickUp, BaseClass, LTNULL, LTNULL, CF_ALWAYSLOAD, CPickupPlugin)
```

The plugin is derived from the **IObjectPlugin** class. To use **IObjectPlugin** class, you must first include the definition by including **iobjectplugin.h**. In the class **CPickupPlugin**, overwrite **PreHook_EditStringList**. **PreHook_EditStringList** gets called by DEdit when the object is read in from the world file. This gives you the chance to modify the contents of an object property.

```

#include <iobjectplugin.h>
class CPickupPlugin : public IObjectPlugin
{
public:
    virtual LTRESULT PreHook_EditStringList(
        const char* szRezPath,
        const char* szPropName,
        char** aszStrings,
        uint32* pcStrings,
        const uint32 cMaxStrings,
        const uint32 cMaxStringLength);
};

```

The following implementation looks for the **PickupType** property. If it finds an incoming property with that name, then it adds our types to the list. We have setup **g_aszPickupTypes** in **pickup.h** with the desired types.

```

LTRESULT CPickupPlugin::PreHook_EditStringList(
    const char* szRezPath,
    const char* szPropName,
    char** aszStrings,
    uint32* pcStrings,
    const uint32 cMaxStrings,
    const uint32 cMaxStringLength)
{
    // If prop name is "PickupType", fill in the dropdown with g_aszPickupTypes[]
    if (strcmp("PickupType", szPropName) == 0)
    {

```

```

        for (int i=0; i < IPT_NUM_TYPES; i++)
        {
            if (cMaxStrings > (*pcStrings) + 1)    // sanity check
            {
                if (g_aszPickupTypes[i][0])    // skip null strings
                    strcpy(aszStrings[(*pcStrings)++], g_aszPickupTypes[i]);
            }
        }
        return LT_OK;
    }
    return LT_UNSUPPORTED;
}

```

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Programming Guide



Creating Projectiles Tutorial

The Creating Projectiles Tutorial is an extension of the foundation created by the [Creating Pickups Tutorial](#). In most action oriented games players are able to throw or shoot objects. These are commonly referred to as, projectiles. In this tutorial we walk through, how to add this functionality to the game. This code for the Creating Projectiles tutorial is located in the following directory:

\\Samples\\tutorials\\projectiles.

It is very common for action game developers to want players to be able to throw or shoot objects from the origin of other objects. These are usually called projectiles and typically cause damage to other objects and players (though this may not always be the case). This section explains one possible way to implement a Projectile game object. This requires you to make changes to input handling on the client as well as add a brand new game object class to the server.

This section contains the following projectile creation topics:

- [Starting the Tutorial](#)
- [Client-side Changes](#)
- [Server-side Changes](#)
- o [Implementing the Projectile Class](#)

Starting the Tutorial

As with Pickups, when you startup Projectiles, the only thing visible is text from the console. You need to specify the game type that you want to start. The options are listed below:

- **Normal**—normal single player game. type: normal
- **Server**—start this as a multiplayer server. This is NOT a standalone server. type: server
 - **Join**—connect to a server. type: join <ip.ip.ip> or join <computername>

Client-side Changes

This section lists changes to the client side code.

Autoexec.cfg

To create a projectile, you must modify the autoexec.cfg file to add a new key binding for "shoot". Add the following entries to the autoexec.cfg file:

```

AddAction Shoot 10
rangebind "##mouse" "Button 0" 0.000000 0.000000 "Shoot"

```

Pay close attention to the location of these entries in the file. The "Action" must be added before you attempt to bind a key/button to that action.

Commandids.h

The command IDs file (commandids.h) contains a list that corresponds to the AddAction list in the autoexec.cfg.

```

enum ECommands
{
    ...
    COMMAND_SHOOT    = 10,
    ...
}

```

Notice that both **COMMAND_SHOOT = 10**, and "AddAction Shoot" in the autoexec.cfg file are set to 10. This is because the **COMMAND_SHOOT** property gets used when polling input.

Itclclientshell.cpp

Listen for a command to execute a "shoot" operation in: **CLTClientShell::OnCommandOn**. Use the **OnCommandOn** method for commands that you only want to get each time the key/button is pressed, but not every frame in which the key is down, as is the case with **IsCommandOn**.

```

void CLTClientShell::OnCommandOn(int command)

```

```

        {
            switch (command)
            {
                ...
                case COMMAND_SHOOT:
                {
                    g_pLTClient->CPrint("CLIENT SHOOT!");
                    ILTMessage_Write *pMessage;
                    LTRESULT nResult = g_pLTCommon->CreateMessage(pMessage);

                    if( LT_OK == nResult)
                    {
                        pMessage->IncRef();
                        pMessage->Writeuint8(MSG_CS_SHOOT);
                        g_pLTClient->SendToServer(pMessage->Read(), 0);
                        pMessage->DecRef();
                    }
                    break;
                    ...
                }
            }
        }

```

You may notice that this code sends a message to the server instead of creating the projectile on the client. This enables all players in the game to see and react to the projectile.

msgids.h

In the **OnCommandOn** example above you are passing **MSG_CS_SHOOT** into the message that you send to the server. You must define **MSG_CS_SHOOT** in the msgids.h shared file. Both the client and server use the **MSG_CS_SHOOT** identifier.

```

enum EMessageID
{
    ...
    MSG_CS_SHOOT, // client->server
    ...
};

```

This concludes the changes and additions to the client-side. The following section explains what must take place on the server.

[Top](#)

Server-side Changes

The sections lists changes to the server side code.

ltservershell.cpp

First you must catch the messages sent to the server. This is done using the **CLTServerShell::OnMessage** method.

```

void CLTServerShell::OnMessage(HCLIENT hSender, ILTMessage_Read *pMessage)
{
    BaseClass *pBaseClass = (BaseClass *)g_pLTServer->GetClientUserData(hSender);
    HOBJECT hPlayer = g_pLTServer->ObjectToHandle(pBaseClass);

    pMessage->SeekTo(0);
    uint8 messageID = pMessage->Readuint8();

    switch(messageID)
    {
        ...
        case MSG_CS_SHOOT:
        {
            LTVector vPos;
            LTRotation rRot;

            g_pLTServer->GetObjectPos(hPlayer, &vPos);
            g_pLTServer->GetObjectRotation(hPlayer, &rRot);

            // Create Projectile
            HCLASS hClass = g_pLTServer->GetClass("Projectile");

            ObjectCreateStruct ocs;
            ocs.m_ObjectType = OT_MODEL;
            ocs.m_Flags = FLAG_VISIBLE;
            ocs.m_Rotation = rRot;
            vPos += rRot.Forward() * 25.f;
            ocs.m_Pos = vPos;

            strncpy(ocs.m_Filename, "Models/GenCan.ltb", MAX_CS_FILENAME_LEN - 1);
            strncpy(ocs.m_SkinName, "ModelTextures/GenCan1.dtx", MAX_CS_FILENAME_LEN - 1);
            Projectile* pObject = (Projectile*)g_pLTServer->CreateObject(hClass, &ocs);

            if(pObject)
            {
                g_pLTServer->CPrint("Shoot!!!");
                pObject->SetOwner(hPlayer);
            }
            break;
            ...
        }
    }
}

```

Notice that **MSG_CS_SHOOT** is being handled.

[Top](#)

Implementing the Projectile Class

In the previous section you created an instance of the **Projectile** class, and placed it in your game world. The following topics explain how to implement the **Projectile** class, orient the projectile with the player, and place the projectile a few units in front of the player.

projectile.cpp

You must create the **Projectile** class in projectile.cpp/h. The Projectile class is a new game code class created for this tutorial.

```
BEGIN_CLASS(Projectile)
END_CLASS_DEFAULT_FLAGS(Projectile, BaseClass, LTNUL, LTNUL, CF_ALWAYSLOAD | CF_HIDDEN)
```

You must use the **CF_HIDDEN** class flag on Projectile. **CF_HIDDEN** prevents the projectile object from appearing in DEdit as a placable object. This prevents Level Designers from placing Projectile objects in the level by mistake.

```
uint32 Projectile::InitialUpdate(void *pData, float fData)
{
    ...
    // Don't ignore small collisions
    g_pLTSPHysics->SetForceIgnoreLimit( m_hObject , 0.0f );
    ...
}
```

One important thing to note is the use of the **SetForceIgnoreLimit** method. By default, objects are setup to ignore very small collisions. So, when the Projectile hits a wall instead of detecting a collision, it slides against the wall. To prevent projectiles from sliding against polygons you want to remove the projectile from the world. In order to remove a projectile, you must tell the physics interface not to ignore any collisions.

```
uint32 Projectile::Update(void *pData, float fData)
{
    LTVector vVel;
    LTRotation rRot;

    g_pLTServer->GetObjectRotation(m_hObject, &rRot);
    vVel = rRot.Forward() * 300.0f;
    g_pLTSPHysics->SetVelocity(m_hObject, &vVel);
    g_pLTServer->SetNextUpdate(m_hObject, 0.1f);
    return 1;
}
```

Now that you have placed the Projectile in the world, you must move the projectile. To move the projectile you must set its velocity. You could simply call the **MoveObject**, or the **SetObjectPos** method to move the projectile, but the **SetVelocity** method allows the clients to perform client-side prediction for this object. Because of this, you should use the **SetVelocity** method for moving objects whenever possible.

Since your object moves in a straight line, you only need to set the velocity once. The following example sets the velocity 10 times per second. If your Projectile has a more complicated flight path, you will want to update the velocity frequently.

```
uint32 Projectile::TouchNotify(void *pData, float fData)
{
    HOBJECT hObj = (HOBJECT)pData;

    CollisionInfo Info;
    memset(&Info, 0, sizeof(Info));
    g_pLTServer->GetLastCollision(&Info);

    LTRESULT result = g_pLTSPHysics->IsWorldObject( hObj );

    if (result == LT_YES)
    {
        g_pLTServer->CPrint( "Projectile::I hit the world at poly %d", Info.m_hPoly );
    }
    else
    {
        if(hObj != m_hOwner)
        {
            g_pLTServer->CPrint( "Projectile::I hit an object %p", hObj );
        }
        if(hObj != m_hOwner)
        {
            g_pLTServer->RemoveObject(m_hObject);
            return 1;
        }
    }
}
```

The last piece of important information in about your Projectile class is the handling of Touch Notify events. When an object gets a **MID_TOUCHNOTIFY** engine message it passes a void pointer "pData," which contains the **HOBJECT** of the object the projectile collided with. Typically you will want to find out whether the target object your projectile connected with is another game object or a polygon on the world, (such a a wall or the floor).

To determine if the object is a world polygon, call **LTRESULT LTPhysics::IsWorldObject(HOBJECT)**. It returns **LT_YES** if the object is a world polygon or **LT_NO** if otherwise. You can then make logical decisions based on the result.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Programming Guide



—Tutorial— Creating Animations

The Creating Animations tutorial is an extension of the foundation created by Projectiles. This tutorial focuses on how to play simple model animations, and how to do so in a multiplayer environment. More specifically this section covers playing simple animations for movement event such as walking or standing idle. This tutorial exists in the following directory:

\Samples\tutorials\animations.

This section contains the following simple animation topics:

- [Starting the Tutorial](#)
- [Client-side Changes](#)
- [Server-side Changes](#)

Starting the Tutorial

As with Projectiles, when you startup Animations, the only thing visible is text from the console. You need to specify the game type that you want to start. The options are listed below:

- **Normal**—normal single player game. type: normal
- **Server**—start this as a multiplayer server. This is NOT a standalone server. type: server
 - **Join**—connect to a server. type: join <ip.ip.ip.ip> or join <computername>

Client-side Changes

PlayerClnt.cpp

The first step you must take is to use a model which has the animation set that you want to use. For this task the Animations tutorial uses the Cate Archer player model from No One Lives Forever 2.

```
void CPlayerClnt::CreatePlayer()
{
    ...
    strcpy(objCreateStruct.m_Filename, "Models\\player_action.ltb");
    strcpy(objCreateStruct.m_SkinNames[0], "ModelTextures\\CateAction.dtx");
    strcpy(objCreateStruct.m_SkinNames[1], "ModelTextures\\CateActionHead.dtx");
    strcpy(objCreateStruct.m_SkinNames[2], "ModelTextures\\CateActionHair.dtx");
    strcpy(objCreateStruct.m_SkinNames[3], "ModelTextures\\CateActionLash.dtx");
    ...
}
```

For this tutorial, you want to have the model play the appropriate animation in response to your player's movement. So, when the player moves forward, you want to play a "walking forward" animation. When your player move backwards, you want to play a "walking backwards" animation, and so forth.

With that in mind, a general purpose "PlayAnimation" function has been created to call when you want the animation to change. Notice that all you need to do is pass in the string name of the animation to play. A member variable has been added to keep track of the current animation. (m_hCurAnim) This variable gets used to prevent calling the same animation two frames in a row. If you set the same animation every frame, you would only see the first frame of that animation. To prevent that from happening, a check exists to ignore redundant animation requests. For example, if the "walk forward" animation is playing, if the player continues to move forward, then the animation gets set once and continues to loop. If however, the player starts moving backwards, then the requested animation and the current animation becomes different, at which point the animation is allowed to change.

```
void CPlayerClnt::PlayAnimation(const char* sAnimName)
{
    HMODELANIM hAnim = g_pLTClient->GetAnimIndex(m_hObject, (char *)sAnimName);
    if (hAnim != m_hCurAnim)
    {
        m_hCurAnim = hAnim;
        g_pLTClient->SetModelAnimation(m_hObject, hAnim);
        g_pLTClient->SetModelLooping(m_hObject, LTRUE);
        ...
    }
}
```

Now that your PlayAnimation() functionality has been added, you need to hook it in to your movement code. Notice that you only need to add one line to each movement code. For your animation name, pass in "LRF", which is the same name as used in ModelEdit for the walk forward animation. Add a call to PlayAnimation with the appropriate animation for each movement code, forward, backwards, left, right, and idle.

```
void CPlayerClnt::UpdateMovement()
{
    ...
    // Look at our input flags
    if ( m_dwInputFlags & MOVE_FORWARD)
    {
        vVel += rRot.Forward() * MOVEMENT_RATE;
        PlayAnimation("LRF");
    }
    ...
}
```

There is one more consideration to be made, and that is the multiplayer aspect. The game server needs to be informed of your choice of currently playing model animations, so that it can send that information to each of the connected clients. This information is not automatically propagated to the server, so you need to write a small amount of client-side code to handle this.

To create this code, first create a message object, and set the message type. MSG_CS_ANIM is defined in msgids.h which is covered in the next section. Next you must write the animation string name to the message. Finally, you send the message to the server.

```
void CPlayerClnt::PlayAnimation(const char* sAnimName)
{
    ...
    if (hAnim != m_hCurAnim)
    {
        ...
    }
}
```

```

// Send animation to server
ILTMessage_Write *pMessage;
LTRESULT nResult = g_pLTCommon->CreateMessage(pMessage);

if( LT_OK == nResult)
{
    pMessage->IncRef();
    pMessage->Writeuint8(MSG_CS_ANIM);
    pMessage->WriteString(sAnimName);
    g_pLTClient->SendToServer(pMessage->Read(), 0);
    pMessage->DecRef();
}
...
}

```

msgids.h

You must add one more additional message type to your enumeration. This allows both the client and the server to deal with your custom animation change messages.

```

enum EMessageID
{
    ...
    MSG_CS_ANIM, // client->server
    ...
};

```

Server-side Changes

ltservershell.cpp

You only need to make two changes to your servershell in order to allow other players to see your client-set animations. First you need to change the loaded model to match the model that is loaded on the client-side.

```

LPBASECLASS CLTServerShell::OnClientEnterWorld(HCLIENT hClient, void *pClientData, uint32 nClientDataLen)
{
    ...
    strcpy(objCreateStruct.m_Filename, "Models\\player_action.ltb");
    strcpy(objCreateStruct.m_SkinNames[0], "ModelTextures\\CateAction.dtx");
    strcpy(objCreateStruct.m_SkinNames[1], "ModelTextures\\CateActionHead.dtx");
    strcpy(objCreateStruct.m_SkinNames[2], "ModelTextures\\CateActionHair.dtx");
    strcpy(objCreateStruct.m_SkinNames[3], "ModelTextures\\CateActionLash.dtx");
    ...
}

```

The second change is to handle the MSG_CS_ANIM message sent from the client.

```

void CLTServerShell::OnMessage(HCLIENT hSender, ILTMessage_Read *pMessage)
{
    ...
    case MSG_CS_ANIM:
    {
        char sAnimName[128];
        pMessage->ReadString(sAnimName, 128);
        HMODELANIM hAnim = g_pLTServer->GetAnimIndex(hPlayer, (char *)sAnimName);
        g_pLTServer->SetModelAnimation(hPlayer, hAnim);
        g_pLTServer->SetModelLooping(hPlayer, LTTRUE);
    }
    break;
    ...
}

```

This is all that is required to implement the use of simple animations with multiplayer considerations using the LithTech Jupiter System.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Programming Guide



—Tutorial— Animations 2

The Animations2 tutorial is an extension of the foundation created by Animations. This tutorial describes how to play more complex model animations in a multiplayer environment. More specifically this article covers playing animations using separate animation trackers for upper and lower body. This allows the ability to play an attack animation while playing walking/running animations. This tutorial can be found in the following directory:

- \Samples\tutorials\animations2.

This section contains the following simple animation topics:

- [Starting the Tutorial](#)
- [Client-side Changes](#)
- [Server-side Changes](#)

Starting the Tutorial

As with Projectiles, when you startup Animations, the only thing visible is text from the console. You must specify the game type that you want to start. Enter one of the following options into the console:

- **Normal**—normal single player game. type: normal
- **Server**—start this as a multiplayer server. This is NOT a standalone server. type: server
- **Join**—connect to a server. type: join <ip.ip.ip> or join <computername>

Client-side Changes

PlayerCnt

The first step in adding support for animation blending is setting up the animation trackers and setting the weight sets. In this example you are setting up upper and lower body animation trackers. However, you are not limited to lower and upper body animation trackers. For example, you could add trackers for animations such as a character's eyes blinking, which is done in No One Lives Forever 2. For the purposes of this tutorial, only support for upper and lower body animations is being added. The following code shows how to add this support.

PlayerCnt.h

You must first create the member variables for keeping track of your trackers and weight sets.

```
class CPlayerCnt
{
    ...
private:
    ...
    ANIMTRACKERID m_idUpperBodyTracker;
    ANIMTRACKERID m_idLowerBodyTracker;
    HMODELWEIGHTSET m_hWeightUpper;
    HMODELWEIGHTSET m_hWeightLower;
    ...
}
```

PlayerCnt.cpp

You now need to set and use the member variables created in the previous header.

```
void CPlayerCnt::CreatePlayer()
{
    ...
    if(m_hObject)
    {
        // Set up animation trackers
        m_idUpperBodyTracker = ANIM_UPPER;
        g_pLTCModel->AddTracker(m_hObject, m_idUpperBodyTracker);
        m_idLowerBodyTracker = ANIM_LOWER;
        g_pLTCModel->AddTracker(m_hObject, m_idLowerBodyTracker);
        // Set up weight sets
        if ( LT_OK == g_pLTCModel->FindWeightSet(m_hObject, "Upper", m_hWeightUpper) )
        {
            g_pLTCModel->SetWeightSet(m_hObject, m_idUpperBodyTracker, m_hWeightUpper);
        }
        if ( LT_OK == g_pLTCModel->FindWeightSet(m_hObject, "Lower", m_hWeightLower) )
        {
            g_pLTCModel->SetWeightSet(m_hObject, m_idLowerBodyTracker, m_hWeightLower);
        }
    }
}
```

shared\animids.h

In the previous code, you use the variables **ANIM_UPPER** and **ANIM_LOWER** when you create your animation trackers. For convenience, these variables are defined in animids.h (as shown in the following code snippet).

```
enum
{
    ANIM_NONE = 0,
    ANIM_UPPER = 1,
    ANIM_LOWER = 2
};
```

Note: **ANIMTRACKERIDs** are defined as type **uint8** or unsigned char. This ID number gets used when dealing with animation trackers. There is a global animation tracker define by the engine, called **MAIN_TRACKER**. The ID number for **MAIN_TRACKER** is defined as 255 (0xFF). The weightset for **MAIN_TRACKER** consists of all the nodes in the model. When **MAIN_TRACKER** gets passed in for any given animation, it gets applied to all nodes in the model.

In order to support multiple animations, you must modify your **PlayAnimation** function to handle multiple animation trackers. Just as you have done previously, in **PlayAnimation** you must check to see if the animation you want to play is already playing. If so, you do not set the animation again. The following code has been modified to include support for using multiple trackers.

PlayerCnt.h

Instead of keeping track of the current animation as a member variable, check the current animation on a per/tracker basis when **PlayAnimation** is called. This gives you maximum flexibility.

```
void PlayAnimation(const char* sAnimName, uint8 nTracker, bool bLoop = LTTRUE);
```

PlayerCnt.cpp

```

void CPlayerClnt::PlayAnimation(const char* sAnimName, uint8 nTracker, bool bLoop)
{
    HMODELANIM hAnim = g_pLTClient->GetAnimIndex(m_hObject, (char *)sAnimName);

    HMODELANIM hCurAnim;
    g_pLTModel->GetCurAnim(m_hObject, nTracker, hCurAnim);
    if(hAnim != hCurAnim)
    {
        g_pLTModel->SetCurAnim(m_hObject, nTracker, hAnim);
        g_pLTModel->SetLooping(m_hObject, nTracker, bLoop);
        // Send animation to server
        ILTMessage_Write *pMessage;
        LTRESULT nResult = g_pLTCommon->CreateMessage(pMessage);
        if( LT_OK == nResult)
        {
            pMessage->IncRef();
            pMessage->Writeuint8(MSG_CS_ANIM);
            pMessage->WriteString(sAnimName);
            pMessage->Writeuint8(nTracker);
            pMessage->Writebool(bLoop);
            g_pLTClient->SendToServer(pMessage->Read(), 0);
            pMessage->DecRef();
        }
    }
}

```

PlayerClnt.cpp

In addition to upgrading the movement code, the "attack" code has also been modified. Now when you click the left mouse button, your Cate player model plays a throwing animation. To make this new functionality work, add the following code.

```

bool CPlayerClnt::Attack()
{
    if(m_bAttacking)
    {
        return false;
    }
    // Play anims
    PlayAnimation("UMFi", m_idUpperBodyTracker, LTFALSE);
    m_bAttacking = true;
    return m_bAttacking;
}

```

On every update, check to see if the attack animation has completed. If it has, then reset **m_bAttacking** to false. This allows "attack" to get called again by **ltclientshell**.

```

void CPlayerClnt::UpdateAttacking(uint8 nTracker)
{
    HMODELANIM hCurAnim;
    g_pLTModel->GetCurAnim(m_hObject, nTracker, hCurAnim);
    uint32 nAnimLen, nAnimTime;
    g_pLTModel->GetCurAnimLength(m_hObject, nTracker, nAnimLen);
    g_pLTModel->GetCurAnimTime(m_hObject, nTracker, nAnimTime);
    if(nAnimTime >= nAnimLen)
    {
        PlayAnimation("Lst", m_idUpperBodyTracker);
        m_bAttacking = false;
    }
}

```

Server-side Changes

You need to update your server-side player model with the same changes as your client-side representation for two reasons. First you do so for multiplayer support, secondly you want to create your projectiles at a certain point in your "attack" animation. This is done using model string keys and is discussed in this section.

PlayerSrvr.cpp

The code for creating the animation trackers should look very familiar. The only new addition is that you are storing a handle to the socket on your model's "RightHand" socket. You need the socket handle in order to get its position and rotation use when you create your projectile.

```

uint32 CPlayerSrvr::InitialUpdate(void *pData, float fData)
{
    // Set up animation trackers
    m_idUpperBodyTracker = ANIM_UPPER;
    g_pLTModel->AddTracker(m_hObject, m_idUpperBodyTracker);

    m_idLowerBodyTracker = ANIM_LOWER;
    g_pLTModel->AddTracker(m_hObject, m_idLowerBodyTracker);

    // Set up weight sets

    if ( LT_OK == g_pLTModel->FindWeightSet(m_hObject, "Upper", m_hWeightUpper) )
    {
        g_pLTModel->SetWeightSet(m_hObject, m_idUpperBodyTracker, m_hWeightUpper);
    }

    if ( LT_OK == g_pLTModel->FindWeightSet(m_hObject, "Lower", m_hWeightLower) )
    {
        g_pLTModel->SetWeightSet(m_hObject, m_idLowerBodyTracker, m_hWeightLower);
    }

    // Get hand socket
    LTRESULT SocketResult = g_pLTModel->GetSocket(m_hObject, "RightHand", hRightHandSocket);
    if(LT_OK == SocketResult)
    {
        g_pLTServer->CPrint("Got the RightHand socket");
    }

    return 1;
}

```


You also need to update your **PlayAnimation** function to support multiple animation trackers.

```
void CPlayerSrvr::PlayAnimation(const char* sAnimName, uint8 nTracker, bool bLooping)
{
    HMODELANIM hAnim = g_pLTServer->GetAnimIndex(m_hObject, (char *)sAnimName);
    g_pLTModel->SetCurAnim(m_hObject, nTracker, hAnim);
    g_pLTModel->SetLooping(m_hObject, nTracker, bLooping);
}
```

In ModelEdit, you can setup model strings during certain keyframes. You can listen for these strings in game code. To do so, you need to enable the **FLAG_MODELKEYS** object flag.

```
uint32 CPlayerSrvr::PreCreate(void *pData, float fData)
{
    ...
    pStruct->m_Flags |= FLAG_MODELKEYS;
    ...
}
```

Now that you have told the engine you want to receive these events, you will receive a **MID_MODELSTRINGKEY** engine message when a model string key is encountered during the playback of an animation. The following code shows how to handle reception of the **MID_MODELSTRINGKEY** engine message:

```
uint32 CPlayerSrvr::EngineMessageFn(uint32 messageID, void *pData, float fData)
{
    ...
    case MID_MODELSTRINGKEY:
    {
        ArgList* pArgList = (ArgList*)pData;
        char szBuffer[256];
        sprintf(szBuffer, "");

        for ( int i = 0 ; i < pArgList->argc ; i++ )
        {
            if(strcmp("Throw", pArgList->argv[i]) == 0)
            {
                //create projectile
                CreateProjectile();
            }
            return 1;
        }break;
    }
    ...
}
```

The final change is in the **CreateProjectile** function. This is where you use the socket handle that you saved earlier. You need to get the position and rotation information that the socket is currently in. Use this information to place the projectile created in the following function.

```
void CPlayerSrvr::CreateProjectile()
{
    ...
    LTransform tSocketTransform;
    LTRESULT SocketTrResult = g_pLTModel->GetSocketTransform(m_hObject,
                                                            hRightHandSocket,
                                                            tSocketTransform,
                                                            LTTRUE);

    ocs.m_Pos = tSocketTransform.m_Pos;
    ...
}
```

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Programming Guide



—Tutorial— Creating Doors

This section shows how to create a simple door that opens when touched by the player or a projectile. It also allows placement of the door through the DEdit application. The Doors sample can be found in the following directory:

\Samples\tutorials\doors.

This section contains the following topics related to the creation of a simple door:

- [Starting the Tutorial](#)
- [Server-side Changes](#)

Starting the Tutorial

As with SpecialEffects1, when you startup the sample file for the Doors Tutorial, the only thing visible is text from the console. You need to specify the game type that you want to start. The options are listed below:

- **Normal**—normal single player game. type: normal
- **Server**—start this as a multiplayer server. This is NOT a standalone server. type: server
 - **Join**—connect to a server. type: join <ip.ip.ip.ip> or join <computername>

Server-side Changes

When creating a door using the techniques described in this tutorial, the only required change to your server-side game code is the addition of a "door" class. This object must be setup to support placement using DEdit. Your door opens when touched by the player, or a projectile.

Door.cpp

Just as with any DEdit-placable game object, you must setup the macro block with the proper information. You should notice the **CF_WORLDMODEL** flag in the closing macro. This lets DEdit and Processor know that this object should be treated as a worldmodel, and hence has geometry associated with it.

```
BEGIN_CLASS(Door)
ADD_REALPROP(MaxOpenAngle, 90.0f) // The angle offset when the door is fully open
ADD_VECTORPROP(HingeOffset) // Offset from the center of the door, to the rotation point of the door
ADD_REALPROP(UpdateRate, 0.0333f) // Time delay, in milliseconds between updates
ADD_REALPROP(DegreesPerUpdate, 2.5f) // How many degrees to rotate each update
END_CLASS_DEFAULT_FLAGS(Door, BaseClass, LTNULL, LTNULL, CF_ALWAYSLOAD | CF_WORLDMODEL)
```

The **PreCreate** function of your Door contains some important setup information. Most importantly, you must inform the engine that this object is a WorldModel by specifying the **OT_WORLDMODEL** object type flag. You must also set up the following special flags on your door object:

- Use **FLAG_GOTHRUWORLD** because, as your door rotates, it is very likely that it will clip the world slightly. So you want to disable door to world collisions.
 - Use **FLAG_TOUCH_NOTIFY** to allow the player to open the door by simply touching it.
 - Use **FLAG_FULLPOSITION** to ensure that the position information sent to the client is as accurate as possible.
- Use **FLAG2_DISABLEPREDICTION** to force the client to not assume the position of the door since you are performing special movement patterns.

The following code shows the implementation of the **PreCreate** method.

```
uint32 Door::PreCreate(void *pData, float fData)
{
    // Let parent class handle it first
    BaseClass::EngineMessageFn(MID_PRECREATE, pData, fData);

    // Cast pData to a ObjectCreateStruct* for convenience
    ObjectCreateStruct* pStruct = (ObjectCreateStruct*)pData;

    // Set the object type to OT_NORMAL
    pStruct->m_ObjectType = OT_WORLDMODEL;

    pStruct->m_Flags |= (FLAG_SOLID
                       | FLAG_VISIBLE
                       | FLAG_GOTHRUWORLD
                       | FLAG_TOUCH_NOTIFY
                       | FLAG_FULLPOSITIONRES);
    pStruct->m_Flags2 |= FLAG2_DISABLEPREDICTION;
    ...
}
```

The movement of your door is managed by the **Door::Update** method. Your door object has four states, Open, Closed, Opening, Closing. The following code shows the implementation of the **Update** method.

```
uint32 Door::Update()
{
    switch(m_nMode)
    {
        case DOOR_OPENING:
        {
            m_fOpenRot -= m_fDegreesPerUpdate;
            if((m_fOpenRot - m_fDegreesPerUpdate) < 0.0f)
            {
                // Place door in fully opened position
                OnMoveDone(DOOR_OPEN, 0.0f, m_rOpenRot);
                break;
            }

            // Continue opening door. If the door is touched during this,
            // then close the door
            OnMove(m_fDegreesPerUpdate, DOOR_CLOSING);
            break;
        }
        case DOOR_CLOSING:
        {
            m_fOpenRot += m_fDegreesPerUpdate;
            if((m_fOpenRot + m_fDegreesPerUpdate) > m_fMaxOpenAngle)
            {
                // Place door in fully closed position
                OnMoveDone(DOOR_CLOSED, m_fMaxOpenAngle, m_rClosedRot);
                break;
            }

            // Continue closing door. If the door is touched during this,
            // then open the door
            OnMove(-m_fDegreesPerUpdate, DOOR_OPENING);
            break;
        }
        case DOOR_OPEN:
        {
            m_nMode = DOOR_CLOSING;
            g_pLTServer->SetNextUpdate(m_hObject, m_fUpdateRate);
            break;
        }
        default:
        {
            g_pLTServer->SetNextUpdate(m_hObject, 0.0f);
            break;
        }
    }

    return 1;
}
```

When your door finishes a movement cycle (of either opening or closing completely) then call **Door::OnMoveDone** to set the final resting position of the door. The following code shows the implementation of the **OnMoveDone** method.

```
void Door::OnMoveDone(int nState, float fAngle, LTRotation &rMode)
{
    m_nMode = nState;
    m_fOpenRot = fAngle;
    LTRotation rRot;
    LTVector vPos;
    g_pLTServer->GetObjectRotation(m_hObject, &rRot);

    // Calculate our final position
    LTMatrix mRotation;
    rRot = m_hackInitialRot * rMode * ~m_hackInitialRot;
    rRot.ConvertToMatrix( mRotation );
    vPos = (mRotation * m_vHingeOffset) + (m_vClosedPos - m_vHingeOffset);

    // Set our position and rotation
    g_pLTServer->SetObjectPos(m_hObject, &vPos);
    g_pLTServer->SetObjectRotation(m_hObject, &rMode);
    g_pLTServer->SetNextUpdate(m_hObject, 1.0f);
}
```

While the door is in motion, whether opening or closing, call your **Door::OnMove** method. The following code shows the implementation of the **OnMove** method.

```
void Door::OnMove(float fDegrees, int nOnInterrupt)
{
    // Get the current rotation
    LTRotation rRot;
    g_pLTServer->GetObjectRotation(m_hObject, &rRot);

    // Set our new rotation
    rRot.Rotate(rRot.Up(), DegreesToRadians(fDegrees));
    g_pLTServer->RotateObject(m_hObject, &rRot);

    // Get our current position
    LTVector vPos;
    g_pLTServer->GetObjectPos(m_hObject, &vPos);
    // Save our current position.
    LTVector vOldPos;
    vOldPos = vPos;

    // Calculate the position to move to.
    LTMatrix mRotation;
    rRot = m_hackInitialRot * rRot * ~m_hackInitialRot;
    rRot.ConvertToMatrix( mRotation );
    vPos = (mRotation * m_vHingeOffset) + (m_vClosedPos - m_vHingeOffset);

    // Try to move the object
    g_pLTServer->MoveObject(m_hObject, &vPos);

    // Check to see if anything is in our way.
    // If it is, set the mode opposite of the current mode,
    // and reset the position.
    LTVector vCurPos;
    g_pLTServer->GetObjectPos( m_hObject, &vCurPos );
    if( !vCurPos.NearlyEquals( vPos, MATH_EPSILON ) )
    {
        m_nMode = nOnInterrupt;
        g_pLTServer->SetObjectPos(m_hObject, &vOldPos);
    }
    g_pLTServer->SetNextUpdate(m_hObject, m_fUpdateRate);
}
```

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Programming Guide



—Tutorial— Special Effects1

The Special Effects tutorial combines the effects of the [Animations2](#) tutorial and sample base - clientfx sample. We recommend that you read the documentation for the [sample base](#) and [clientfx](#) samples before working with this tutorial. The Special Effects sample code can be found in the following directory:

\\Samples\tutorials\specialeffects1.

This section contains the following simple specialeffects topics:

- [Starting the Tutorial](#)
- [Server-side Changes](#)

Starting the Tutorial

As with Animations2, when you startup the SpecialEffects1 sample code, the only thing visible is text from the console. You need to specify the game type that you want to start. The options are listed below:

- **Normal**—normal single player game. type: normal
- **Server**—start this as a multiplayer server. This is NOT a standalone server. type: server

- **Join**—connect to a server. type: join <ip.ip.ip.ip> or join <computername>

Server-side Changes

The only new material to cover is your addition of a special effect being created upon a projectile being destroyed. (ie. hitting a wall or other player)

Projectile.cpp

The first thing to examine is the **TouchNotify** method of the Projectile class. When a collision is detected, you must determine if the object connected with is part of the world, or if the object is a game object. The following code shows the implementation of the **TouchNotify** method.

```
uint32 Projectile::TouchNotify(void *pData, float fData)
{
    HOBJECT hObj = (HOBJECT)pData;

    //Prevent projectile from hitting the owner.
    if((hObj != m_hOwner) && (m_hOwner != NULL))
    {
        CollisionInfo Info;
        memset(&Info, 0, sizeof(Info));
        g_pLTServer->GetLastCollision(&Info);

        LTRESULT result = g_pLTSPHysics->IsWorldObject( hObj );

        if (result == LT_YES)
        {
            //g_pLTServer->CPrint( "Projectile::I hit the world at poly %d", Info.m_hPoly);

            // Send clientfx message
            uint32 dwFxFlags = 0;
            PlayClientFX("CanImpact", m_hObject, LTNULL, LTNULL, dwFxFlags);
        }
        else
        {
            //g_pLTServer->CPrint( "Projectile::I hit an object %p -> %p", hObj, m_hOwner );

            // Send clientfx message
            uint32 dwFxFlags = 0;
            PlayClientFX("PlayerImpact", m_hObject, LTNULL, LTNULL, dwFxFlags);
        }
        g_pLTServer->RemoveObject(m_hObject);
    }
    return 1;
}
```

Serverutilities.cpp

The next step to perform is to send a message to the ClientFX system requesting an FX to play. The **PlayClientFX** function is stored in serverutilities.cpp, and is responsible for creating the clientfx message to send to your clientshell. The message then gets handled by the clientfx system. The **PlayClientFX** function is only meant to be used with non-looping FX. It is set up to create a "fire and forget" FX. The following code shows the implementation of the **PlayClientFX** function.

```
void PlayClientFX(char* szFXName,
                 HOBJECT hParent,
                 HOBJECT hTarget,
                 LTVector* pvPos,
                 LTRotation* prRot,
                 LTVector* pvTargetPos,
                 uint32 dwFlags)
{
    LTVector vPos;
    LTRotation rRot;
    if(hParent)
    {
        g_pLTServer->GetObjectPos(hParent, &vPos);
        g_pLTServer->GetObjectRotation(hParent, &rRot);
    }
    else if( !pvPos || !prRot )
    {
        g_pLTServer->CPrint("ERROR in PlayClientFX()! Invalid position specified for FX: %s!", szFXName);
        return;
    }
    else
    {
        vPos = *pvPos;
        rRot = *prRot;
    }

    CAutoMessage cMsg;
    cMsg.Writeuint8(SFX_CLIENTFXGROUPINSTANT);
    cMsg.WriteString(szFXName);
    cMsg.Writeuint32(dwFlags);
    cMsg.WriteCompPos(vPos);
    cMsg.WriteCompLTRotation(rRot);
    cMsg.WriteObject(hParent);
    // Write the target information if we have any...

    if( hTarget || pvTargetPos )
    {
        cMsg.Writeuint8( true );
        cMsg.WriteObject( hTarget );
        LTVector vTargetPos;
        if( hTarget )
        {
            g_pLTServer->GetObjectPos( hTarget, &vTargetPos );
        }
        else
        {
            vTargetPos = *pvTargetPos;
        }
    }
}
```

```

        cMsg.WriteCompPos( vTargetPos );
    }
    else
    {
        cMsg.Writeuint8( false );
    }
    g_pLTServer->SendSFXMessage( cMsg.Read(), vPos, 0);
}

```

These are the only changes needed in order to create one time FX using the ClientFX system. As mentioned above, please read the [Animations2](#) tutorial and [base\Clientfx](#) sample.

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Programming Guide



Interface Manager

The Interface Manager provides a consistent, modular method by which game and engine code can define, implement, instantiate, and retrieve interfaces. To retrieve engine interface instances, your game code must use the Interface Manager. Also, your game code must use implement the **IClientShell** and **IServerShell** interfaces using the Interface Manager.

An interface is a C++ class that contains functions that can be called. An interface implementation is a C++ class that can be instantiated and that is either an interface or is derived from an interface class. Interface instantiations are actual instances of an interface implementation class.

The Interface Manager exposes a set of macros that provide the following functionality:

- interface versioning
- instance interface implementations expected by the engine
- access engine interface instances

Each of the preceeding macros are defined in the ltmodule.h header file.

This section provides information on the following topics:

[Implementing Classes](#)

Interfaces that are part of the Interface Manager system must derive from a base class called IBase. Interface implementations must use certain macros to add functionality required by the Interface Manager. Not all interfaces in your game code are required to be part of the Interface Manager system.

[Accessing Interface Instances](#)

The engine instantiates and manages many interface instances (such as ILTPhysics, ILTClient, ILTServer, and so on). These engine interface instances expose functions that your game code uses to conduct various operations. The Interface Manager provides a consistent method to access interface instances and call the exposed interface functions.

[Class Hierarchies](#)

Describes some of the different interface hierarchy arrangements you can use when implementing your class structure.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Programming Guide



Implementing Classes

The Interface Manager uses two types of classes: interface classes and implementation classes. A class may be both an interface class and an implementation class.

Interface Class

Interface classes managed by the Interface Manager must derive from a class called **IBase**. Ideally, an interface should contain only function pointers and pure virtual functions. Constructors/destructors, non-class type member variables, class type member variables that have constructors/destructors, inline functions, non-pure virtual functions, and non-virtual functions all have their own issues which may or may not cause problems depending on how they are used. Here is an example of an interface:

```

class IMyInterface : public IBase {

    public:

    interface_version(IMyInterface, 17);

    //a function

```

```
virtual void foobar(int32 num) = 0;

};
```

The **interface_version** macro is required in the class and takes the name of the interface, and the version number of this interface. A version number can technically be any signed integer, but it is recommended that they be non-negative and less than 100. In this example, the version number is 17. Any time a new function is added to or removed from the interface, function parameters change, or the ordering of members is modified the version number should be changed.

It is possible to have classes between **IBase** and **IMyInterface** that are used as containers for function declarations that are going to be common between two related interfaces.

Implementation Class

An implementation class is either a class derived from an interface class, or an interface class directly. When implementation classes are derived from an interface class (and not exposed for direct use themselves,) there are no restrictions to the kinds of variables or functions that they may have. However, the implementation class must always be able to be instantiated. Therefore, it must implement all pure virtual functions in the parent class. Here is an implementation of the interface **IMyInterface** shown above:

```
class IMyInterfaceImp : public IMyInterface {

    public:

    declare_interface(IMyInterfaceImp);

    //constructor

    IMyInterfaceImp();

    //our function.

    virtual void foobar(int32 num);

};
```

Class **IMyInterfaceImp** derives from **IMyInterface**, and overrides the interface's pure virtual functions. It also includes the **declare_interface** macro. **declare_interface** defines some required virtual functions declared in **IBase**.

Instantiating Interface Implementations

Interface implementation classes are instantiated using the **define_interface** or the **instantiate_interface** macros. These macros are used at global scope in a .cpp file.

The **define_interface** macro is the simpler of the two, and is used like this:

```
define_interface(IMyInterfaceImp, IMyInterface);
```

define_interface takes the name of the implementation class, and the name of the class that it implements. It performs the following operations:

- Declares a global variable of the implementation class type. This causes the constructor of the implementation class to be called. Remember though that the constructor is called before main(), so you are limited as to what you should be doing at that time. In general, all you should do is initialize function pointers.
- Declares a global variable of an internal type. The constructor of this variable places the instance of the implementation class into the Interface Manager.

The **instantiate_interface** macro is similar to **define_interface**, but has an additional parameter (instance_name).

```
instantiate_interface(impl_class, interface_class, instance_name);
```

The instance name of an implementation instance is used to identify a specific instance of an interface implementation when there is more than one. Several interfaces, such as **ILTCommon** and **ILTModel**, are instantiated twice each with the names Client and Server.

Using the **define_interface** macro is the same as using the **instantiate_interface** macro with the third parameter being **Default**.

Creating Interface and Implementation Classes

To create the classes for the Interface Manager

1. Create a header file that defines your interface class. This interface class must be derived from **IBase**. The interface class should not contain any non-virtual functions. Use the **interface_version** macro to set the version number of the interface.
2. Create an implementation class derived from your interface class. At the top of this implementation class insert the **declare_interface** macro. Pass in the name of the implementation class.
3. In the .CCP file that defines the implementation class, use the **define_interface** or **instantiate_interface** macro to instantiate the implementation class and register it with the Interface Manager.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.



Accessing Interface Instances

The Interface Manager provides the **define_holder** and **define_holder_to_instance** macros as a method of accessing a specific interface instance.

```
#define define_holder(interface_name, ptr_var);
```

The macro takes the name of the interface class that is desired, and the name of a pointer variable that will be set to point at an instance of an implementation of that interface. The pointer variable should be global (either static or non-static). After the **define_holder** macro has been used, the pointer variable can be used to access functions and variables in the interface class.

Accessing Engine Interfaces

The most common usage of the **define_holder** macro is to access engine interfaces and the many functions that they expose. The **define_holder** macro is used to access interfaces that have only a single implementation in the engine. Engine interfaces are instantiated by the engine. You cannot instantiate your own versions of engine interfaces. Instead, you must acquire a pointer to the interface instances created by the engine. After you have this pointer, you can use it to call the interface's functions as desired.

To obtain a pointer to an engine interface

1. Include the appropriate header file for the interface (for example, **iltclient.h** for the **ILTClient** interface).
2. Declare a pointer variable of the appropriate interface type.
3. Use the **define_holder_to_instance** or **define_holder** macro to set the pointer variable to the interface instance.

Example Code

The following code snippet shows how to access the **ILTClient** interface instance.

```
#include "iltclient.h"

static ILTClient *g_pLTClient;

define_holder(ILTClient, g_pLTClient);
```

First, the header file containing the **ILTClient** definition is included. Then, a global static pointer is declared. Finally, the **define_holder** macro is used to set the **g_pLTClient** variable to the appropriate address of the **ILTClient** interface instance. The **g_pLTClient** pointer can now be used within that .CPP file to call **ILTClient** interface functions.

For engine interfaces that have both a client and a server implementation, you must use the **define_holder_to_instance** macro. This macro takes as the third parameter the instance name. For example, the engine instantiates the **ILTCommon** interface with the Client and Server instance names. To access the Client instance use the following type of code.

```
#include "ILTCommon.h"

static ILTCommon *g_pLTCommonClient;

define_holder_to_instance(ILTCommon, g_pLTCommonClient, Client);
```

Using **define_holder** is equivalent to using **define_holder_to_instance** with the third parameter as **Default**.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Programming Guide



Class Hierarchies

Implementation classes derived directly from the interface class, which is derived directly from **IBase**, are the simplest, and probably the most frequent, case. However, there are more complicated arrangements that can be made depending on what is needed. Here are a couple of the more useful arrangements that can be used. There are many more that are possible.

Interface Hierarchies

Interfaces do not need to be directly derived from **IBase**. This might be useful if two interfaces have a subset of common functions declared, but have separate implementations of these functions. This is done currently with the **ILTClient** and **ILTServer** interfaces, which are both derived from **ILTCBase**. When interfaces are derived from other interfaces (whether or not there are direct implementations of the base interface class), there is a special version of the **interface_version** macro that should be used in the derived class.

```
class IDerived : public IMyInterface {
public:
    interface_version_derived(IDerived, IMyInterface, 1);

    virtual void AddedFunc() = 0;
```

```
};
```

What the `interface_version_derived` macro does is combine the version number specified in the macro and combines it with the version number of the base class. The exact formula is:

$$100 * \text{version_base} + \text{version_derived}$$

So in the case of `IDerived`, the real version number would be 1701, since the version of **IMyInterface** was defined as 17. In this way if the version number of either the base or derived class changes, it has an effect on anyone who references the version number of the derived class. If the hierarchy is deeper than 4 levels, however, this system will run into problems.

Stub Implementations

An implementation class does not need to be directly derived from the interface class it is implementing. Instead, there can be a class between the instantiated implementation and interface class that supplies empty or default bodies for the interface functions. Then the real implementation class can derive from the stub implementation class and implement only those functions it wants to.

The engine expects user game code to implement and instantiate the **IClientShell** and **IServerShell** interfaces. When creating the implementation of these interfaces, users can derive their implementation classes from the provided stub implementation classes, **IClientShellStub** and **IServerShellStub**.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Programming Guide



ClientFX

This section provides a detailed technical explanation of the required components and implementation efforts needed to successfully use the ClientFX system within your current project under the LithTech Jupiter Development System. A sample project is provided with this SDK to further help developers utilize this flexible system. The sample is located in the following directory:

```
\\Samples\\base\\clientfx\\
```

This section contains the following topics:

Topic	Description
Using ClientFX	Provides instructions for implementing ClientFX.
ClientFX Sample	Describes the code and operation of the clientfx sample.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Programming Guide



Using ClientFX

The ClientFX System provides a simplified interface with which to create interesting and elaborate special effects. The ClientFX DLL code base sits on the game code layer. It interfaces with the LithTech engine to create blazing fast particle systems, sprite systems, lighting effects, poly trails, and many more “effect” types.

Any or all of the effect types (keys) can be added together to make a single playable effect. The FxED application is how the artists interface with the ClientFX System. It allows artists to create effects without the need for any additional engineering. In short, it puts the power in the hands of the artists.

This section contains the following topics specific to implementing ClientFX:

- [Core Components](#)
- [Implementation Details](#)
- [ClientShell Specifics](#)

Core Components

ClientFX DLL (Clientfx.fxd)

The ClientFX DLL contains the “keys”, that can be used in FxED when creating your own custom effects, and all of the “behind the scenes” source code associated with controlling those effects. The source code for this project can be found in the following directory:

\LT_Jupiter_Xxx\Engine\clientfx.
(replace Xxx with Bin or Src depending on your version)

FxED application

This is the application used by artists to create effects that utilize the ClientFX system. The details of it's use are documented in the JupiterHelp.chm document found in the Docs folder.

Game code objects:

Client-side

ClientFXDB

This object is the database of effects containing all the cached properties, and the constant, non-instanced ClientFX data. It is also responsible for loading and unloading the ClientFX DLL(ClientFx.fxd).

ClientFXMgr

This object controls input to the ClientFX system. The ClientShell interfaces with this object to inform the ClientFX system to update, when to render, and what effects to play.

ClientShell code

The ClientFXDB and ClientFXMgr objects are contained within the ClientShell. Several messages, such as rendering, updating of the effects are passed to the ClientFXMgr here.

Server-side

ServerSpecialFX

This object is placed in a level using DEdit, and sends a message to the ClientShell informing ClientFX which effect it should play. This object reads in the ClientFX.fcf file in order to generate a list that allows the level designer to pick the FX by name out of a drop down menu in DEdit. It expects the .fcf file to be located in \rez\Clientfx\ of your project. If you plan to change this, you will need to make the appropriate changes to this object.

[Top](#)

Implementation Details

ClientShell project

Preprocessor Defines:

_CLIENTBUILD must be added to the preprocessor define section in Visual Studio, for debug and release projects.

Includes:

The proper path to \Engine\clientfx\Shared must be added to the includes section for both debug and release projects. This path contains several files required by both ClientFXDB and ClientFXMgr.

Source files:

The following files should be moved/copied to your actual project source directory.

- ClientFXDB.cpp
- ClientFXDB.h

The following files should be moved/copied to your actual project source directory.

- ClientFXMgr.cpp
- ClientFXMgr.h

The following files are referenced from the \Engine\clientfx\Shared directory. They are required for proper linking.

- CommonUtilities.cpp
- ParsedMsg.cpp
- WinUtil.cpp

ServerShell project

Includes:

The proper path to \Engine\clientfx\Shared must be added to the includes section for both debug and release projects. This path contains several files required by the ServerSpecialFX object.

Source files:

The following files need to be placed in your project source directory.

- ServerSpecialFX.cpp
- ServerSpecialFX.h

Linking Files:

The following file is referenced from the \Engine\clientfx\Shared directory. It is needed for proper linking.

- ParsedMsg.cpp

[Top](#)

ClientShell Specifics

Several pieces game code need to be added to your ClientShell source which will initialize, and pass control parameters to the ClientFXMgr and ClientFXDB objects. As noted in the summary, this is done in the clientfx sample project. Developers should review the sample project for a working example of these methods, in-use.

CClientFXMgr

```
bool CClientFXMgr::Init( ILTClient *pLTClient )
```

This method is used to grant access to the current instance of the ILTClient interface.

```
void CClientFXMgr::SetCamera( HOBJECT hCamera )
```

In order for ClientFX to render, we need to set a camera object for the ClientFX instance.

```
void CClientFXMgr::ShutdownAllFX()
```

This will stop all FX from playing and remove their instances from memory.

```
bool CClientFXMgr::UpdateAllActiveFX( bool bRender )
```

This is used to allow the FX to update.

```
bool CClientFXMgr::RenderAllActiveFX( bool bRender )
```

This method is used to tell ClientFX to render it's FX now.

```
void CClientFXMgr::OnRendererShutdown()
```

This method is used to tell ClientFX that the renderer is no longer available. For example, if the application window loses "focus".

```
void CClientFXMgr::OnSpecialEffectNotify( HOBJECT hObj, ILTMessage_Read *pMessage )
```

The ServerSpecialFX object sets a special effect notify message for each FX placed in DEdit.

This is how the client is informed of server side effects. That message is handled here.

```
bool CClientFXMgr::OnObjectRemove( HOBJECT hObj )
```

When an object is removed from the game world, ClientFX needs to know that the object associated with an effect is no longer available. This allows the system to shutdown and remove the effect.

Additional Methods of Interest

```
void CClientFXMgr::SetDetailLevel( int nLOD )
```

This method allows developers to setup performance options within the ClientFX system

```
void CClientFXMgr::SetGoreEnabled(bool bEnabled)
```

This method allows developers to turn effects marked as "gore" off.

```
void CClientFXMgr::Pause(bool bPause)
```

This method is used to pause and un-pause the ClientFX system.

```
bool CClientFXMgr::IsPaused()
```

This method returns the current state of pause on the ClientFX system.

CClientFXDB

The only action required for dealing with the ClientFXDB is the initialization of that object.

```
CClientFXDB::GetSingleton().Init( ILTClient *pLTClient )
```

This method sets the current instance of the ILTClient interface for the ClientFXDB object.

[Top](#)

Programming Guide

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.



ClientFX Sample

The ClientFX sample shows a working example of how you can use the ClientFX system.

The ClientFX sample project is located in the following directory:

```
\\Samples\base\clientfx.
```

This sample allows developers to browse and view effects created by artists. This sample reads through all of the effects in the current .fxf file and creates a graphical selection menu allowing you to scroll through and play any effects in that list. You can use this sample as a viewer application for the listed effects. Please refer to the [Using ClientFX](#) section for a detailed description of the components of the ClientFX system.

This section contains the following ClientFx topics:

- [Client-side Changes](#)
- [Server-side Changes](#)

Client-side Changes

ltclientshell.cpp

First you must initialize the ClientFXDB object. This object holds all of the information for the artist created FX.

```
LTRESULT CLTClientShell::OnEngineInitialized(RMode *pMode, LTGUID *pAppGuid)
{
    ...
    // Initialize Our ClientFX Database
    if(!CClientFXDB::GetSingleton().Init(g_pLTClient))
    {
        g_pLTClient->ShutdownWithMessage( "Could not init ClientFXDB!" );
        return LT_ERROR;
    }
    ...
}
```

Next, you initialize the ClientFXMgr object. This object is used to control the ClientFX system. m_ClientFXMgr is a member variable added to your clientshell in the form: CClientFXMgr m_ClientFXMgr; You also want to give ClientFX access to your camera.

```
LTRESULT CLTClientShell::OnEngineInitialized(RMode *pMode, LTGUID *pAppGuid)
{
    ...
    // Init the ClientFX mgr... (this must become before most other classes)
    if( !m_ClientFXMgr.Init( g_pLTClient ) )
    {
        // Make sure ClientFX.fxd is built and in the game dir
        g_pLTClient->ShutdownWithMessage( "Could not init ClientFXMgr!" );
        return LT_ERROR;
    }
    // We need to make sure to setup the camera for the FX Mgr
    m_ClientFXMgr.SetCamera(m_hCamera);
    ...
}
```

You must now tell the ClientFXMgr control object that you want to update and render the currently active clientfx FX.

```
LTRESULT CLTClientShell::Render()
{
    ...
    //update all the effects status as well as any that might effect the camera
    m_ClientFXMgr.UpdateAllActiveFX( m_bRender );
    // Render the effects
    m_ClientFXMgr.RenderAllActiveFX( m_bRender );
    ...
}
```

If the application loses window focus, then you must inform ClientFx of this event.

```
void CLTClientShell::OnEvent(uint32 dwEventID, uint32 dwParam)
{
    ...
    case LTEVENT_RENDERTERM:
    {
        m_bRender = false;
    }
}
```

```

        // Let the ClientFx mgr know the renderer is shutting down
        m_ClientFXMgr.OnRendererShutdown();
    }
    break;
    ..
}

```

ClientFx needs to know about any messages attached to objects. This is commonly done on the server when an effect is created. ClientFx uses these message to associate FX with objects. This is useful the the FX is supposed to follow the object around. A "rocket smoke trail" is an example of that.

```

void CLTClientShell::SpecialEffectNotify(HLOCALOBJ hObj, ILTMessage_Read *pMessage)
{
    ..
    m_ClientFXMgr.OnSpecialEffectNotify( hObj, pMessage );
    ..
}

```

When an object is removed, ClientFx needs to know. This allows the system to remove any FX that were attached to that object.

```

void CLTClientShell::OnObjectRemove(HLOCALOBJ hObj)
{
    ..
    m_ClientFXMgr.OnObjectRemove( hObj );
    ..
}

```

When the client exits from the game world, ClientFX needs to know about it, so that all the currently active effects can be shutdown and their instances removed from memory.

```

void CLTClientShell::OnExitWorld()
{
    ..
    // Stop all the client FX
    m_ClientFXMgr.ShutdownAllFX();
    ..
}

```

[Top](#)

Server-side Changes

ServerSpecialFX object

The only major change to our server-side code is the addition of the ServerSpecialFX object. This object is placeable in DEdit by level designers. This object reads the .fcf to build a dynamic list of effects, from which level designers can select from instead of needing to type the effect's name manually. This is done through the IObjectPlugin class.

In the TurnOn() function we set the object special effects message, as mentioned above. This message gets sent to the client, which in turn gets sent to the ClientFX system.

```

void SpecialFX::TurnON()
{
    ..
    CAutoMessage cMsg;
    cMsg.Writeuint8( SFX_CLIENTFXGROUP );
    cMsg.WriteString( m_sFxName );
    cMsg.Writeuint32( m_dwFxFlags );
    if( m_hTargetObj )
    {
        cMsg.Writeuint8( true );
        cMsg.WriteObject( m_hTargetObj );
        LTVector vPos;
        g_pLTServer->GetObjectPos( m_hTargetObj, &vPos );
        cMsg.WriteCompPos( vPos );
    }
    else
    {
        cMsg.Writeuint8( false );
    }

    g_pLTServer->SetObjectSFXMessage( m_hObject, cMsg.Read() );
    // Set flags so the client knows we are on...
    g_pLTSCCommon->SetObjectFlags( m_hObject, OFT_Flags, FLAG_FORCECLIENTUPDATE, FLAG_FORCECLIENTUPDATE );

    g_pLTSCCommon->SetObjectFlags( m_hObject, OFT_User, USRFLG_SFX_ON, USRFLG_SFX_ON );
    ..
}

```

ServerUtilites.cpp

There is one more useful function to mention is PlayClientFX() located in serverutilities.cpp. This function can be used to spawn instant effects.

```

void PlayClientFX(char* szFXName,
                 HOBJECT hParent,
                 HOBJECT hTarget,
                 LTVector* pvPos,
                 LTRotation *prRot,
                 LTVector *pvTargetPos,
                 uint32 dwFlags)

```

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Programming Guide



Client Shell

Client Shell is a term used to encompass all client-side game code operations and logic. This code is compiled into CShell.dll. The TO2 client shell code is located in the GameClientShell.h header file and GameClientShell.cpp file. You can use these files as a base or an example for your own client shell.

The client shell communicates with the Jupiter to initiate a variety of operations. Jupiter, in return, notifies the client shell of various events, such as engine initialization, messages from the server, input status changes, and so on.

Messaging between the client shell and the server shell is initiated and handled by the game code. Jupiter is not involved with initiating client-to-server or server-to-client game code-related communication. However, Jupiter is used as the mechanism to communicate these messages.

This section introduces the client shell through the following topics:

Client Shell Topic

[SETUP_CLIENTSHELL](#)

Description

This macro sets up required DLL exports and some global variables.

[IClientShell](#)

Your code must include an IClientShell-derived class instance, whether it is one of your own creation or merely based on the TO2 code. Jupiter sends notifications to this derived class to inform your game of various events. There are a variety of notification-handling functions that you may implement. The two most important of these are OnEngineInitialized and Update. You will use this class as a starting point for your game code.

[ILTClient](#)

The ILTClient class is instantiated by Jupiter. You can obtain a pointer to the instance using the define_holder_to_instance macro.

[Collecting Input](#)

Describes how to collect input from the users hardware interface and associate the input with game code. Collecting input is largely a function of the Client Shell.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Programming Guide



SETUP_CLIENTSHELL

SETUP_CLIENTSHELL is a macro definition and can be found in the IClientShell.h header file. It performs two operations:

1. It uses the define_holder macro to acquire a pointer to Jupiter's ILTClient interface instance. After using the SETUP_CLIENTSHELL macro you can access the ILTClient functions using the g_pLTClient global pointer.
2. It sets up required DLL function exports.

```
SETUP_CLIENTSHELL () ;
```

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Programming Guide



IClientShell Class

The **IClientShell** class is used by Jupiter to notify the game code of events. You must derive a new class from **IClientShell** to provide communication from Jupiter to your game code.

Your **IClientShell**-derived class must include the **declare_interface** macro at the top of the class definition. This macro provides functions required for proper communication with Jupiter.

IClientShell Functions

There are many event callback functions in **IClientShell**. You may implement each of these functions to provide any desired functionality whenever Jupiter sends notification of a specific event. Some of these functions are more important than others. Depending on the functionality you want to offer in your game, some of these functions may be left un-implemented. However, implementation of certain functions are imperative to basic game functionality. For example, if you do not override **OnEngineInitialized**, it will always return **LT_ERROR**, immediately shutting down the game.

The notification functions available in **IClientShell** include:

OnCommandOff—Called when commands (defined in AUTOEXEC.CFG) are released.

OnCommandOn—Called when commands (defined in AUTOEXEC.CFG) are triggered.

OnEngineInitialized—Called when Jupiter is initialized. Required function.

OnEngineTerm—Called when Jupiter is terminated.

OnEnterWorld—Called when the player enters a world. This is useful for creating or initializing lists for referencing engine objects.

OnEvent—This function gives you a chance to hook into Jupiter’s event handling to trap events such as the renderer being reinitialized, or the game losing or regaining focus.

OnExitWorld—Called when the player exits a world. This is useful for cleaning up lists for referencing engine objects. When the player exits a world, the game loses handles to every engine object and so it is important to clear out references to them.

OnKeyDown—Used to access PC virtual key data. DO NOT USE THESE IF YOU INTEND TO PORT YOUR GAME TO ANOTHER PLATFORM. Virtual keys are specific to Windows and, unlike Jupiter commands (see OnCommandOn/OnCommandOff), are not portable to any other computer or console configuration.

OnKeyUp—Used to access PC virtual key data. DO NOT USE THESE IF YOU INTEND TO PORT YOUR GAME TO ANOTHER PLATFORM. Virtual keys are specific to Windows and, unlike Jupiter commands (see OnCommandOn/OnCommandOff), are not portable to any other computer or console configuration.

OnMessage—Called when the server game code sends a normal server-to-client message to the client.

OnModelKey—Used for client-side MODELKEY handling. Model keys are strings coupled with frames of model animation that are used to target events.

OnObjectMove—Used to aid in handling client-side physics and movement prediction.

OnObjectRemove—Called when an object is removed.

OnObjectRotate—Used to aid in handling client-side physics and movement prediction.

OnTouchNotify—Used for client-side collision detection.

PostUpdate—Called each update cycle after Update and each engine object have been updated.

PreLoadWorld—Called before a world is loaded, and anything drawn to the screen is automatically displayed after the call exits. This is useful for displaying a “Now Loading...” screen or something similar.

PreUpdate—Called each update cycle, just before Update. Currently, nothing happens between this call and the call to Update, so this function is primarily used for conceptual organization.

SpecialEffectNotify—Called when the server sends a special effect message to the client.

Update—Called as frequently as possible. You should use this function to conduct frequent game operations.

[Top](#)

Instantiating Your IClientShell-derived Class

The **IClientShell** instance can be created on the heap (using the new operator or malloc) or instantiated on the stack.

To instantiate your **IClientShell**-derived class, you must declare a global pointer and then use the **define_interface** macro.

```
impl_class *g_pImpl_Class = NULL;

define_interface(impl_class, interface_class);
```

The **impl_class** parameter is the name of your derived class. The **interface_class** parameter is the name of the class from which the **impl_class** parameter is derived (in this case, **IClientShell**).

[Top](#)

OnEngineInitialized

The **OnEngineInitialized** function is called when Jupiter has been started and fully initialized. At this point, the Jupiter engine has loaded the client .dll’s, ensured that the proper operating system support is available, and loaded and parsed the autoexec.cfg file. Any console variables and key bindings defined in the autoexec.cfg and/or the command-line have been set.

The game implementation of this function must set the proper display resolution and initialize the renderer on the PC. The game should also be set to an initial state. In general, this would mean loading a startup screen and giving the user some options.

```
LTRESULT OnEngineInitialized(RMode *pMode, LTGUID *pAppGuid)

{
```

```
// Your game code goes here

// Return something other than LT_OK if you want to abort the game

}
```

Jupiter passes in a pointer to the default rendering mode (**pMode**) to initialize the renderer. Rendering modes are defined using **RMode** structures. The **RMode** identified by the **pMode** parameter is created by Jupiter based on the following:

- The 3d graphics card. If Jupiter finds a 3D graphics card, the **pMode->m_bHardware** variable is set to TRUE.
- Default resolution and bitdepth. Command line parameters get the highest priority and will be used if specified.
 - Console variable settings (**RenderDLL**, **ScreenWidth**, and **ScreenHeight**) in the autoexec.cfg file.

If none of these can be found, Jupiter defaults to 640x480 with 16-bit color (the lowest possible resolution that Jupiter can use).

Jupiter does not automatically initialize the renderer. Your game code must do this by calling the **ILTCClient::SetRenderMode** function.

The **pAppGuid** out-parameter is a pointer to a **LTGUID** structure. You should fill in the GUID before returning from this function. Jupiter will store this information and use it to determine which net games can be queried and connected to.

If the **OnEngineInitialized** function returns anything other than **LT_OK**, LithTech shuts down immediately.

[Top](#)

Set the Rendering Mode

The **OnEngineInitialized** function receives a rendering mode from Jupiter. However, Jupiter does not automatically initialize the renderer. Your implementation of the **OnEngineInitialized** function must explicitly set the rendering mode by calling the **ILTCClient::SetRenderMode** function.

```
LTRESULT (*SetRenderMode)(RMode *pMode);
```

This function is accessible to your code using the **ILTCClient** pointer, **g_pILTCClient**. The **pMode** parameter is a pointer to an **RMode** structure identifying the rendering mode with which you would like to initialize the renderer. Your code may use the **RMode** structure passed in to the **OnEngineInitialized** function, or you may create and utilize other **RMode** structures of your choice. However, Jupiter may not use the rendering mode you requested based on internal engine logic and hardware/software limitations. After calling **SetRenderMode**, you can confirm the actual rendering mode used with the **ILTCClient::GetRenderMode** function.

[Top](#)

Update

The Jupiter engine calls the **IClientShell::Update** function on each update loop. It is the main function that is called while Jupiter is running. This is where your game should poll input, perform any client game-code logic, and render the scene.

Jupiter calls the **Update** function as often as possible. The frequency with which the function is called is dependant upon the hardware limitations of the computer and the number of operations the game is currently processing.

[Top](#)

void Update()

Jupiter does not pass any information into the **Update** function, nor does it expect any return value. Jupiter merely calls the function to give the client shell the opportunity to conduct game operations as often as possible.

[Top](#)

Render

The **CgameClientShell::RenderCamera** function uses client engine API calls to clear the screen, start a 3D optimized block, render the camera, end the 3D block, and flip the screen buffer.

[Top](#)



ILTCClient

The **ILTCClient** class provides significant functionality to your game code. You cannot instantiate **ILTCClient** in your code. Jupiter instantiates the **ILTCClient**

interface. The [SETUP_CLIENTSHELL](#) macro gives your game code a pointer to LithTech's **ILTCClient** instance.

Use this pointer to access the **ILTCClient** functions. For example, to pause music in a game, you would call **ILTCClient::PauseMusic**. Various **ILTCClient** functions are discussed in throughout this book. The **ILTCClient** class is defined in the `iltclient.h` header file.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Programming Guide



Collecting Input

Enabling input device recognition is a two-step process. First, input devices and commands must be bound so LithTech will send notifications when input occurs. Secondly, game code must process the input event notifications.

This section explains the following two steps:

- [About Collecting Input](#)
- [Editing Autoexec.cfg](#)
- [Input Notification Game Code](#)

About Collecting Input

The LithTech engine handles the bulk of input collection operations. Generally, the process of capturing user input proceeds as follows:

1. The user presses a key, moves the mouse, or operates some other input device.
2. LithTech detects the input.
3. LithTech determines whether or not the input is recognized by the game (using the information in the `autoexec.cfg` file).
4. If the input is not recognized by the game, LithTech does not send an event notification and the process for this specific input event ends.
5. If the input is recognized by the game, LithTech sends an event notification using the **IClientShell::OnCommandOn** and **IClientShell::OnCommandOff** functions, as appropriate.
6. Game code conducts operations based on the input.

The input detection process is reactive. Game code waits for an event notification from LithTech. The game can also poll LithTech to determine the state of the input devices. This is accomplished using the **ILTCClient::IsCommandOn** function.

[Top](#)

Editing Autoexec.cfg

The `autoexec.cfg` file is a text file containing commands for LithTech. This file tells LithTech what type of input a game recognizes. Many options besides input handling can also be found in this file. You must set the input devices that you want to enable for your game in this file.

These files should be placed in the same directory as `LithTech.exe`. They are automatically loaded and run by LithTech.

To provide input recognition

1. Set names to command integers.
2. Enable the input device.
3. Bind input device to command integers.

Step One—Set names for command integers

To ease readability, you must set descriptive names to integers that you have decided represent commands to LithTech.

The `AddAction` command is used to set a name to an integer. A game action is a name that defines a specific input action for a game. For instance, **FirePistol** could be a game action bound to a mouse click event.

AddAction action_name value

The **action_name** parameter may be any string (no spaces). This string is used to identify the command in the **rangebind** command described below.

The **value** parameter may be any integer between 0 and 255. Each different command must have a different integer. Do not use the same integer for more than one command.

Names should be set at the beginning of the `autoexec.cfg`. You should not alter the names in your game. However, additional bindings may be added.

Step Two—Enable the input device

You must enable each device for which you want to receive input. To enable a device, use the `enabledevice` command in the `autoexec.cfg` file:

```
enabledevice "devicename"
```

The **devicename** parameter is the identifier for the input device. It may be one of the following values:

For PC:

```
##mouse

##keyboard
```

Step Three—Bind input device to command integers

Finally, you must bind the specific inputs to specific commands using the **rangebind** command. A binding is the association of a specific object and range (such as a keyboard key, a mouse click, or a joystick movement) to a specific action (such as a crouch, open, or move forward).

```
rangebind "device_name" "object_name" min_value max_value "action_name"
```

The **device_name** parameter identifies the device to which the action will be set. The device must be enabled (using the **enabledevice** command) before using it in the **rangebind** command.

The **object_name** parameter sets the input that triggers the action. The LithTech engine supports all the **DIK_*** scan codes in the DirectInput `dinput.h` file. For maximum portability, use the scan codes whenever possible. For keyboard keys, the name of the key is also an option. For instance, use “E” to set the E key as a trigger for an action. LithTech recognizes the name of keys supported by Microsoft’s DirectInput.

The **min_value** and **max_value** parameters set ranges for the object. This is used primarily for joysticks. For keyboard key input, set both of these to 0.0.

The **action_name** parameter identifies the action that is triggered by this binding. The action name must be set using the **AddAction** command prior to using it in the **rangebind** command.

The last three parameters can be repeated in groups to add more actions. For example:

```
rangebind "#Controller" x 0 22767 left 42767 65535 right
```

Mouse and Joystick Axes

While the mouse and joystick buttons are enabled in the same manner as keyboard keys, the mouse and joystick axes require special definition. An axis is first bound to an in-game axis, and then it can be scaled for sensitivity.

The axes actions must be defined and the mouse/joystick device enabled as described above.

To bind the axes, the trigger names in the **rangebind** command must be set to a specific string: X-axis, Y-axis, and Z-axis. You must use a negative number to bind an axis. Commonly, -1, -2, and -3 correspond to X, Y, and Z. For example:

```
rangebind "##mouse" "Y-axis" 0.000000 0.000000 "Axis2"
```

After you have bound each axis, you can scale it using the **scale** command.

```
scale "device_name" "axis" "scaling_value"
```

The **device_name** variable is a string identifies the device being scaled, usually “##mouse”.

The **axis** variable is a string that identifies the axis to scale. This can be X-axis, Y-axis, or Z-axis.

The **scaling_value** variable sets the scaling factor for the axis. This scaling value is usually set through a sensitivity slider inside the game (and coded with the **InputMgr::ScaleTrigger** function).

The mappings you have enabled will be all that your game receives. If you are not receiving messages, be sure to check that these bindings are set up properly.

User-specific Configuration Files

Your **IClientShell** class can also load user-specific .CFG files. This allows users to have separate preferences in the game. The command to load a config file is **ILTClient::ReadConfigFile**.

```
LTRESULT ILTClient::ReadConfigFile(char *pFileName);
```

[Top](#)

Input Notification Game Code

Your game code must bind the input events to appropriate game actions using the **ILTClient** interface.

At launch, LithTech reads the `autoexec.cfg` file and sets up all of the devices that you listed with the appropriate bindings. Whenever LithTech detects input, it sends an event notification to your **IClientShell** instance if you have bound a command for that type of input in your game code.

The functions that are notified of input events include **IClientShell::OnCommandOn** and **IClientShell::OnCommandOff**. To support these functions, you must create the same command bindings in game code as you set in the `autoexec.cfg` file.

OnCommandOn and OnCommandOff

LithTech calls your **IClientShell** instance on every press of a key that you have mapped to a command in the `autoexec.cfg` file. It will notify you of key presses and key releases using the **IClientShell::OnCommandOn** and **OnCommandOff** functions, respectively. These functions are defined in the `IClientShell.h` header file.

Each of these functions receives an integer, a range, and control data from LithTech. The integer identifies the specific command. The integer value for a given command is set in the `autoexec.cfg` file.



Server Shell

Server Shell is a term used to encompass all server-side game code operations and logic. This code is compiled into Object.Ito. The TO2 server shell code is located in the GameServerShell.h header file and GameServerShell.cpp file. You can use these files as a base or an example for your own server shell.

The server shell communicates with the Jupiter engine to initiate a variety of operations. The Jupiter engine, in return, notifies the server shell of various events.

Messaging between the client shell and the server shell is initiated and handled by the game code. The engine is not involved with initiating client-to-server or server-to-client communication. However, the engine is used as the mechanism to communicate these messages.

This chapter introduces the basics of the server shell. It has the following major sections:

Server Shell Topic

[SETUP_SERVERSHELL](#)

[IServerShell](#)

[ILTServer](#)

Description

This macro sets up required DLL exports and some global variables.

Your code must create (or derive) an instance of this class. Jupiter sends notifications to this class to inform your game of various events. There are a variety of notification-handling functions that you may implement. Two important functions are **OnClientEnterWorld** and **OnClientExitWorld**.

The **ILTServer** class is instantiated by Jupiter. You can obtain a pointer to the instance using the **define_holder** macro.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.



SETUP_SERVERSHELL

SETUP_SERVERSHELL is a macro definition and can be found in iservershell.h. It performs two operations:

1. It uses the **define_holder** macro to acquire a pointer to Jupiter's **ILTServer** interface instance. After using the **SETUP_SERVERSHELL** macro, you can access the **ILTServer** functions using the **g_pILTServer** global pointer.
2. It sets up required DLL function exports.

```
SETUP_SERVERSHELL () ;
```

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.



IServerShell Class

The **IServerShell** class is used by the Jupiter engine to notify the game code of events. You must derive a new class from **IServerShell** to provide communication between Jupiter and your game code.

Your **IServerShell**-derived class must include the **declare_interface** macro at the top of the class definition. This macro enables functions required for proper communication with Jupiter.

IServerShell Functions

There are many event callback functions in **IServerShell**. You may implement each of these functions to provide any desired functionality whenever Jupiter sends a notification of a specific event. Some of these functions are more important than others. Depending on the functionality you want to offer in your game, some of these functions may be left un-implemented. However, implementation of certain functions is imperative to basic game functionality.

The event callback functions include:

- CacheFiles

- FileLoadNotify
 - OnAddClient
- OnClientEnterWorld
- OnClientExitWorld
 - OnCommandOff
 - OnCommandOn
 - OnMessage
- OnObjectMessage
- OnPlaybackFinish
- OnRemoveClient
 - PostStartWorld
 - PreStartWorld
 - ProcessPacket
- ServerAppMessageFn
 - Update

[Top](#)

Instantiating Your IServerShell-derived Class

The IServerShell instance should be instantiated on the stack.

To instantiate your IServerShell-derived class, you must declare a global pointer and then use the Interface Manager's define_interface macro.

```
define_interface(impl_class, IServerShell);
```

The impl_class parameter is the name of your derived class. The second parameter, interface_class, is the name of the class from which the impl_class parameter is derived (in this case, IServerShell).

[Top](#)

OnClientEnterWorld

Jupiter calls the **IServerShell::OnClientEnterWorld** function when a client enters a world. You must create an object to represent the new client. Returning NULL will deny the client the ability to enter the world.

Note: You must implement **OnClientEnterWorld** in the Object.lto file.

The following example shows how to implement **OnClientEnterWorld**:

```
LPBASECLASS OnClientEnterWorld(HCLIENT hClient, void *pClientData, uint32 clientDataLen)
{
    // Your game code goes here
}
```

Jupiter passes three parameters to **OnClientEnterWorld**:

hClient—A handle to an HCLIENT instance representing the client that has just entered the world.

pClientData—A pointer to an address containing the StartGameRequest data for the client that just entered the world.

clientDataLen—The length of the data identified by the pClientData pointer parameter.

The function must return a handle to a new object that is associated with the new client. This object will be the client's representative on the server. If you do not return a valid BaseClass derived object from this function, the client will not be added to the world.

[Top](#)

OnClientExitWorld

Jupiter calls the **OnClientExitWorld** function when a client exits a world.

Note: You must implement **OnClientExitWorld** in the Object.lto file.

The following example shows how to implement **OnClientExitWorld**:

```
void OnClientExitWorld(HCLIENT hClient)
{
    //Your game code goes here
}
```

Jupiter provides your game code with a handle to the client that has just exited the world (**hClient**). You should use this handle to delete the associated client object from your game.

[Top](#)

Update

The Jupiter engine constantly calls the **IServerShell::Update** function.

Jupiter calls the **Update** function as often as possible. The frequency with which the function is called is dependant upon the hardware limitations of the computer and the number of operations the game is currently processing.

Note: The Update function should be implemented in the Object.lto.

[Top](#)

void Update()

Jupiter does not pass any information into the Update function, nor does it expect any return value. Jupiter merely calls the function to give you the opportunity to conduct game operations as often as possible.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Programming Guide



ILTServer

The **ILTServer** class provides significant functionality to your game code. You cannot instantiate **ILTServer** in your code. Jupiter instantiates the **ILTServer** interface. The [SETUP_SERVERSHELL](#) macro gives your game code a pointer (**g_pILTServer**) to Jupiter's ILTServer instance.

Use this pointer to access the **ILTServer** functions. For example, use the **ILTServer::KickClient** function to kick a client off of the server.

Various **ILTServer** functions are discussed in this book. The **ILTServer** class is defined in the `iltserver.h` header file.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Programming Guide



Simulation Objects

LithTech utilizes a system of objects to simulate items in the game environment. These simulation objects can be such things as tables, chairs, monsters, robots, levers, lights, players, donuts, signs, and more. Simulation objects need not be tangible. Starting positions, key framers, paths, and cameras are also simulation objects.

There are two primary implementations of objects in LithTech.

1. Your game code manually creates simulation objects for your game.
2. You use DEdit to create a world. The level designer places objects that have been defined in your game code and published to DEdit in the world. When the game starts, LithTech automatically creates the simulation objects, instantiating both game objects and associated engine objects.

This section introduces the basics of LithTech's use of simulation objects.

Simulation Object Topics

[Engine Objects and Game Objects](#)

[Defining Simulation Objects](#)

[Publishing Simulation Objects to DEdit](#)

[Creating Simulation Objects](#)

Description

A simulation object is composed of a game object and an associated engine object.

Before you can include a simulation object in your game, you must derive a new game object class from the BaseClass class.

All simulation objects must be published to DEdit using the **BEGIN_CLASS** and **END_CLASS** macros, making them accessible to your level designers.

Simulation objects can be manually created in your game code and are also automatically created based on world files. Creation of a simulation object results in the instantiation of a game object and its associated engine object.

After a simulation object has been created in your game you will often need to modify it. Accomplish

[Using Simulation Objects](#)

[Client-Side Objects](#)

this by accessing and modifying the game object and engine object member variables.

You can create client-side objects in your game code. Client-side objects are not simulation objects because they do not include the game object portion of the simulation object, since the `BaseClass` class is not supported on the client. This means that client-side objects require some extra work to be done in your game code.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Programming Guide



Engine Objects and Game Objects

Simulation object is an abstract term. There is no simulation object class in LithTech. A simulation object is composed of an engine object and a game object. These two objects, in combination, form a simulation object. Each simulation object must have both an engine object and an associated game object. Game objects can be manually or automatically instantiated. Engine objects are automatically instantiated by LithTech.

Engine Object

The engine object keeps track of a simulation object for LithTech. Engine objects define LithTech related details of simulation objects such as the object's physical manifestation and networking details. Engine objects contain information about the position, rotation, mass, velocity, acceleration, color, bounding box, and model of a simulation object. Engine objects are defined and instantiated by the engine code. The engine code for the engine objects resides on the server and on the client.

Game Object

The game object defines the customized logical behavior of a simulation object. The code implementing such logic is written by you. For example, while the engine object defines the position of a lever, your game object defines the results of a player pulling that lever. Game objects also include the logic defining how and when doors open and close, the path a guard takes on patrol, what causes the dragon to awaken, and so on. All game objects derive from the `BaseClass` class defined in the `ltengineobjects.h` file. The game code for game objects resides in your server shell implementation in `Object.lto`.

Messaging

LithTech provides communication from LithTech to game objects using a variety of event notification message types (the `MID_*` defines) and the `BaseClass::EngineMessageFn`. Communications from game object to game object are transmitted by the `BaseClass::ObjectMessageFn` and implemented in your game code.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Programming Guide



Defining Simulation Objects

Simulation objects that you use in your game must be declared as game object classes in your game code. Game object classes must derive from the `BaseClass` class. In addition to the `BaseClass` functions and variables, your inherited class can implement all the customized functionality that your game requires.

Engine objects have a type, as defined by the `OT_*` defines in `ltbasedefs.h`. Game objects must also define both messaging functions: `EngineMessageFn` and `ObjectMessageFn`. All simulation objects must be initialized using the `BEGIN_CLASS/END_CLASS` macros. This also publishes the object to DEdit. Simulation objects can have associated aggregates, which can be used to provide generic functionality across different objects. Finally, simulation objects automatically have universal properties and may have custom properties.

This section contains the following simulation object topics:

- [Object Types](#)
- [Volume Effects](#)
- [EngineMessageFn Function](#)
- [ObjectMessageFn Function](#)
 - [Aggregates](#)
- [Simulation Object Properties](#)

Object Types

Each engine object has a type. A type gives LithTech information on how to handle the engine object. Object types are defined by the `OT_*` defines. The type is part of the `ObjectCreateStruct` used to create the engine object. LithTech supports the following object types:

Camera (OT_CAMERA)—Camera objects define specific views with position and rotation in the world.

Canvas (OT_CANVAS)—A canvas object is an object that is rendered by the game code, not the engine code. For example, canvases can be used in conjunction with the **ILTDrawPrimitivesInterface** and the **LT_VERT***, **LT_LINE***, and **LT_POLY*** structures.

Container (OT_CONTAINER)—A container object defines a volumetric space. Other objects can be tested to determine whether or not they are within the container.

Dynamic light (OT_LIGHT)—A light object defines a radial or directional light located at a specific point in the world.

Invisible object (OT_NORMAL)—Normal objects are the most basic types of objects. Clients are often normal objects.

Line system (OT_LINESYSTEM)—A line system object is composed of many lines, all relative to a specific origin.

Model object (OT_MODEL)—A model object defines a model, such as a player, a monster, a gun, and so on.

Particle system (OT_PARTICLESYSTEM)—A particle system object defines a procedurally generated system that consists of textured particles.

Poly grid (OT_POLYGRID)—A polygrid object defines a moveable, dynamic grid of polygons. This can be used to simulate waving flags or curtains, shimmering forcefields, or water surfaces.

Sprite (OT_SPRITE)—A sprite object defines a two-dimensional image that may or may not be animated.

WorldModel (OT_WORLDMODEL)—A world model object defines an object in worldspace that possesses physics data. Doors, windows, and destructible walls are examples of worldmodels. WorldModels are added in DEdit and have brushes bound to them that define their structure.

Volume Effect (OT_VOLUMEEFFECT)—A volume effect. The object defines an area in which some type of volumetric effect transpires, such as snow.

[Top](#)

Volume Effects

The **OT_VOLUMEEFFECT** object type is used for generic effect rendering by game code. An object of this type basically defines a volume in which an effect will take place. Currently the only type of volume effect is the Dynamic Particles type. This effect basically passes a vertex buffer to the client side game code, which fills in the vertex information, as well as some optional lighting information. The best example within the TO2 code base is the SnowFX.cpp file in ClientShell.

Creating Volume Effects

Your client code creates a client-side object just as you would for any other object, of type **OT_VOLUMEEFFECT**. The key flags are **FLAG_VISIBLE** and **FLAG2_FORCETRANSLUCENT**. The former is used to determine if the volume effect should be updated and rendered. The latter renders the triangles with AlphaRef=0 and no Z Buffering. Both of these flags can be changed every frame if needed.

After creating the object, the volume effect specific information needs to be set. This is done through the **ILTClient::SetupVolumeEffect** function defined in **iltclient.h**.

```
LTBOOL (*SetupVolumeEffect)(HLOCALOBJ hObj, VolumeEffectInfo& info);
```

The **hObj** parameter is the **OT_VOLUMEEFFECT** object. The **info** parameter is a populated **VolumeEffectInfo** structure.

VolumeEffectInfo

The **VolumeEffectInfo** structure is the initialization structure for volume effect objects. It is defined in **ltbasedefs.h**. Every volume effect contains the same two pieces of information, the effect type (**m_EffectType**), and the dimensions of the effect (**m_Dims**).

Currently, the only valid setting for **m_EffectType** is **kDynamicParticles**. The **kUninitialized** value will result in nothing being rendered.

The following data must be set when calling **SetupVolumeEffect** on a Dynamic Particle volume effect (**VolumeEffectInfo.m_EffectType=kDynamicParticles**):

m_DPUpdateFn—A function that fills in the vertex buffer. The function is of this form:

```
bool VolumeEffectDPUpdateFn(
    void* pUser,
    void* pVertexBufferData,
    void* pLightingData,
    uint32 nVBNumVerts,
    uint32 nPrevFilledVerts,
    uint32& nNumFilledVerts
);
```

pUser is set to the value the client passed in **m_DPUserData**. **pVertexBufferData** is a pointer to a locked, write only (!) vertex buffer. **pLightingData** is a pointer to an array of information that should be set if lighting is something other than **kNone**. **nVBNumVerts** is the number of free vertices in the vertex buffer. **nPrevFilledVerts** is the number of vertices already filled this frame. **nNumFilledVerts** should be set to the number of vertices (less than or equal to **nVBNumVerts**) that the game code filled in the vertex buffer.

The passed vertex buffer may not be large enough to store all of the particles in the particle system. In this case the update function should return false until it is done filling the vertex buffer. This is where the **nPrevFilledVerts** is useful to know.

m_DPUserData—User data pointer passed back to the update function.

m_DPPrimitive—Primitive type to be used for rendering. If this is set to type **kTrilist**, then the update function should fill in all three vertices in sequence for each particle in the particle system. (Note that UV wrapping is set to clamp when rendering these tris, so you can treat the tri as a quad with overhangs) If the primitive type is **kQuadlist**, then the update function should fill in four vertices in sequence for each particle in the particle system.

m_DPLighting—Lighting type to be used. If this is set to **kNone**, then the update function needs to fill in the vertex color information manually. If **m_DPLighting** is **kSinglePointNonDirectional**, then the update function should place the light sample point into **pLightingData** (one point per primitive, not per vertex) as well as the alpha to use for the particle.

m_DPLightConstant—Constant light value to be used if lighting is **kConstant**.

m_DPSaturate—True if saturated lighting (**MODULATE2X**) should be used.

m_DPTextureName—Filename of the texture to be used for this particle system.

[Top](#)

EngineMessageFn Function

BaseClass::EngineMessageFn is an event notification function that receives calls from LithTech. All classes derived from BaseClass must define this function.

```
uint32 EngineMessageFn(
    uint32 messageID,
    void *pData,
    LTFLOAT IData
);
```

LithTech calls this function with three parameters: a message identifier (**messageID**), a pointer to data appropriate to the message (**pData**), and a float appropriate to the message (**IData**). Only the first parameter is required, the other two are provided only if the message requires them. The return value of this function may be used. It is dependant on the particular message.

EngineMessageFn receives predefined message types from LithTech (the **messageID** parameter). These messages are the **MID_*** defines and are defined in the `iltbaseclass.h` header file:

```
MID_PRECREATE
MID_INITIALUPDATE
MID_UPDATE
MID_TOUCHNOTIFY
MID_LINKBROKEN
MID_MODELSTRINGKEY
MID_CRUSH
MID_LOADOBJECT
MID_SAVEOBJECT
MID_AFFECTPHYSICS
MID_PARENTATTACHMENTREMOVED
MID_GETFORCEUPDATEOBJECTS
MID_ACTIVATING
MID_DEACTIVATING
```

At the end of your implementation of **EngineMessageFn** you should call the **EngineMessageFn** function of the parent class, passing in the same parameters received from LithTech. This allows behaviors to be inherited down the class hierarchy. When the notification reaches the **BaseClass::EngineMessageFn**, it will handle any aggregates (see [Aggregates](#) below).

[Top](#)

ObjectMessageFn Function

BaseClass::ObjectMessageFn handles messages sent from game object to game object. All such messages are defined by the game code, not the engine. Classes derived from **BaseClass** should define this function if they will receive messages from other game objects.

```
uint32ObjectMessageFn(
    HOBJECT hSender,
    uint32 messageID,
    HMESSAGEREAD hRead
```

```
);
```

At the end of your implementation of **ObjectMessageFn** you should call the **ObjectMessageFn** function of the parent class, passing in the same parameters received from LithTech. This allows object behaviors to be inherited down the class hierarchy.

[Top](#)

Aggregates

Game objects may have aggregates. An aggregate is a class that extends the functionality of a game object. When a game object receives an event notification from LithTech or another game object (through the **EngineMessageFn** or **ObjectMessageFn**), the **BaseClass** forwards that message on to all associated aggregates.

Aggregates are useful for enabling functionality common to several different game objects. For example, monsters, players, and vehicles can be damaged in a game. Instead of implementing damage logic in each of the monster, player, and vehicle game object classes, you can create a damage aggregate and associate instances of it to the monster, player, and vehicle instances. You can implement your damage logic within the damage aggregate and communicate important information back to the associated game object.

Aggregates have unique advantages over inheritance. They can be added and removed dynamically and they can be flexibly included in multiple inheritance trees without conflict.

Associating Aggregates with Objects

Aggregates are associated with and disassociated from a game object using the **AddAggregate** and **RemoveAggregate** functions of the **BaseClass** class. These functions are defined in the `ltengineobjects.h` header file.

```
void AddAggregate(LPAGGREGATE pAggregate);
```

```
LTBOOLRemoveAggregate(LPAGGREGATE pAggregate);
```

Each of these functions takes an **LPAGGREGATE**, a pointer to an **IAggregate** instance, defined in `ltbasedefs.h`. The **AddAggregate** function associates an aggregate instance with a game object instance by prepending it to the game object's linked list of aggregates. The **RemoveAggregate** function removes an association between a game object and an aggregate by removing it from the list.

One instance of an aggregate can be associated with only one game object. If another game object requires the same aggregate functionality, another aggregate instance must be created and associated with the second game object.

The IAggregate Class

The **IAggregate** class receives notifications from the associated game objects. It is defined in the `iaggregate.h` header file. In addition to its constructor and destructor, the **IAggregate** class has two pre-defined functions and one member variable.

Both the **IAggregate::EngineMessageFn** and **IAggregate::ObjectMessageFn** receive notifications from the associated object. The associated object forwards the event notifications that it receives to the aggregate's **EngineMessageFn** and **ObjectMessageFn**. All your **IAggregate**-derived classes must implement these two functions.

Note: The **IAggregate** base class does not implement any functionality beyond receiving notifications and maintaining pointers to the next aggregate in the linked list. If you use an aggregate, you must add your desired logic to an **IAggregate**-derived class of your own creation.

The following example code shows how **EngineMessageFn** and **ObjectMessageFn** might be implemented.

```
uint32 IAggregate::EngineMessageFn(
    LPBASECLASS pObject,
    uint32 messageID,
    void *pData,
    TFLOAT lData);

uint32 IAggregate::ObjectMessageFn(
    LPBASECLASS pObject,
    HOBJECT hSender,
    uint32 messageID,
    HMESSEAGEREAD hRead);
```

In addition to message type (**messageID** parameter) and data information (**pData** and **lData** or **hRead**), these functions also receive a pointer back to the associated game object (**pObject**). **ObjectMessageFn** also expects a handle (**hSender**) to the game object that originated the message.

The **IAggregate::m_pNextAggregate** member variable is a pointer to the next **IAggregate** instance in the linked list of aggregates associated with an object. When an aggregate is associated with an object, the aggregate instance is prepended to the beginning of the linked list.

The **IAggregate** class provides an interface to engine and object notifications and maintains a pointer to the next aggregate, but does not provide any other functionality. When you use aggregates, you must derive your own class from **IAggregate** and extend it with the appropriate logic. For example, if you create a **CDamageAggregate** class, you could add death and unconsciousness logic to the notification functions.

Aggregate Example

A **BaseClass**-derived class, **CPlayerObject**, is defined. All **CPlayerObject** instances (and possibly other game objects) are subject to damage, encumbrance, and experience. Instead of incorporating this functionality directly into the **CPlayerObject** class, three aggregate classes are created: **CDamageAggregate**,

CEncumbranceAggregate, and **CExperienceAggregate**.

In order to associate these aggregates with the **CPlayerObject**, **AddAggregate** must be used, most likely in the **CPlayerObject** constructor. The aggregates are added in the following order: **CExperienceAggregate**, **CEncumbranceAggregate**, and then **CDamageAggregate**.

The **CPlayerObject** now has three associated aggregates, as shown in the chart below.



The order in which the aggregates are listed in the aggregate list determines in what order they receive notifications from the game object. In this case, **CPlayerObject** forwards notifications to **CDamageAggregate**, then to **CEncumbranceAggregate**, and then to **CExperienceAggregate**.

The aggregates can also be associated with other game objects. For example, **CDamageAggregate** instances can also be associated with **CMonsterObject**, **CVehicleObject**, or **CBarricadeObject**.

[Top](#)

Simulation Object Properties

A simulation object can have numerous properties. Simulation object properties define specific attributes, such as position, color, filename, and so on. There are two different kinds of properties: universal and custom.

Properties can be accessed and modified before and during the creation of a simulation object. After a simulation object has been created, you can access and modify the member variables of the simulation object's game object and engine object.

Universal Properties

Universal properties are those properties that are handled by LithTech and are applied to the engine object. LithTech manages all universal properties. Your code can access and modify universal properties, but LithTech handles the logic behind them.

The **ObjectCreateStruct** structure is used to set values for the universal properties.

The universal properties include the following **ObjectCreateStruct** member variables:

```
m_ClassName
m_ContainerCode
m_CreateFlags
m_fDeactivationTime
m_Filename
m_Flags
m_Flags2
```

the following **m_flags** are supported:

```
FLAG_VISIBLE
FLAG_SHADOW
FLAG_ROTATEABLESPRITE
FLAG_RAYHIT
```

FLAG_SOLID

FLAG_TOUCH_NOTIFY

FLAG_GRAVITY

the following **m_flags2** are supported:

FLAG2_CHROMAKEY

m_Name

m_NextUpdate

m_ObjectType

m_Pos

m_RenderStyleNames

m_Rotation

m_Scale

m_SkinNames (for non-model objects)

m_SkinNames (for model objects)

m_UserData

These are the only properties that are automatically assigned to the engine object by LithTech. All other properties are called custom properties.

For manually created simulation objects, you set the universal property values by instantiating an **ObjectCreateStruct** structure. You assign values to the structure's member variables and then pass the structure to the **ILTServer::CreateObject** function. The universal properties are applied to the engine object.

For automatically created simulation objects, including those placed in a world using DEdit, you can access the universal properties in two ways:

- Prior to engine object instantiation, your game code can read and modify universal properties by using the **ObjectCreateStruct** pointer received in the **MID_PRECREATE** notification to the game object's **EngineMessageFn**.
- After an engine object is instantiated the **ObjectCreateStruct** longer exists. Instead your game code can read and modify the engine object's member variables by using the **HOBJECT** handle received in the **MID_INITIALUPDATE** notification to the game object's **EngineMessageFn**. The following functions use the **HOBJECT** handle to access engine object member variables, defined in `iltserver.h`, `iltclient.h`, and `ILTCommon.h`:

ILTServer::GetObjectPos

ILTServer::GetObjectRotation

ILTServer::SetObjectPos

ILTServer::SetObjectRotation

ILTClient::GetObjectFlags

ILTClient::GetObjectPos

ILTClient::GetObjectRotation

ILTClient::SetObjectFlags

ILTClient::SetObjectPos

ILTClient::SetObjectPosAndRotation

ILTClient::SetObjectRotation

ILTCommon::GetObjectFlags

ILTCommon::SetObjectFilenames

ILTCommon::SetObjectFlags

The **ILTServer** functions should be used on server-side simulation objects. The **ILTClient** functions are for client-side objects. The **ILTCommon** functions are for either client or server-side objects.

Custom Properties

Custom properties are properties defined by the game code and applied to the game object. Custom properties do not interact with the engine object.

Custom properties set by the level designer in DEdit are not automatically applied to the game object instance because the engine object does not recognize custom properties. Your game code must retrieve and apply custom properties available to the game object. To do this, use the **ILTServer::GetProp*** functions, defined in `iltserver.h`:

GetPropBool

GetPropColor

GetPropFlags

GetPropGeneric

GetPropLongInt

GetPropReal

GetPropRotation

GetPropRotationEuler

GetPropString

GetPropVector

Note: These functions only operate within the MID_PRECREATE notification to the game object's EngineMessageFn. That is, you can only call them successfully in the EngineMessageFn during a MID_PRECREATE notification.

Your game code can retrieve custom properties of various data types using these functions. For example, the **GetPropBool** function retrieves the value of a Boolean custom property.

```
LTRESULT (*GetPropBool)(
    char *pPropName,
    LTBOOL *pRet
);
```

The **pPropName** parameter is a string identifying the custom property that you want to retrieve. The **pRet** return parameter is a pointer to the Boolean value of the property. If the game object does not support the property identified by the **pPropName** parameter, the function returns **LT_NOTFOUND**. The other **GetProp*** functions operate in a similar manner.

For each of a game object's custom properties, your **EngineMessageFn** should include a **GetProp*** retrieval block in the **MID_PRECREATE** case. Within this retrieval block, you can store the custom property value for use by your game code, perhaps by assigning them to game object member variables. The custom properties are only accessible when the game object receives the **MID_PRECREATE** message, so be sure to save any properties that you will need later.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

Programming Guide



Publishing Objects to DEdit

All objects must be initialized using the **BEGIN_CLASS** and **END_CLASS*** macros (defined in `ltserverobj.h`). These macros initialize an object in LithTech and instruct DEdit to include the object in its various menus and dialog boxes. Universal properties are automatically exposed to DEdit. The optional property macros, **ADD_*PROP** and **ADD_*PROFLAG**, define which custom properties are exposed to DEdit for modification.

Note: To prevent an object from appearing in DEdit menus, use the **CF_HIDDEN** flag with the **END_CLASS_DEFAULT_FLAGS** macro.

This section contains the following topics related to publishing objects to DEdit:

- [Adding a Macro Block](#)
- [Adding Custom Properties](#)
- [Publishing to DEdit Example](#)

Adding a Macro Block

The following code example shows a typical macro block:

```
BEGIN_CLASS (Classname)

    [add_properties]

    ...

END_CLASS_DEFAULT (Classname, Parentclass, NULL, NULL);
```

This macro block is traditionally placed at the beginning of the .cpp file for a game object class.

The **BEGIN_CLASS** macro takes the name of the game object class to be published to DEdit.

The **END_CLASS_DEFAULT** macro takes the name of the game object class, the name of the game object's parent class, and two null values (These are holdovers from the original C version of LithTech, where you needed to provide the two required functions. In the current C++ version of LithTech, they are already defined as you derive from **BaseClass** at your top level.)

Optionally, you can use the **END_CLASS_DEFAULT_FLAGS** macro instead of **END_CLASS_DEFAULT**. This allows you to add a combination of **CF_** flags

after the second NULL.

CF_HIDDEN—Instances of the class cannot be placed in DEdit.

CF_NORUNTIME—This class does not get used at runtime. LithTech will not instantiate these objects out of the world file.

CF_STATIC—This is a special class that the server creates an instance of at the same time that it creates the server shell. The object is always around. This can be used as an alternative to adding code to the server shell. Using a **CF_STATIC** object can make the code easier to reuse.

CF_ALWAYSLOAD—Objects of this class and sub-classes are always loaded from the level file and cannot be saved to a save game.

Adding Custom Properties

The **BEGIN_CLASS/END_CLASS_DEFAULT** block is enough to register the object and universal properties in the DEdit menus and make them accessible to the level designer. However, if you want to give the level designer the power to modify the custom properties of the game object, you must use the **ADD_*PROP** (or **ADD_*PROP_FLAG**) macros inside the block.

The **ADD_*PROP** macros add a custom property with default values to the DEdit interface for the object.

ADD_REALPROP(name, val)—Adds a real floating point property named name, with val as the default.

ADD_STRINGPROP(name, val)—Adds a string property named name, with val as the default.

ADD_VECTORPROP(name)—Adds a vector property named name. No default.

ADD_VECTORPROP_VAL(name, defX, defY, defZ)—Adds a vector property named name, with the defX, defY, and defZ default X, Y and Z values.

ADD_LONGINTPROP(name, val)—Adds a long integer property with name name, and default value val.

ADD_ROTATIONPROP(name)—Adds a rotation property named name

ADD_BOOLPROP(name, val)—Adds a Boolean property named name, with val default setting.

ADD_COLORPROP(name, valR, valG, valB)—Adds a color property named name, with the default R, G and B values specified.

ADD_OBJECTPROP(name, val)—Adds an object property, a link to another object where val is the name of the object.

To affect how DEdit displays the custom properties on a given object, use the **ADD_*PROP_FLAG** macros instead of the **ADD_*PROP** macros. With these, you can append a combination of **PF_* flags**.

PF_HIDDEN—The property does not show up in DEdit.

PF_RADIUS—The property is a number to use as radius for drawing circle. There can be more than one.

PF_DIMS—The property is a vector to use as dimensions for drawing a box. There can be only one.

PF_DISTANCE—This property defines a measurement or other value that is in, or relative to, world coordinates. If this flag is specified, any scaling done to the world as a whole for unit conversion will also be applied to this property.

PF_FIELDOFVIEW—The property is a field of view.

PF_LOCALDIMS—Causes DEdit to show dimensions rotated with the object. Used with **PF_DIMS**.

PF_GROUPOWNER—This property owns the group it is in.

PF_GROUP1...6—This property is in group 1-6. This is used for grouping properties into subscreens.

PF_FOVRADIUS—If **PF_FIELDOFVIEW** is set, this defines the radius for it.

PF_OBJECTLINK—If the object is selected, DEdit draws a line to any objects referenced by name in **PF_OBJECTLINK** properties. It will not draw any more than **MAX_OBJECTLINK_OBJECTS**.

PF_FILENAME—This indicates to DEdit that a string property is a filename in the resource.

PF_BEZIERPREVTANGENT—This vector property changes draws an object's path from this vector to the previous vector as a bezier curve.

PF_BEZIERNEXTTANGENT—This vector property changes draws an object's path from this vector to the next vector as a bezier curve.

PF_STATICLIST—This string property has a populatable combo box with dropdown-list style (that is, a list box with no edit control).

PF_DYNAMICLIST—This string property has a populatable combo box with dropdown style (that is, a list box with edit control).

[Top](#)

Publishing to DEdit Example

The following code example publishes the **CNamelessThug** object to DEdit. It exposes two custom properties: **Model** and **Suspicious**.

```
BEGIN_CLASS( CNamelessThug )
    ADD_STRINGPROP_FLAG( Model, "", PF_FILENAME )
    ADD_BOOLPROP( Suspicious, LTFALSE )
```

```
END_CLASS_DEFAULT( CNamelessThug, BaseClass, NULL, NULL )
```

[Top](#)

Programming Guide

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.



Creating Simulation Objects

Simulation objects are created in two ways, manually in your game code and automatically by LithTech. You can manually create objects in your game code using **LTServer::CreateObject**. When the LithTech engine loads a world, it automatically creates simulation objects placed by level designers using DEdit.

This section contains the following two methods for creating simulation objects:

- [Manual Creation](#)
- [Automatic Creation](#)

Manual Creation

You can manually create a simulation object in your game code using the **LTServer::CreateObject** function (defined in `iltserver.h`). Before calling this function you must define the simulation object class using the **BEGIN_CLASS/END_CLASS** macros, as explained in the [Publishing Objects to DEdit](#) section.

```
LPBASECLASS (*CreateObject)(
    HCLASS hClass,
    ObjectCreateStruct *pStruct
);
```

The **hClass** parameter is a handle to the simulation object definition as set with the **BEGIN_CLASS/END_CLASS** macros. You can retrieve the **HCLASS** of a simulation object using the **LTServer::GetClass** function.

The **pStruct** parameter is an **ObjectCreateStruct** that you have instantiated and populated with the desired values for the engine object. After simulation object creation, you can access and modify the engine object's member variables using the **LTServer::GetObject*** and **SetObject*** functions.

The function returns a **BaseClass** pointer to the newly instantiated game object.

Automatic Creation

The simulation objects placed in a world by a level designer are automatically created by the engine when a world is loaded. Only those simulation objects placed in the world using DEdit are automatically created. Most simulation objects in a level are automatically created.

At a high level, a simulation object is automatically created as follows:

1. Using DEdit, a level designer adds a game object to a world.
2. DEdit stores the simulation object information in an .LTA file.
3. Processor.exe converts the .LTA file to a .DAT file. There is a .DAT file for each world.
4. When a world is loaded, LithTech.exe reads the .DAT file and instantiates the required game objects and engine objects and associates them with one another.

At a programmatic level, automatic creation of a simulation object proceeds as follows:

1. LithTech instantiates a new game object.
2. An **ObjectCreateStruct** is instantiated. LithTech assigns values to the structure's member variables based on the simulation object's universal properties set in DEdit by the level designer.

Important: Only the universal properties are automatically assigned by LithTech. Custom properties are not a part of the **ObjectCreateStruct**. If your simulation object includes any custom properties, your **EngineMessageFn** game code must read them manually. For more information about custom properties, see [Custom Properties](#) and [Adding Custom Properties](#).

3. LithTech sends a **MID_PRECREATE** notification to the game object's **EngineMessageFn** function. This notification includes a pointer to the **ObjectCreateStruct**. Your game code can use this pointer to read and modify the structure, if desired. You can use the **LTServer::GetProp*** functions to acquire any custom property values and apply them to the game object.
4. LithTech instantiates an engine object.
5. LithTech applies the universal properties from the **ObjectCreateStruct** to the engine object.
6. LithTech sends a **MID_INITIALUPDATE** notification to game object's **EngineMessageFn**. The game object's **m_hObject** parameter is set to point to the engine object (thereby associating the engine object with the game object).

Note: The third parameter of the **MID_INITIALUPDATE** notification to the **EngineMessageFn** is the **INITIALUPDATE_WORLDFILE** flag. This indicates that the engine object and game object instance are created based on a .DAT file.)

7. The game object can now access and modify the associated engine object using the **m_hObject** handle.

Note: The **LTServer::SetNextUpdate** function can be used to specify that the game object should receive a **MID_UPDATE** notification to the **EngineMessageFn** after a period of time. **SetNextUpdate** can be called at the end of each **MID_UPDATE** notification to receive updates regularly.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Programming Guide



Using Simulation Objects

Interaction with simulation objects after they have been created is accomplished in two ways. The game code can access the engine object member variables using the **HOBJECT** handle. The game code can access the game object member variables using the **BaseClass** derived game object.

Using the Engine Object

There are numerous API functions in **ILTServer**, **ILTModel**, **ILTCommon** and other interfaces that modify engine object member variables using **HOBJECT**s.

The **ILTServer::GetObject*** and **SetObject*** functions access and modify engine object variables. For example, **ILTServer::SetObjectPos** changes the location of an object.

Using the Game Object

The game object is defined by your code; as such, you define its use. It may expose public functions that are called by other objects, or it may interact through object-to-object messages.

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Programming Guide



Client-side Objects

LithTech implements simulation objects in the server shell. Simulation objects do not exist in the client shell because the **BaseClass** class is not supported there. However, you can emulate simulation objects by creating client-side objects in your game code in the client shell.

Client-side objects are not complete simulation objects because they do not include the game object portion of the simulation object, since the **BaseClass** class is not supported on the client. This means that client-side objects require some extra work to be done in your game code.

You should only create client-side objects for items that exist solely on the client. Such things as menus, HUD displays, and targeting crosshairs are good candidates for client-side objects.

This section contains the following topics related to the creation of client-side objects:

- [Defining Client-side Objects](#)
- [Creating Client-side Objects](#)
- [Using Client-Side Objects](#)

Defining Client-side Objects

Client-side objects are not derived from the **BaseClass** class. Your game code must define a client-side object from scratch.

Client-side objects do not have access to the **EngineMessageFn** or **ObjectMessageFn** notification functions. Aggregate functionality is also not available to client-side objects. Your implementation of a client-side object may or may not replicate some or all of this functionality based on the requirements of the client-side object.

Client-side objects do benefit from client-side engine objects. Universal properties are available to them.

[Top](#)

Creating Client-side Objects

You can create a client-side object using the **ILTClient::CreateObject** function, defined in **iltclient.h**.

```
HLOCALOBJ (*CreateObject)(
    ObjectCreateStruct *pStruct);
```

The **pStruct** parameter is an **ObjectCreateStruct** that is used to instantiate a client-side engine object.

The function returns a handle to the newly instantiated client-side engine object. You can use this handle to access the engine object's member variables.

Since **BaseClass** is not available on the client, there are no object-centric notification functions, such as **EngineMessageFn** or **ObjectMessageFn**. You can manually

replicate this functionality using the **IClientShell::Update** notification function with appropriate object cases within it.

[Top](#)

Using Client-Side Objects

You can access and modify the client-side engine object's member variables using the **ILTClient::GetObject*** and **SetObject*** functions.

ILTClient::GetObjectFlags

ILTClient::GetObjectPos

ILTClient::GetObjectRotation

ILTClient::SetObjectFlags

ILTClient::SetObjectPos

ILTClient::SetObjectPosAndRotation

ILTClient::SetObjectRotation

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Programming Guide



Audio

The LithTech engine supports a wide range of sound manipulation and modification functionality. This functionality varies depending on the target hardware and platform.

The **ILTSoundMgr** class provides the primary sound functions that you will use in your game. Among other things, it allows you to determine the specific 3D sound providers available on the system, initialize the sound engine, initialize sound-related structures, play sounds, and alter sound properties.

LithTech uses DirectSound. The dx8.snd sound driver should be placed in the same directory as Lithtech.exe.

The **ILTSoundMgr** interface is instantiated by LithTech. You can acquire a pointer to it using the Interface Manager's `define_holder_to_instance` macro:

```
define_holder_to_instance(ILTSoundMgr, g_pLTSSoundMgr, Server);
```

Before you can play sounds in your game you will need to determine whether or not to support 3D sound providers and initialize the sound engine. After this has been done, you can play sounds by preparing a sound structure and calling the **ILTSoundMgr::PlaySound** function.

By default, sounds are automatically deleted once they have been played. However, you have the option of keeping a handle to the sound. In this case, LithTech does not delete the sound and you can alter the sound in many ways. When you are done, delete the sound using the **ILTSoundMgr::KillSound** function.

This section discusses the following audio topics:

Audio Topics

[3D Sound Providers](#)

[Initializing the Sound Engine](#)

[About Playing Sounds](#)

[Playing Sound Code Examples](#)

[Using HLTSOUND](#)

[Client-Side vs. Server-Side](#)

[Looping Sounds With Markers](#)

[Compressed Sounds](#)

Description

LithTech provides two choices for 3D sound, DirectX Hardware and DirectX Software.

Before you can play any sounds in your game you must initialize the LithTech sound engine using the **InitSoundInfo** structure and the **ILTSoundMgr::InitSound** function.

For each sound that your game will play you must prepare a **PlaySoundInfo** structure and pass it to the **ILTSoundMgr::PlaySound** function.

Provides three examples of how to play sound using Jupiter.

The LithTech engine automatically deletes sounds after they are played unless you specifically instruct LithTech not to do so by requesting an **HLTSOUND** handle to the sound. If you have an **HLTSOUND** handle to a sound, you can alter the sound and/or play it again without repopulating the **PlaySoundInfo** structure.

Both server-side and client-side sound controls are supported by the LithTech engine.

The LithTech engine supports markers for looping sounds.

Information about using compressed sounds with the LithTech engine.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Programming Guide



3D Sound Providers

LithTech supports DirectX 8 DirectSound. To be used, the dx8.snd file must be in the current working directory of the game (the same folder as your game's lithtech.exe file). The dx8.snd file is provided with the LithTech installation and is located in the \development\TO2\ folder.

It is not possible to mix and match hardware and software 3D sounds in Jupiter. Your game must choose one or the other when it initializes the sound engine as explained below. If using 3D hardware sound you can only use mono sounds.

You can use the **ILTClientSoundMgr::GetSound3DProviderLists** function(defined in `iltclientsoundmgr.h`) to determine if your sound card can support the number of hardware voices your game requires.

```
LTRESULT ILTClientSoundMgr::GetSound3DProviderLists(
    Sound3DProvider *&pSound3DProviderList,
    LTBOOL bVerify
);
```

This function returns a linked list (the **pSound3DProviderList** out-parameter) of Sound3DProvider structures identifying the providers available on the system. If the bVerify parameter is set to true, the pSound3DProviderList only includes providers that are completely supported by the system.

The **uiMax3DVoces** parameter lets you pass in a baseline number of hardware voices. If the soundcard does not meet this level then hardware 3D is not available. TO2 uses 32 voices. SBLive and newer cards support this level, so it's a reasonable setting. It's important that the baseline value is bigger than the number of hardware voices requested in **m_nNum3DVoces** parameter of the **SoundInfo** structure passed to the sound engine **ILTClientSoundMgr::InitSound** function, if the game wants to use hardware 3D.

This verification can take several seconds and may cause speaker popping.

To release the list when you are finished with the it, call **ILTClientSoundMgr::ReleaseSound3DProviderList** with the first item of the list.

The Sound3DProvider structure contains information about a 3D sound provider. It is defined in `ltbasedefs.h`.

```
struct Sound3DProvider
{
    uint32      m_dwProviderID;
    char        m_szProvider[_MAX_PATH+1];
    uint32      m_dwCaps;
    Sound3DProvider * m_pNextProvider;
};
```

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Programming Guide



Initializing the Sound Engine

You must initialize the LithTech sound engine on the client to add support for sound. This section discusses the following topics involving initialization of the sound system:

- [Initializing the Sound Engine](#)
- [Using the InitSoundInfo Structure](#)
- [Using the InitSound Function](#)
- [Example Sound Initialization Code](#)

Initializing the Sound Engine

Initialization of the LithTech sound engine typically occurs via the game's **IClientShell::OnEngineInitialized** function. You cannot play any sound through LithTech until the sound engine is initialized.

To initialize the sound engine

1. Create an [InitSoundInfo](#) structure and fill it out with the options your game will use.

2. Call [ILTClientSoundMgr::InitSound](#).

[Top](#)

Using the InitSoundInfo Structure

The **InitSoundInfo** structure identifies a 3D sound provider. It also defines several options and settings that determine how sounds are played in your game. The structure is defined in ILTSoundMgr.h.

```
struct InitSoundInfo
{
    char            m_sz3DProvider[_MAX_PATH+1];
    uint8           m_nNumSWVoices;
    uint8           m_nNum3DVoices;
    unsigned short   m_nSampleRate;
    unsigned short   m_nBitsPerSample;
    unsigned long    m_dwFlags;
    unsigned long    m_dwResults;
    unsigned short    m_nVolume;
    float            m_fDistanceFactor;
    float m_fDopplerFactor;
    inline void Init();
};
```

The value for **m_nNum3DVoices** must not exceed the value for **uiMax3DVoices** passed in to the **ILTClientSoundMgr::ReleaseSound3DProviderList** function.

[Top](#)

Using the InitSound Function

After creating an **InitSoundInfo** structure, pass it in to the **ILTClientSoundMgr::InitSound** function to initialize a sound driver. The **InitSound** function is defined in the ILTSoundMgr.h file.

```
ILTClientSoundMgr::InitSound(
    InitSoundInfo *pSoundInfo
);
```

After successfully calling this function, the game code can play sounds.

[Top](#)

Example Sound Initialization Code

In this example, a global **ILTClientSoundMgr** pointer is created near the top of ClientInterfaces.cpp.

```
ILTClientSoundMgr* g_pLTCSoundMgr = NULL;
```

This global pointer is then assigned to LithTech's sound interface using the Interface Manager's **define_holder** macro:

```
define_holder(ILTClientSoundMgr, g_pLTCSoundMgr);
```

The **ILTClientSoundMgr** interface is used to play sounds later in the example.

The **InitSoundInfo** structure is created in the LTClientShell.cpp in the **CLTClientShell::InitSound** function:

```
LRESULT CLTClientShell::InitSound()
{
    InitSoundInfo sndInfo;

    sndInfo.Init();

    //Initialize sound with defaults.

    return g_pLTCSoundMgr->InitSound(&sndInfo);
}
```

```
}

```

For this basic example, the **Init** function is used to initialize the **InitSoundInfo** structure with default values. It is likely that your game may require a more customized **InitSoundInfo** structure.

The audio device identified in the **InitSoundInfo** structure is then initialized using the **ILTClientSoundMgr::InitSound** function. The example can now play sounds.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Programming Guide



About Playing Sounds

This section covers a variety of material related to playing sounds using Jupiter, and provides an instructions on the following topics:

- [Types of Sounds](#)
- [The PlaySoundInfo Structure](#)
- [Attaching a Sound to an Object](#)
 - [Applying Sound Filters](#)
- [Setting Sound Class Volume](#)

Types of Sounds

After the sound engine has been initialized, your game code can play sounds. There are three types of sounds: local, ambient, and 3D. The following list describes each of these three types.

- **Local**

A local sound is something that has no distance or orientation and seems to be played inside the listener's head.

- **Ambient**

An ambient sound has no orientation, but gets quieter with distance. Ambient sounds are used for things like the hum of machinery, wind, etc.

- **3D**

3D sounds have orientation (left and right speaker spatial separation) and distance.

Note: The number of sounds played in one area can affect performance.

Playing a sound is a two-step process.

1. Create a **PlaySoundInfo** structure.
2. Call **ILTSoundMgr::PlaySound**.

Both of these steps are discussed in more detail in the following topics.

[Top](#)

The PlaySoundInfo Structure

The **PlaySoundInfo** structure defines a variety of options for sounds. The options you choose can drastically change the performance of the sound rendering. This structure must be populated for each sound to be played.

To initialize the **PlaySoundInfo** struct to default values, use the **PLAYSOUNDINFO_INIT** macro.

The **PlaySoundInfo** structure is defined in the `ltbasedefs.h` file.

```
struct PlaySoundInfo
{
    uint32    m_dwFlags;
    char      m_szSoundName[_MAX_PATH+1];
    HOBJECT   m_hObject;
    HLTSOUND  m_hSound;
    unsigned char m_nPriority;
    float     m_fOuterRadius;
    float     m_flInnerRadius;
    uint8     m_nVolume;
}
```

```

float      m_fPitchShift;

LTVector   m_vPosition;

uint32     m_UserData;

uint8 m_nSoundVolumeClass;

};

```

[Top](#)

Attaching a Sound to an Object

A sound can be attached to the position of an object by setting the **PLAYSOUND_ATTACHED** flag and assigning the **m_hObject** parameter to the object to attach to. If the host object is removed before the sound ends, the sound will be stopped and deleted. If that sound also had **PLAYSOUND_GETHANDLE** set, it will still be stopped but it will not be deleted. The game must still delete the sound.

Different Sound Types for the Same Sound on Different Clients

Some sounds (such as footsteps or dialogue) sound better when they are played locally to the client that played them, but 3D to everyone else. This can be accomplished by setting the **PLAYSOUND_3D** flag to play the sound in 3D.

To play the sound locally on any given client

1. Set the **PLAYSOUND_CLIENTLOCAL** and **PLAYSOUND_ATTACHED** flags in the **m_dwFlags** parameter.
2. Assign the walking player's engine object to the **m_hObject** parameter.
3. Play the sound to all clients using **ILTSoundMgr::PlaySound**.

After creating an **PlaySoundInfo** structure, pass it in to the **ILTSoundMgr::PlaySound** function. The **PlaySound** function is defined in the **ILTSoundMgr.h** file.

```

LTRESULT PlaySound(

    PlaySoundInfo *pPlaySoundInfo,

    HLTSOUND &hResult

);

```

You pass in a pointer to the **PlaySoundInfo** structure you created in the **pPlaySoundInfo** parameter. The **hResult** out-parameter is a handle to this sound. You can also get a handle to this sound using the **m_hSound** member variable of the **PlaySoundInfo** structure. This handle is not guaranteed to be valid unless the **PLAYSOUND_GETHANDLE** flag is set in **m_dwFlags**.

[Top](#)

Applying Sound Filters

Sound filters can be applied to sounds using the **ILTSoundMgr::SetSoundFilter** and **SetSoundFilterParam** functions. Only EAX 2.0 hardware sound filters are supported (although there is code to support DX8 software filters, they are currently too slow).

To use a filter

- place a volume brush in DEdit and then assign a filter to it.

Filters are defined in the **soundfilters.txt** file (located in the **\development\TO2\game\attributes** folder). Parameters are found in **iltsound.h** in the engine. At game load time, the sound filters are loaded from the this file and then referenced by their name in the **SetSoundFilter** and **SetSoundFilterParam** functions.

Additionally, to enable/disable a sound filter, use the **ILTSoundMgr::EnableSoundFilter** function.

[Top](#)

Setting Sound Class Volume

The **ILTSoundMgr::SetSoundClassMultiplier** function can be used to set up sound classes that have a volume relative to the current global volume setting. For example, if the current global setting is .5 and you have a sound class called Weapons set to .5, all sounds with the Weapons class will play at .25 volume.

```

LTRESULT SetSoundClassMultiplier(

    uint8 nSoundClass,

    float fMult );

```

This function defines the sound class in the **nSoundClass** parameter and equates a volume multiplier with the **fMult** parameter.

When you play a sound you reference the sound class in the **nSoundVolumeClass** parameter of the **PlaySoundInfo** structure passed in to **ILTSoundMgr::PlaySound**.

[Top](#)



Playing Sound Code Examples

This section includes three different examples of playing sounds:

- [Server-Controlled Ambient Sound](#)
 - [Client-Only Local Sound](#)
 - [Client-Only 3D Sound](#)

Server-Controlled Ambient Sound

This example creates a looping, ambient sound on the server that is played on all clients.

After the world is loaded, the **IClientShell::PostStartWorld** event notification function is called by LithTech. **PostStartWorld** is a good place to set up dynamic sounds and other global events that will be played on each client. Unless the **PLAYSOUND_GETHANDLE** flag is set, the sound will be deleted when it ends.

Note: Most ambient sounds would typically be associated with a particular world, and thus they are usually created as a response to a message from the server (or more specifically, a “sound object” on the server that has been loaded from a world). **PostStartWorld** is used in this example only for convenience.

The following code illustrates how to use the implement your **PostStartWorld** object to play a sound.

```
void CLTServerShell::PostStartWorld()
{
    HLTSOUND hSound;    // dummy handle for PlaySound()

    PlaySoundInfo snd;  // sound info description

    // Initialize sound description
    PLAYSOUNDINFO_INIT(snd);

    // This sound should loop constantly. Ambient means it has
    // distance falloff, but no left/right speaker shifting.
    snd.m_dwFlags = PLAYSOUND_AMBIENT | PLAYSOUND_LOOP;

    // Specify properties of this ambient sound.
    strncpy(snd.m_szSoundName, "sounds/cruise.wav", _MAX_PATH - 1);

    snd.m_nPriority      = 250;
    snd.m_fOuterRadius   = 500.0f;
    snd.m_fInnerRadius  = 50.0f;

    snd.m_vPosition      = LTVector(0.0f, 200.0f, -500.0f);

    // Play the sound on each client. The handle is returned to
    // hSound, but because PLAYSOUND_GETHANDLE was not specified,
    // the hSound is not guaranteed to be valid.
    g_pSoundMgr->PlaySound(&snd, hSound);
}
```

First, the **HLTSOUND** variable **hSound** is declared. This variable is required for the call to **PlaySound**, which will assign an address to it. However, the code does not use it for any other reason.

A **PlaySoundInfo** structure (**snd**) is then declared and initialized (using the **PLAYSOUNDINFO_INIT** macro). The sound is set to ambient and looping by assigning the **PLAYSOUND_AMBIENT** and **PLAYSOUND_LOOP** flags to the **m_dwFlags** parameter. Several of the member variables (**m_szSoundName**, **m_nPriority**, **m_fOuterRadius**, **m_fInnerRadius**, and **m_vPosition**) are then manually reset, over-riding the default settings.

The **ILTClientSoundMgr::PlaySound** function is called on the global interface pointer (**g_pSoundMgr**) to play the sound on all the clients. Note that the **hSound** out-parameter is not guaranteed to be valid because the **PLAYSOUND_GETHANDLE** flag was not set.

[Top](#)

Client-Only Local Sound

This example plays a sound just once (non-looping) and only locally (sounding as though it were in the client’s head). This type of sound is useful for HUD and

powerup cues.

The sound is played when the View Reset key is pressed. Therefore, the **IClientShell::OnCommandOn** function supports sound in the **CMD_RESET_VIEW** value in the case statement.

```
// --- sound (begin block) ----- //

// Play a sound

HLTSOUND hSound;    // dummy handle for PlaySound()

PlaySoundInfo snd;  // sound info description

// Initialize sound description

PLAYSOUNDINFO_INIT(snd);

// Set flags to play in the client's head, and only play on this
// client. (i.e. no distance fading or left/right speaker shifts.)

snd.m_dwFlags = PLAYSOUND_LOCAL | PLAYSOUND_CLIENT;

// Specify properties of this sound.

strncpy(snd.m_szSoundName, "sound/footstep_metal.wav", _MAX_PATH - 1);

snd.m_nPriority = 254;

// Play the sound. The handle is returned to hSound,
// but because PLAYSOUND_GETHANDLE was not specified, the hSound is
// not guaranteed to be valid.

g_pLTCSoundMgr->PlaySound(&snd, hSound);

// --- sound (end block) ----- //
```

As with the server-controlled ambient sound example above, the local sound example begins by declaring the **HLTSOUND** variable **hSound**.

A **PlaySoundInfo** structure (**snd**) is then declared and initialized (using the **PLAYSOUNDINFO_INIT** macro). The sound is set to local and client-only by assigning the **PLAYSOUND_LOCAL** and **PLAYSOUND_CLIENT** flags to the **m_dwFlags** parameter. Two of the member variables (**m_szSoundName** and **m_nPriority**) are then manually reset, over-riding the default settings.

The **ILTClientSoundMgr::PlaySound** function is called on the global interface pointer (**g_pSoundMgr**) to play the sound. Note that the **hSound** out-parameter is not guaranteed to be valid because the **PLAYSOUND_GETHANDLE** flag was not set. Unless the **PLAYSOUND_GETHANDLE** flag is set, the sound will be deleted when it ends.

[Top](#)

Client-Only 3D Sound

This sound plays on the client at a 3D position. Like the ambient sound, the distance from the sound source determines the volume (given the inner and outer ranges). However, spatial separation between left and right speakers is possible with 3D sounds. For example, if the sound is to the right of the camera, most of the sound will come from the right speaker.

For this example, three looping 3D sounds are played at different areas of the room. To accomplish this, the sounds are played in the first notification to the **IClientShell::Update** function by calling the **FirstUpdate** function.

```
void CLTClientShell::FirstUpdate()

{

    g_pLTClient->CPrint("CLTClientShell::FirstUpdate");

// --- sound (begin block) ----- //

// Start music

HLTSOUND hSound;    // dummy handle for PlaySound()

PlaySoundInfo snd;  // sound info description

// Initialize sound description

PLAYSOUNDINFO_INIT(snd);

// This looping sound plays at a 3D location with distance
// falloff and left/right separation.

snd.m_dwFlags = PLAYSOUND_3D | PLAYSOUND_LOOP;
```

```

        // Specify properties of this 3D sound.

        strncpy(snd.m_szSoundName, "sound/music_loop1.wav", _MAX_PATH - 1);

        snd.m_nPriority      = 255;

        snd.m_fOuterRadius  = 500.0f;

        snd.m_fInnerRadius  = 50.0f;

        snd.m_vPosition      = LTVector(-500.0f, 200.0f, 0.0f);

        // Play the sound. The handle is returned to hSound,
        // but because PLAYSOUND_GETHANDLE was not specified, the hSound is
        // not guaranteed to be valid.

        g_pLTCSoundMgr->PlaySound(&snd, hSound);

        // Do the same for music_loop2. All that's different is filename
        // and position. The rest of snd is still intact.

        strncpy(snd.m_szSoundName, "sound/music_loop2.wav", _MAX_PATH - 1);

        snd.m_vPosition = LTVector(500.0f, 200.0f, 0.0f);

        g_pLTCSoundMgr->PlaySound(&snd, hSound);

        // Do the same for music_loop3. All that's different is filename
        // and position. The rest of snd is still intact.

        strncpy(snd.m_szSoundName, "sound/music_loop3.wav", _MAX_PATH - 1);

        snd.m_vPosition = LTVector(0.0f, 200.0f, 500.0f);

        g_pLTCSoundMgr->PlaySound(&snd, hSound);

        // --- sound (end block) ----- //
    }

```

After some object operations, the sound code begins. As with the other sound examples explained previously, the **HLTSOUND** is created and **PlaySoundInfo** structure is created and initialized. The first sound is set to 3D and looping by assigning the **PLAYSOUND_3D** and **PLAYSOUND_LOOP** flags to the **m_dwFlags** parameter. Several of the member variables (**m_szSoundName**, **m_nPriority**, **m_fOuterRadius**, **m_fInnerRadius**, and **m_vPosition**) are then manually reset, overriding the default settings.

The **ILTClientSoundMgr::PlaySound** function is called on the global interface pointer (**g_pSoundMgr**) to play the first sound. Note that the **hSound** out-parameter is not guaranteed to be valid because the **PLAYSOUND_GETHANDLE** flag was not set.

For the second sound, the same **PlaySoundInfo** structure instance is used. However, the file name (**m_szSoundName**) and position (**m_vPosition**) are changed. The **PlaySound** function is then called again to begin playing the second sound.

The third sound again uses the same **PlaySoundInfo** structure and the **PlaySound** function, changing only the filename and position.

Even though the structure is re-used, the first and second sounds are unaffected. Repeated uses of the same **PlaySoundInfo** structure do not affect sounds started previously. To alter a sound, you have to use the handle to the sound (**HLTSOUND**).

The priority level for all three of these looping sounds is the same (255). However, if you move between the sounds, you can still hear each of them if sufficient voices are available on the sound system.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.



Using HLTSOUND

This section discusses the **HLTSOUND** handle used to keep track of sounds played in your game. This section contains the following topics related to tracking and playing sounds using the **HLTSOUND** handle:

- [About HLTSOUND](#)

- [HLTSOUND Functions](#)
- [Example HLTSOUND Code](#)
 - [Tracking Variables](#)
 - [Startup Sound](#)
 - [Looping Fire Sound](#)
 - [Wind-Down Sound](#)

About HLTSOUND

The **HLTSOUND** handle is used to keep track of sounds played in your game. Unless you explicitly instruct LithTech to give you a valid **HLTSOUND** to the sound, LithTech will delete the sound as soon as it expires. Also, your code will have no way to access the sound before it is deleted. Therefore, if you want to control when the sound is deleted or have the ability to alter the sound, you must acquire an **HLTSOUND** to it.

To instruct LithTech to give you an **HLTSOUND** to a sound, you must set the **PLAYSOUND_GETHANDLE** flag in the **PlaySoundInfo** structure's **m_dwFlags** member variable. If this is specified, LithTech returns a valid **HLTSOUND** in the **PlaySoundInfo** structure's **m_hSound** member variable, and LithTech will not remove the sound when it finishes playing. Subsequently, the sound can be accessed and/or modified using this handle.

If the **PLAYSOUND_GETHANDLE** flag is set as explained above, the **HLTSOUND** is also returned in the **ILTSoundMgr::PlaySound** function's **hResult** out-parameter.

Because LithTech does not automatically delete a sound if you have an **HLTSOUND** to it, you must manually do so yourself. In your game code, be sure to use the **ILTSoundMgr::KillSound** function to delete such a sound after you are through with it.

HLTSOUND Functions

Using the **HLTSOUND** to a played sound, you can use the following functions to modify the sound while it is being played or after it has expired. If you do not have an **HLTSOUND** to a sound, you have no way to access it to use these functions.

ILTCClient::GetSoundData

ILTCClient::GetSoundOffset

ILTCClient::GetSoundTimer

ILTCClient::IsDone

ILTSoundMgr functions that use an HLTSOUND handle:

GetSoundDuration

IsSoundDone

KillSound

KillSoundLoop

PlaySound

Various ILTCClientSoundMgr functions also use HLTSOUND:

GetSoundPosition

SetSoundPosition

[Top](#)

Example HLTSOUND Code

This example demonstrates the use of **HLTSOUND** functionality by implementing a gunfire sound with wind-up, looping, and wind-down sounds. The code implements this as follows:

1. When the fire button is pushed, a wind-up sound is played once. The code for this is implemented in the **CMD_ACTION1** case block of the **IClientShell::OnCommandOn** callback function.
2. If the fire button is held down, a looping gunfire sound is played. The code for this is implemented in the **CLTClientShell::PollInput** function located in the **IClientShell::Update** event notification function. The **ILTCClient::IsCommandOn** function is used to query LithTech to determine if the fire button is currently depressed.
3. When the fire button is released, a wind-down sound is played. The code for this is implemented in the **CMD_ACTION1** case block of the **IClientShell::OnCommandOff** event notification function.

This type of logic is useful for just about any moving object, such as vehicles and spinning minigun barrels.

Tracking Variables

First, the code declares a variable to track the sound.

```
HLTSOUND m_hFireSound;
```

This is the **HLTSOUND** that will be used to access the various sounds. As one sound stops and another starts, **m_hFireSound** is reassigned to the new sound.

Because the wind-up sound is a special case, the code declares a special flag that will be used to track when the wind-up sound is playing.

```
bool m_bFireWindup;
```

Both of these variables are declared in `ltclientshell.h` and initialized in `CLTClientShell`'s default constructor.

Startup Sound

The wind-up sound is played when the fire button is pressed. This is implemented in the `CMD_ACTION1` case block of the `IClientShell::OnCommandOn` event notification function.

```
case CMD_ACTION1:
{
    g_pLTClient->CPrint("CMD_ACTION1: start");

    m_bFireWindup = true;

    //

    // Play wind-up sound.

    //

    PlaySoundInfo snd; // sound info description

    // Initialize sound description

    PLAYSOUNDINFO_INIT(snd);

    // This sound will play locally in the client's head, only on the
    // client, and we want to have control over the volume levels.

    // Specifying PLAYSOUND_GETHANDLE ensures that the sound is

    // not deleted upon completion.

    snd.m_dwFlags = PLAYSOUND_LOCAL | PLAYSOUND_CLIENT | PLAYSOUND_CTRL_VOL | PLAYSOUND_GETHANDLE;

    // Specify properties of this sound.

    strncpy(snd.m_szSoundName, "sounds/fire_start.wav", _MAX_PATH - 1);

    snd.m_nPriority = 254;

    snd.m_nVolume = 40;

    // Play the sound. This time we're keeping the handle to be used later.

    g_pSoundMgr->PlaySound(&snd, m_hFireSound);
}
```

First, the `m_bFireWindup` variable is set to `TRUE` to indicate that the startup sound is being played.

A `PlaySoundInfo` structure is created and initialized. Several flags are set:

PLAYSOUND_LOCAL—Play a local sound (in the client's head).

PLAYSOUND_CLIENT—Play the sound only on this client.

PLAYSOUND_CTRL_VOL—Enable volume control.

PLAYSOUND_GETHANDLE—Do not automatically delete the sound. Return an **HLTSOUND** to the sound so the code can track it.

Three of the structure's member variables are also explicitly assigned values: `m_szSoundName`, `m_nPriority`, and `m_nVolume` (setting the **PLAYSOUND_CTRL_VOL** enables the code to control the volume of the sound). Other member variables retain their default values.

Finally, the sound is played using the `PlaySound` function. The **HLTSOUND** for the sound is returned by LithTech to the `m_hFireSound` variable. This will be used to track and query the wind-up sound.

Looping Fire Sound

The looping sound is the gunfire report of the fully automatic weapon. Therefore, the code must query LithTech to determine if the fire button is depressed. If it is, the sound keeps playing. If it isn't, the sound stops and the wind-down sound plays.

Because the example wind-up sound has a slightly abrupt transition into the looping sound, the example starts the looping sound before the startup sound is finished. This could have been avoided by editing the startup sound. However, this provides an opportunity to demonstrate other uses of **HLTSOUND**.

The looping sound code is located in the `PollInput` function in `LTClientShell.cpp` (`PollInput` is called in the `IClientShell::Update` callback function).

```
if (g_pLTClient->IsCommandOn(CMD_ACTION1))
{
    g_pLTClient->CPrint("ACT_FIRE: loop");

    // For each time you start/loop/stop the sequence of sounds, this

    // loop sound is only started ONCE, due to this if() statement.

    // This way, you don't continually start new sounds ON TOP OF each
```



```

        // other.

        if (m_bFireWakeup && m_hFireSound)
        {
            // Clear these, because if not, GetSoundOffset() won't clear
            // them if it fails.

            dwFireOffset = dwFireDuration = 0;

            // Get sound duration and current position in playing
            // buffer.

            g_pLTClient->GetSoundOffset(m_hFireSound, &dwFireOffset,
                                     &dwFireDuration);

            // Because the transition from startup sound to looping
            // sound is a little abrupt in this sample, start the
            // looping sound 90% of the way into the startup sound.

            if (((double)dwFireOffset >
                (double)dwFireDuration * double(0.9)))
            {
                // Kill the startup sound and clear our trigger
                // flags.

                g_pSoundMgr->KillSound(m_hFireSound);

                m_hFireSound = NULL;
                m_bFireWakeup = false;

                // Play looping sound

                PlaySoundInfo snd;    // sound info description

                // Initialize sound description
                PLAYSOUNDINFO_INIT(snd);

                // This sound plays in the client's head, only on the
                // client, uses volume control, preserves the sound
                // until KillSound() is called, and loops.

                snd.m_dwFlags=        PLAY_SOUND_LOCAL |
                                     PLAY_SOUND_CLIENT |
                                     PLAY_SOUND_CTRL_VOL |
                                     PLAY_SOUND_GETHANDLE |
                                     PLAY_SOUND_LOOP;

                // Specify properties of this ambient sound.

                strncpy(snd.m_szSoundName, "sounds/fire_loop.wav",
                     _MAX_PATH - 1);

                snd.m_nPriority        = 254;

                snd.m_nVolume         = 70;

                // Play the looping sound and store the handle in
                                     // m_hFireSound.

                g_pSoundMgr->PlaySound(&snd, m_hFireSound);

            }
        }
    }
}

```

The looping sound is played if the fire button is down. The code, therefore, queries LithTech to determine if this is the case (using the **ILTClient::IsCommandOn** function with the **CMD_ACTION1** type).

The code then checks to see if the wind-up sound is currently playing (**m_bFireWakeup** is **TRUE**) and makes sure that an **HLTSOUND** for the startup sound exists. If **m_bFireWakeup** is **FALSE**, nothing else is done in the **CMD_ACTION1** if block. The example will only start the looping sound if the startup sound is playing. This prevents the code from continually starting the looping sound on top of itself.

If the startup sound is playing, **ILTClient::GetSoundOffset** is used to retrieve the sound duration and current position in the playing buffer. (Note that the **HLTSOUNDm_hFireSound** handle is used in this function.) If the startup sound is less than 90% complete, nothing more is done in the **CMD_ACTION1** if block. If the startup sound is over 90% complete, the wind-up sound is killed by calling **KillSound(m_hFireSound)**, the **m_hFireSound** variable is cleared, and the **m_bFireWakeup** Boolean variable is set to **FALSE** (since the wind-up sound is no longer playing). A **PlaySoundInfo** structure is created, initialized and set with the appropriate flags and values.

Finally, the looping sound is played using the **PlaySound** function. Again, the **HLTSOUND** (the recycled **m_hFireSound** variable) is retrieved for the sound for future use. From this point, the sound is not checked or altered in any way until the fire key is released, which triggers **IClientShell::OnCommandOff**.

Wind-Down Sound

To complete the gunfire sequence, the code must play the wind-down sound when the fire button is released. This is implemented in the **CMD_ACTION1** case block of the **IClientShell::OnCommandOff** callback function (in **LTClientShell.cpp**).

```

        case CMD_ACTION1:
        {
            HLTSOUND hSound;    // dummy handle for PlaySound()

            g_pLTClient->CPrint("ACT_FIRE: stop");

            g_pLTClient->CPrint("");

            // If m_hFireSound exists, then we need to kill it first and

            // clear our trigger flags.

            if (m_hFireSound)
            {
                g_pSoundMgr->KillSound(m_hFireSound);

                m_bFireWindup = false;

                m_hFireSound = NULL;

            }

            //

            // Play wind-down sound

            //

            PlaySoundInfo snd;    // sound info description

            // Initialize sound description

            PLAYSOUNDINFO_INIT(snd);

            // This sound plays in the client's head, only on the client, and

            // with sound control.

            snd.m_dwFlags =        PLAYSOUND_LOCAL |

                                   PLAYSOUND_CLIENT |

                                   PLAYSOUND_CTRL_VOL;

            // Specify properties of this ambient sound.

            strncpy(snd.m_szSoundName, "sounds/fire_stop.wav", _MAX_PATH - 1);

            snd.m_nPriority        = 254;

            snd.m_nVolume         = 70;

            // Play the stop/wind-down sound. The handle is returned to

            // hSound, but because PLAYSOUND_GETHANDLE was not specified, the

            // hSound is not guaranteed to be valid.

            g_pSoundMgr->PlaySound(&snd, hSound);

        }

```

Because the wind-down sound is the last sound in the sequence of gunfire sounds, the code does not need to keep track of it after it has finished playing. Therefore, a dummy **HLTSOUND** is declared to be used in the **PlaySound** call at the end of the block.

Next, the wind-up or looping fire sound must be stopped. If **m_hFireSound** exists then one of the two previous sounds is playing. The code calls the **KillSound** function to stop the previous sound. Both the tracking variables, **m_bFireWindup** and **m_hFireSound**, are cleared for subsequent use.

A **PlaySoundInfo** structure is created, initialized, and set with appropriate flags and values. Because this is the last sound in the gunfire sequence, the code does not need to keep track of it. LithTech can delete the wind-down sound as soon as it is finished playing. Therefore, the **PLAYSOUND_GETHANDLE** flag is not set.

Finally, the wind-down sound is played using the **PlaySound** function.

[Top](#)



Client-Side vs. Server-Side

If a sound is played from inside the object.lto file, it is a server-side sound. If it is played within cshell.dll, it is a client-side sound. Server-side sounds are tracked on the server and any players that are within the sound's radius will hear it. Client-side sounds can only be heard by the client that played the sound. It is best to minimize the number of server-side sounds to reduce network traffic for multi-player games. Good candidates for client side only sounds are menu sounds and in-game interface sounds.

The **ILTSoundMgr** class provides functions to manage sounds from the server. The **ILTClientSoundMgr** class derives from **ILTSoundMgr** and extends that interface by giving the client-side game code the ability to access and modify sound attributes on the client.

Server-side Sound Tracking

When a server-side sound is played, it is sent to clients within the sound's outer radius. If no clients are within this distance, the server will keep track of the sound if any of the following flags are set: **PLAYSOUND_LOOP**, **PLAYSOUND_ATTACHED**, **PLAYSOUND_GETHANDLE**, **PLAYSOUND_TIMESYNC**, **PLAYSOUND_TIME**.

Tracked sounds will continue to progress in time. If a client comes within the outer radius at a later time before the sound has ended, the client will be notified about the sound and where to start playing.

Because of possible lag between a server and client, the client may play a sound from a position behind where the server is playing. To help make up for this lag, set the **PLAYSOUND_TIMESYNC** flag.

HLTSOUND on the Server

If a server-side sound is played with **PLAYSOUND_GETHANDLE**, then the sound object belongs to the game. It must be removed by the game with **ILTSoundMgr::KillSound** or **ILTSoundMgr::KillSoundLoop** (for looping sound with markers).

A sound played without the **PLAYSOUND_GETHANDLE** is handled by LithTech and removed automatically. A server-side sound that is removed either automatically or by the game will notify all clients playing the sound to remove their copies.

Example Sound Server-Side Code

The sound engine does not need to be initialized on the server because it does not actually play sounds. However, if you want to play server-controlled sounds and track or modify them you must create a sound interface for the server.

The following code can be found in ServerInterfaces.cpp. To provide server tracking of sounds, the example declares a global **ILTSoundMgr** pointer: **g_pSoundMgr**.

```
ILTSoundMgr* g_pLTSSoundMgr = NULL;

define_holder_to_instance(ILTSoundMgr, g_pLTSSoundMgr, Server);
```

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.



Looping Sounds with Markers

Looping sounds play until they reach the last sample of the sound then start over from the first sample. They continue to do this until they are killed by the game or the level is unloaded.

Note: Looping sounds with markers are not available for the PlayStation 2 platform.

There is a special kind of looping sound that can contain markers. These markers specify the loop begin sample and the loop end sample of the loop. This technique of looping a sound is good for sounds that have ramp-up and ramp-down effect, such as an elevator lift might sound like it charges up before it loops. Then when it hits its destination, it might cool down. The charge up sound would be placed before a begin loop marker and the cool down sound would be after an end loop marker.

If LithTech is told to loop a sound that contains markers, it will start the sound from the first sample. When it hits the loop end sample, it rewinds to the loop begin sample and continues playing from there. The sound is then looped between these two markers until **ILTSoundMgr::KillSound** or **ILTSoundMgr::KillSoundLoop** is called. **ILTSoundMgr::KillSound** will just stop the sound. If **ILTSoundMgr::KillSoundLoop** is called, the sound will continue to play but the next time it hits the loop end marker, it will ignore it and continue to play until the last sample of the sound. LithTech will then automatically clean up the sound. The game can discard its **HLTSOUND** after it calls either **ILTSoundMgr::KillSound** or **ILTSoundMgr::KillSoundLoop**.

Markers exist in the actual sound file and are placed using a tool such as CoolEdit®.

[Top](#)

[Send feedback regarding this page.](#)
2006, All Rights Reserved.

Programming Guide



Compressed Sounds

LithTech can play sounds in .WAV format. Compressed sounds, such as .MP3, can be played if they are embedded in a .WAV file. There are various third-party tools available to embed other file formats into the .WAV format. You will need to arrange for a license from Fraunhofer for use of the .mp3 codec (visit <http://mp3licensing.com/>).

Playing compressed sounds significantly impacts game performance. You should only use compressed sounds when CPU load is low.

The decompression works seamlessly, as long as the file is a .wav file. You can just make the normal **PlaySound** calls and the engine will decompress, if necessary. IMA-ADPCM compression is also supported.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

Programming Guide



Models and Animation

Models are a collection of geometries and animations used to create complex objects such as characters and other props, such as consoles, weapons, monsters, and vehicles. Artists create models in a third-party editor such as Maya or 3D Studio Max. Using LithTech plugins, the models are saved in the .LTA file format. The .LTA file contains the geometry and animation information used by LithTech to manage and render the model. Artists can use LithTech's ModelEdit tool to modify .LTA files. Models can be highly detailed and can move. For more information on creating models and ModelEdit, see the ModelEdit chapter.

Models are represented in game code by custom BaseClass-derived simulation objects of type OT_MODEL. When a model object is instantiated, the location of the model's .LTB platform-dependent binary file is provided. LithTech uses the referenced .LTB file to render the model. The .LTB file is created from an .LTA file by a packer. The .LTB files are inaccessible in game code. All model functionality is exposed through the ILTModel interface.

Intrinsic to LithTech models are animations. Animations are a collection of time-to-transformation associations that modify the transform hierarchy of the model. The transform hierarchy is used to determine the positions.

Models move using animations. An animation is a series of movements that string together to make the model animate. An animation is created with the model in Max or Maya. When LithTech loads a model the default animation is automatically started. Animations can be looped. Animation trackers can be used to change a model's animation.

Other model, sprite, and light objects can be connected to a model as attachments. Attachments are connected to models with sockets. Attachments can be used for such accessories as guns, knives, swords, bows, and even other less violent objects (such as clipboards and coffee cups).

This Section contains the following models and animations topics:

Client Shell Topic

[Instantiating a Model](#)

[Animations](#)

[Attachments and Sockets](#)

[Transformations](#)

[Animation Code Example](#)

Description

Models are instantiated in game code using the OT_MODEL type in the **ObjectCreateStruct** structure passed in to the **ILTServer::CreateObject** function.

Animations are a specific set of motions for a model. Each model can have any number of animations.

Models, sprites, and lights can be attached to a model. Sockets define the connecting point between a model and an attachment.

Transformations define a position and rotation for attachments, sockets, and other items.

Describes how to set up a multi-tracked animation on an object.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

Programming Guide



Instantiating a Model

Models are represented in game code by simulation objects of type OT_MODEL. To instantiate a model, follow these steps:

1. Create an **ObjectCreateStruct** structure.
2. Set the **m_ObjectType** member variable to OT_MODEL.

3. Set the **m_Filename** to the path to the model's .LTB file.
4. Set the other **ObjectCreateStruct** member variables as desired.
5. Call **ILTServer::CreateObject**, passing in the **ObjectCreateStruct** as a parameter. The function returns a handle to the resulting **BaseClass**-derived simulation object.

After you have created the model, you can manipulate it as any other simulation object. For more information, see the [Simulation Object](#) section.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Programming Guide



Animations

An animation is a specific set of motions for a model. A model can have any number of animations associated with it. Artists create animations in a third-party editor such as Maya or 3D Studio Max at the same time that they create the associated model. Using LithTech plugins, the model and accompanying animations are saved together in the same .LTA file format. Artists can use LithTech's ModelEdit tool to modify .LTA files.

LithTech handles memory management of animation trackers. Therefore, game code does not need to access the animation tracker instances. Instead, animation trackers are referenced and manipulated using the **ANIMTRACKERID** type definition.

Animations for models can be stopped, played, looped, changed or blended. To stop or play an animation, use the **ILTModel::SetPlaying** function. To loop, or unloop, an animation, use the **ILTModel::SetLooping** function. To change a model's animation, use the **ILTModel::RemoveTracker**, **ILTModel::AddTracker** and **ILTModel::SetCurAnim** functions. To blend two or more animations on the same model at the same time, use the **AddTracker** and **SetCurAnim** functions, as well as the **ILTModel::FindWeightSet**, **GetWeightSet**, and **SetWeightSet** functions. See the Animation Trackers section below for more information.

An animation causes the model to engage in some motion without changing the model's position. For instance, a model's walk animation causes the legs of the model to move as if walking. However, unless the model object's position is changed at the same time the animation results in a moonwalk effect.

This section contains the following topics explaining animation code:

- [About the Default Animation](#)
- [Using Animation Trackers](#)
- [Adding Animation Tracker After Object Creation](#)
- [Removing Animation Tracker](#)
- [Associating Animation With Animation Tracker](#)
- [Blending Animations](#)

About the Default Animation

The default animation is always the first animation. The first animation can be seen in ModelEdit as the first animation in the animation list box. The default animation is automatically started when LithTech loads the model. Access to the default animation can be acquired using the **ILTModel::GetTracker** function.

To change the animation from the default, use the **ILTModel::SetCurAnim** function.

[Top](#)

Using Animation Trackers

An animation tracker identifies specific animations for a model. The animations for a model can be accessed using animation trackers. Models by default start with one animation tracker. This tracker points to the first animation found in the model database. The animation tracker keeps track of the current time, and the animation rate for the animation it holds. LithTech manages the creation, network distribution, and deletion of all animation trackers. Game code uses **ANIMTRACKERID** typedefs to identify animation. A model must have at least one tracker.

A model can have any number of animation trackers. Each animation tracker represents a specific animation, and each can be played consecutively or concurrently on the model. Concurrent animation is called blending. Each animation tracker added to the model blends with the base animation tracker. To blend animations together properly an animation blend weight set has to be associated with each animation tracker added after the first.

To change which animation a tracker is pointing to use **ILTModel::SetCurAnim**. This will effectively change the current animation a model is playing.

Enabling animation trackers after object creation is a two-step process. First, an **ANIMTRACKERID** is associated with a model using the **ILTModel::AddTracker** function. Then, the **ANIMTRACKERID** is associated with a specific animation using the **ILTModel::SetCurAnim** function.

[Top](#)

Adding Animation Tracker After Object Creation

To add animation trackers after the model object has been created, use the **ILTModel::AddTracker** function. This function associates an animation tracker with a specific model.

This function creates an animation tracker for the **OT_MODEL**. The **ANIMTRACKERID** is the reference index to the tracker for that model. Different models can

have the same tracker identifier or reference index.

```
LTRESULT AddTracker(
    HOBJECT hModel,
    ANIMTRACKERID TrackerID);
```

Once a tracker is created an animation can be assigned to it with the **ILTModel::SetCurAnim** function. The **AddTracker** function does not actually associate an animation with a model. It merely provides the model with an identifier to an animation tracker that can be associated with an animation using the **ILTModel::SetCurAnim**.

If the animation for a given animation tracker is changed using the **SetCurAnim** function, the previous animation will continue playing until finished before the new animation begins.

Animation tracker identifiers are composed of a value for a given model. Different models may use the same value for the animation tracker identifier without conflict. The animations for these trackers need not be identical.

ANIMTRACKERID0xff (the maximum value of an 8-bit unsigned integer) is reserved for the main tracker, which is always created for all models. You can refer to this resolved value with the name **MAIN_TRACKER**.

[Top](#)

Removing Animation Tracker

You can remove an animation tracker from a model using the **ILTModel::RemoveTracker** function.

```
LTRESULT RemoveTracker(
    HOBJECT hModel,
    ANIMTRACKERID TrackerID);
```

The **TrackerID** parameter identifies the animation tracker to be removed from the model identified by the **hModel** parameter.

[Top](#)

Associating Animation With Animation Tracker

After an animation tracker has been associated with a model, the animation tracker still needs to be associated with an animation using the **ILTModel::SetCurAnim** function.

```
LTRESULT SetCurAnim(
    HOBJECT hModel,
    ANIMTRACKERID TrackerID,
    HMODELANIM hAnim);
```

The **SetCurAnim** function associates the animation trackers identified by the **TrackerID** parameter with the animation identified by the **hAnim** parameter for the model identified by the **hModel** parameter.

The **hModel** parameter is required because the animation tracker identifier exists in the model namespace. That is, any given value for an animation tracker identifier can exist for multiple models but identify different animations.

The play state of the new animation is set to the play state of the previous animation. If there was no previous animation, the **play** state is set to **play**.

[Top](#)

Blending Animations

Multiple animations can be played on the same model at the same time. This is useful, for instance, to have a walking character raise a gun and shoot it. In this case, both a walk animation and a shoot animation would play at the same time.

A weightset is a collection of values used to interpolate from one animation to another. Each weightset contains a number of (possibly) disparate values that apply to each node in a model. Weightsets are created in ModelEdit and are represented in code by the **HMODELWEIGHTSET** type definition.

To set the weightset for a given animation, use the **ILTModel::SetWeightSet** function.

```
LTRESULT SetWeightSet(
    HOBJECT hModel,
    ANIMTRACKERID TrackerID,
    HMODELWEIGHTSET hSet);
```

The **hSet** parameter defines the weightset for the animation identified by the **TrackerID** parameter for the model identified by the **hModel** parameter. If the **hSet** parameter is invalid, the model disappears in-game.

Game code should keep track of the various blending possibilities and appropriately reset the animation blending weightsets for each.

The order in which animations are blended on a model is significant. If three animations are applied to a model, the first two animations are blended using the second animation's weightset, and the result is blended with the third animation using its weightset. This process extrapolates for any number of weightsets.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Programming Guide



Attachments and Sockets

An attachment is another simulation object connected to a model. Any object can be an attachment. For instance, a sword or gun model can be attached to the hand of a character model.

The point at which an attachment joins a model is called a socket. Sockets are added, edited, and named using the ModelEdit tool.

To programmatically attach something to a model, use the **ILTServer::CreateAttachment** function:

```
LTRESULT (*CreateAttachment) (
    HOBJECT hParent,
    HOBJECT hChild,
    char *pSocketName,
    LTVector *pOffset,
    LTRotation *pRotationOffset,
    HATTACHMENT *pAttachment);
```

The **CreateAttachment** function accepts several parameters. The **hParent** and **hChild** parameters identify the model and the object to use as the attachment, respectively. The **pSocketName** parameter identifies the socket (previously created and named in ModelEdit) at which to join the attachment to the model. The **pOffset** and **pRotationOffset** parameters modify the position and rotation of the attachment relative to the socket. The **pAttachment** parameter returns a handle to the newly created attachment.

The **ILTServer::RemoveAttachment** function removes a given attachment. When deleting an object that is attached to model the attachment should be removed first. If the child object is deleted without removing the attachment first, the results are undefined and game code should handle the error.

If a parent model is deleted, LithTech automatically removes all attachments for that model. However, the child objects are not automatically deleted. That is, if the parent model is deleted, the child objects still exist and are rendered.

If the parent model is removed using the **RemoveAttachment** function, the child objects are not automatically deleted.

The **ILTServer::FindAttachment** function returns an attachment handle given a model and an object.

You can also retrieve the position and rotation of any socket at any given point during an animation by calling the **ILTModel::GetSocketTransform** function. This is useful for locating a socket on the end of a gun to know exactly where to create the end of a laser beam, or where to make a rocket appear from the muzzle of a rocket launcher.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Programming Guide



Transformations

A transformation is a position and rotation. Artists create transformations in a third-party editor such as Maya or 3D Studio Max at the same time that they create a model.

In relation to models, transformations can apply to sockets, nodes, and attachments. A transformation is represented in code by the **LTTransform** class, defined in the **ltbasetypes.h** header file.

```
class LTTransform
{
public:
    LTVector    m_Pos;
```

```

        LTRotation    m_Rot;

        LTVector      m_Scale;

    };

```

The **ILTModel** interface, defined in the `iltmodel.h` header file, provides the **GetSocketTransform** and **GetNodeTransform** functions to retrieve the transformations for sockets and nodes, respectively.

The **ILTTransform** interface, defined in the `ilttransform.h` header file, includes functions to create and manipulate transformations in a variety of different ways.

Transformation Hierarchy and Transformation Nodes

Each model has a transformation hierarchy. A transformation hierarchy is an organization of transformations, arranged in a tree-like fashion, that define the position and orientation of various aspects of the model. Each node in the hierarchy has a transformation associated with it. Nodes can have parent nodes and child nodes. There is no limit to the number of nodes in a transformation hierarchy. If presented graphically, a transformation hierarchy would appear similar to a directory tree. Artists create the transformation hierarchy in Max or Maya along with the model.

The artist sets the number of nodes in the transformation hierarchy. Engine code cannot change the number or the arrangement of nodes in the transformation hierarchy. The transformations at the nodes can be queried and modified in game code.

HMODELNODE is a handle to a transformation node in the model database transform hierarchy.

To retrieve a handle to a node, use **ILTModel::GetNode**, or to access the transform hierarchy iteratively use **ILTServer::GetNextModelNode**.

Since the transform hierarchy cannot be modified, there is no set function for transformation nodes. Nodes can only be set, or created, by the artist when creating the model.

To retrieve the name of a node, use the **ILTServer::GetModelNodeName** function.

The **LTransform** instance for a transformation node can be retrieved using the **ILTModel::GetNodeTransform** function.

Modifying Animations

LithTech provides the ability to take the individual transformation nodes, and alter them on the fly per-frame. This allows some very powerful effects, but also can become a serious performance hit.

Note: This technique only works on the client. It is useful for some effects, but it is not a good method of actual animation.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Programming Guide



Animation Code Example

The following code sample shows how to set up multi-tracked animation on an object.

```

HOBJECT hModelObject ;

HMODELANIM hPrimeAnim, hSecondAnim

HMODELWEIGHTSET hSet1, hSet2;

hPrimeAnim = g_pLTServer->GetAnimIndex(hModelObject, "aimup");

if (hPrimeAnim != INVALID_MODEL_ANIM)

{

    //get the anim dims

if (g_pLTCommon->GetModelAnimUserDims(hModelObject, &vDims, hPrimeAnim) == LT_OK)

{

    //set the object dims

    g_pLTServer->Physics()->SetDims(hModelObject, &vDims);

}

//initialize the upper half of the model

pModel->FindWeightSet(hModelObject, "upperbody", hSet1);

pModel->SetWeightSet(hModelObject, MAIN_TRACKER, hSet1);

```



```
pModel->SetCurAnim(hModelObject, MAIN_TRACKER,hPrimeAnim);

pModel->SetLooping(hModelObject, MAIN_TRACKER, LTTRUE);

    }

    //setup the second animation
hSecondAnim = g_pLTServer->GetAnimIndex(hModelObject, "run");

    if (hSecondAnim != INVALID_MODEL_ANIM)
    {

        //add another tracker and setup the lower body
pModel->AddTracker(hModelObject, SECONDARY_TRACKER);

pModel->FindWeightSet(hModelObject, "lowerbody", hSet2);
pModel->SetWeightSet(hModelObject, SECONDARY_TRACKER, hSet2);

pModel->SetCurAnim(hModelObject, SECONDARY_TRACKER, hSecondAnim);
pModel->SetLooping(hModelObject, SECONDARY_TRACKER, LTTRUE);

        //set the animation rate
pModel->SetAnimRate(hModelObject, SECONDARY_TRACKER, 0.1f);

    }
```

[Top](#)

Rendering

Rendering in LithTech is handled by engine code. A variety of exposed engine interfaces allow the game code to render primitives, objects, and world geometry.

This section contains the following rendering topics:

ModelEdit Topics	Description
Initialization	Describes how to initialize the rendering process.
World Rendering	Describes how to prepare a world for rendering.
Model Rendering	Describes issues related to rendering models.
Custom Rendering	Describes low-level interfaces for custom rending or specific effects.
Render Styles	Describes a mechanism for consistent customization of rendering options.

[Top](#)

Initialization

Rendering initialization is a two-step process:

1. Define console variables.
2. Set the rendering mode with `ILTClient::SetRenderMode`.

Define Console Variables

Console variables are defined using console commands. The console variables used for rendering include:

- BitDepth
- ScreenHeight
- ScreenWidth
- Windowed

Set the rendering mode with `ILTClient::SetRenderMode`

To set the rendering mode (including display drivers, devices, and display mode) use the `ILTClient::SetRenderMode` function:

```
LTRESULT( * ILTClient :: SetRenderMode)(RMode *pMode)
```

The **pMode** parameter is a **RMode** structure containing the relevant video card information:

```
struct RMode
{
    LTBOOL m_bHardware;
    char m_InternalName[128];
    char m_Description[128];
    uint32 m_Width, m_Height, m_BitDepth;
    RMode *m_pNext;
};
```

The **m_bHardware** element is a Boolean value indicating whether or not this **RMode** structure supports hardware rasterization. The **m_Description** element is a friendly string describing the video card. The **m_Width**, **m_Height**, and **m_BitDepth** elements define the display mode. The **m_pNext** element is a pointer to the next **RMode** structure (this can be used to cycle through the list of **RMode** structures).

The `ILTClient::GetRenderModes` function returns a list of valid render modes. After reviewing the render modes with this function, call the `ILTClient::RelinquishRenderModes` function to release the **RMode** structure list.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Programming Guide



World Rendering

Before rendering can begin, the game code must put LithTech into the rendering state and select which camera to render from. This is usually done from the client shell's main update loop.

Assuming that you have **hCamera**, an **HCAMERA** variable representing which camera you want to render from:

```
g_pLTClient->Start3D();
g_pLTClient->ClearScreen(NULL,CLEARSCREEN_RENDER);
g_pLTClient->RenderCamera(m_hCamera);
g_pLTClient->End3D(END3D_CANDRAWCONSOLE);
g_pLTClient->FlipScreen(NULL);
```

After you have rendered, you need to call `FlipScreen` with any appropriate parameters, or the new rendering will not be visible.

```
GetClientLT()->ClearScreen(NULL,CLEARSCREEN_RENDER);
GetClientLT()->Start3D();
GetClientLT()->RenderCamera(hCamera);
GetClientLT()->StartOptimized2D();
DoMyHUDDisplay();
GetClientLT()->EndOptimized2D();
GetClientLT()->End3D();
```

```
GetClientLT() -> FlipScreen()
```

World Polygons

When world polygons are rendered they are queued into one of the following buckets:

Gouraud: One texture, non-lightmapped polygons (lighting comes from vertex colors).

LightMapped: Two textured polygons (base texture + lightmap).

Detail Textured: Three textures (base, lightmap, detail texture).

Env Mapped: Three textures (base, lightmap, world env map).

Depending on the graphics card and **force1pass** and **force2pass** console variable settings, world polygons will be rendered in one or two passes, respectively.

In a **force1pass**, detail and environment mapped polygons are not supported. Such polygons will be rendered with only a lightmap.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Programming Guide



Model Rendering

There are two types of models: world models (built in DEdit) and models imported from modeling software (for example, Maya or Max). World models are always rigid (lacking bones). Imported models can be rigid, skeletal animated, or vertex animated.

Both types of models are read into the LithTech engine as .LTB files (binary files packed specifically for the target platform).

Alpha blended models are sorted and rendered after all the opaque models have been rendered.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Programming Guide



Custom Rendering

LithTech handles rendering of the world models and imported models. However, LithTech does expose some lower-level interfaces that provide custom rendering of specific, effects. This section discusses the following lower-level custom rendering interfaces:

- [ILTDrawPrim](#): Renders arbitrary primitives, including triangles, quads, lines, points, strips and fans.
 - [ILTTexInterface](#): Creates and modifies textures.
 - [Canvases](#): Used to get callbacks at object render time (for sorted alpha models).
-

ILTDrawPrim

The **ILTDrawPrim** interface exposes low-level primitive rendering to the game code. **ILTDrawPrim** functions can be used to draw arbitrary polygons, including triangles, quads, strips, lines, points, and fans. These primitives can be flat-shaded or Gouraud-shaded, and also textured or untextured. Alpha blending is supported in both textures and vertex colors.

ILTDrawPrim is not only useful for rendering special polygons that will appear in the final game, but it can also be used for debugging purposes. Various types of debugging geometry can be used to test an object's size, collisions, or just to test the appearance of a texture.

ILTDrawPrim uses structures to contain relevant vertex and polygon information. These structures are defined in the `iltdrawprim.h` header file.

Vertex Structures

The first set of structures defined contain vertex information. These various vertex structures provide a base upon which the polygon structures depend.

For all polygon structures, the vertex structures contain the coordinate information and the Gouraud-shading color information. For textured polygons, the vertex structures contain the u and v information.

Flat-shaded vertex structures do not contain color data and only represent the location of the vertex. The color of a flat-shaded polygon is set in the polygon structure.

The vertex structures include **LT_VERTRGBA**, **LT_VERTF**, **LT_VERTFT**, **LT_VERTG**, and **LT_VERTGT**.

Line Structures

Structures defining primitive lines include **LT_LINEF**, **LT_LINEFT**, **LT_LINEG**, and **LT_LINEGT**.

Triangle Structures

Structures defining primitive triangles include **LT_POLYF3**, **LT_POLYFT3**, **LT_POLYG3**, and **LT_POLYGT3**.

Quadrilateral Structures

Structures defining primitive quadrilaterals include **LT_POLYF4**, **LT_POLYFT4**, **LT_POLYG4**, and **LT_POLYGT4**.

Accessing the ILTDrawPrim Interface

The polygon structures listed above are available for creation when the LithTech header files are included in a project. Before calling the **ILTDrawPrim** functions, however, the game code must acquire a handle to LithTech's instantiation of the **ILTDrawPrim** interface. The game code should not instantiate **ILTDrawPrim**.

The Interface Manager exposes one **ILTDrawPrim** instance for the game code to access. The game code can acquire a handle to this instance by associating it with an **ILTDrawPrim** pointer using the **define_holder** macro (defined in `ltmodule.h`).

The code snippet below first creates an **ILTDrawPrim** pointer (**g_pILTDrawPrim**). The **define_holder** macro is then used to set the address of the **g_pILTDrawPrim** pointer to the location for the **ILTDrawPrim** instance.

```
#include "iltdrawprim.h"

static ILTDrawPrim *g_pILTDrawPrim;

define_holder(ILTDrawPrim, g_pILTDrawPrim);
```

After setting the correct address for the **g_pILTDrawPrim** pointer as shown above, the **ILTDrawPrim** functions can be called.

Note: LithTech uses another unexposed instantiation of the **ILTDrawPrim** interface to provide internal 2D rendering, text and user interface, and console functionality

Primitive Structure Helper Functions

The **ILTDrawPrim** interface provides several helper functions to quickly set values for primitive quadrilateral structures. These helper functions include **SetALPHA**, **SetALPHA4**, **SetRGB**, **SetRGBA**, **SetRGBA4**, **SetUV4**, **SetUVWH**, **SetXY4**, and **SetXYWH**.

These functions are overloaded for two or more quadrilateral structures.

State-setting Functions

The **ILTDrawPrim** interface includes a series of functions to set various states for drawing primitives. The game code only references one **ILTDrawPrim** instance. Therefore, each **DrawPrim*** call implements the same set of states. Whereas these states may be modified by any number of areas in the game code, the current state settings cannot be guaranteed. Therefore, states should be set (or reset) before each block of **DrawPrim** calls to insure that the desired states are actually used.

The state-setting functions include **SetAlphaBlendMode**, **SetAlphaTestMode**, **SetCamera**, **SetClipMode**, **SetColorOp**, **SetCullMode**, **SetFillMode**, **SetTexture**, **SetTransformType**, and **SetZBufferMode**.

Draw Functions

The **ILTDrawPrim** interface exposes several overloaded drawing functions, including **DrawPrim**, **DrawPrimPoint**, **DrawPrimFan**, and **DrawPrimStrip**. When one of these functions is called, the primitive is immediately rendered to the screen.

The current state settings (described above) identify the characteristics of any primitive drawn with one of the **DrawPrim*** functions. The states are not protected from modification from other areas of the code. If you call other functions, they may modify the current states in **DrawPrim**. Therefore, all of the states should be set before each block of **DrawPrim**, **DrawPrimPoint**, **DrawPrimFan**, or **DrawPrimStrip** function calls.

Checkerboard Sample Code

The following code sample implements various functions of the **ILTDrawPrim** interface to render a checkerboard pattern onto the screen.

```
#include "iltdrawprim.h"

// acquire the interface from the Interface Manager

static ILTDrawPrim *g_pLTDrawPrim;

define_holder(ILTDrawPrim, g_pLTDrawPrim);

void DrawCheckerboard()
{
    // initialize

    uint32 idx;

    uint32 x, y;

    uint32 tileWidth, tileWeight;
```

```

uint32 screenWidth, screenHeight;

LT_POLYF4 pPolyArray[25]; // a 5 x 5 grid of 4-sided polygons

g_pLTClient->GetSurfaceDims(g_pLTClient->GetScreenSurface(),
                           &screenWidth, &screenHeight);

tileWidth = screenWidth / 5;
tileWeight = screenHeight / 5;

for (y=0; y<5; y++)
{
    for (x=0; x<5; x++)
    {
        idx = y * 5 + x;

        // set white or black color
        pPolyArray[idx].rgba.r = 255 * (idx % 2);
        pPolyArray[idx].rgba.g = 255 * (idx % 2);
        pPolyArray[idx].rgba.b = 255 * (idx % 2);
        pPolyArray[idx].rgba.a = 255;

        // vertices are clockwise from upper left
        pPolyArray[idx].verts[0].x = float(x * tileWidth);
        pPolyArray[idx].verts[0].y = float(y * tileWeight);
        pPolyArray[idx].verts[0].z = 0;

        pPolyArray[idx].verts[1].x = float(x * tileWidth + tileWidth);
        pPolyArray[idx].verts[1].y = float(y * tileWeight);
        pPolyArray[idx].verts[1].z = 0;

        pPolyArray[idx].verts[2].x = float(x * tileWidth + tileWidth);
        pPolyArray[idx].verts[2].y = float(y * tileWeight + tileWeight);
        pPolyArray[idx].verts[2].z = 0;

        pPolyArray[idx].verts[3].x = float(x * tileWidth);
        pPolyArray[idx].verts[3].y = float(y * tileWeight + tileWeight);
        pPolyArray[idx].verts[3].z = 0;
    }
}

// render (each frame)

// set up the render state
g_pLTDrawPrim->SetTransformType(DRAWPRIM_TRANSFORM_SCREEN);
g_pLTDrawPrim->SetColorOp(DRAWPRIM_NOCOLOROP);
g_pLTDrawPrim->SetAlphaBlendMode(DRAWPRIM_NOBLEND);
g_pLTDrawPrim->SetZBufferMode(DRAWPRIM_NOZ);
g_pLTDrawPrim->SetAlphaTestMode(DRAWPRIM_NOALPHATEST);

```

```

g_pLTDrawPrim->SetClipMode(DRAWPRIM_FASTCLIP);

g_pLTDrawPrim->SetFillMode(DRAWPRIM_FILL);


// draw our checkerboard array

g_pLTDrawPrim->DrawPrim(pPolyArray, 25);

}

```

[Top](#)

ILTTexInterface

The **ILTTexInterface** interface exposes texture manipulation functionality to the game code. It provides direct access to texture data. This direct access means that the texture won't be converted to a new format, and will therefore result in faster texture modification.

The **ILTTexInterface** and associate types are defined in the `ilttexinterface.h` header file.

Create a Texture From a DTX File

The **ILTTexInterface::CreateTextureFromName** function is used to create a texture from a .DTX file. A .DTX file is created using DEdit. This function will always create a new internal texture from the given filename. However, it may read the file data from its internal resource cache. Often you will want to call **FindTextureFromName** to find out if a texture is currently loaded, and then call **CreateTextureFromName** to load it if it is not.

```

LTRESULT CreateTextureFromName

(

    HTEXTURE &hTexture,

    const char *pFilename

);

```

The **pFilename** parameter points to a string containing the path to the resource directory in which the .DTX file is located.

The address of the created texture is returned in the **hTexture** out-parameter.

If the .DTX file was created with the **DTX_32BITSYSCOPY** bit set, the texture will not be converted by LithTech to a platform compatible format.

To determine the format type of the new texture, use the [ILTTexInterface::GetTextureType](#).

Create a Texture From Data

The first version of the overloaded **ILTTexInterface::CreateTextureFromData** function will create a texture from information passed in the **pData** parameter.

```

LTRESULT CreateTextureFromData

(

    HTEXTURE &hTexture,

    ETextureType eTextureType,

    uint32 TextureFlags,

    uint8 *pData,

    uint32 nWidth,

    uint32 nHeight,

    uint32 nAutoGenMipMaps = 1

);

```

The texture data from which to create the texture is passed into the function in the **pData** pointer.

The width and height are passed in the **nWidth** and **nHeight** parameters, respectively.

The **nAutoGenMipMaps** parameter sets the number of mip maps to create and populate for this texture. If it is set to 0, mip maps all the way down to 1x1 will be created and populated.

The created texture is returned to the **hTexture** out-parameter.

The **eTextureType** parameter specifies the bitdepth and format of the texture to create.

The **TextureFlags** parameter is a combination of flags that instruct the function to create the texture in a specific manner. It may be a combination of one or more of the following values:

- **DTX_32BITSYSCOPY**: If there is a system copy of the texture don't convert it to a device specific format (keep it 32 bit).
 - **DTX_FULLBRITE**: Use fullbright colors.
- **DTX_MIPSCALLLOCED**: TextureMipData texture data is allocated.
- **DTX_NOSYSCACHE**: Do not save the texture in the texture cache list.
 - **DTX_PREFER16BIT**: Use 16-bit, even if in 32-bit mode.

- **DTX_PREFER4444**: If in 16-bit mode, use a 4444 texture.
- **DTX_PREFER5551**: If in 16-bit mode, use 5551 texture.
- **DTX_SECTIONSFIXED**: Set for fixed sections counts.

Create a Palettized Texture from Data

The second version of the overloaded **ILTTexInterface::CreateTextureFromData** function creates a palettized texture from information passed in the **pData** parameter.

```
LTRESULT CreateTextureFromData
(
    HTEXTURE &hTexture,
    ETextureType eTextureType,
    uint32 TextureFlags,
    uint8 *pData,
    uint8 *pPalette,
    uint32 nWidth,
    uint32 nHeight,
    uint32 nAutoGenMipMaps = 1
);
```

All of the parameters for this version of the function perform as for the first version of the function (as explained above). The new in-parameter: the **pPalette** pointer, points to the palette to use for the texture.

Create an Empty Texture

Empty textures can be created using the **ILTTexInterface::CreateEmptyTexture**.

```
LTRESULT CreateEmptyTexture
(
    HTEXTURE &hTexture,
    ETextureType eTextureType,
    uint32 TextureFlags,
    uint32 nWidth,
    uint32 nHeight,
    uint32 nAutoGenMipMaps = 1
);
```

There are two differences between this function and the **ILTTexInterface::CreateTextureFromName** function:

- This function lacks the **pData** parameter.
- The mip maps for an empty texture will not be populated.

Texture data can be applied to an empty texture by using the **ILTTexInterface::GetTextureData** function to get a pointer to the empty data location. Texture data can then be assigned to it.

To populate the mip maps, use the **ILTTexInterface::AutoGenMipMaps** function.

Find a Texture

The **ILTTexInterface::FindTextureFromName** function retrieves a handle to an existing texture given a filename. This function enables you to modify an existing texture and all polygons that use this texture will reflect the changes you make.

```
LTRESULT FindTextureFromName
(
    HTEXTURE &hTexture,
    const char *pFilename
);
```

The **pFilename** parameter points to a string containing the relative path to the resource directory in which the .DTX file is located.

The address of the texture is returned in the **hTexture** out-parameter.

Modify Texture Data

Texture data can be modified using the first version of the overloaded **ILTTexInterfaces::GetTextureData** function. This function is useful for assigning texture data to a texture that was created with the **ILTTexInterface::CreateEmptyTexture** function, or modifying an existing texture.

```
LTRESULT GetTextureData
```

```

(
    const HTEXTURE hTexture,

    bool bReadOnly,

    uint8* pData,

    uint32 nPitch,

    uint32 nMipMap = 0

);

```

The texture to be modified is passed into the function using the **hTexture** parameter.

The **bReadOnly** in-parameter indicates whether the data is read only or read/write. To modify the texture data, **bReadOnly** must be set to false.

The **nMipMap** in-parameter identifies the texture or mip map to retrieve. If this is set to 0, the **pData** parameter will point to the base texture. If this is set to 1 or more, the **pData** parameter will point to the appropriate mip map.

The **pData** out-parameter is a pointer to the texture data location. The data can be modified or overwritten if the **bReadOnly** parameter is set to false.

The **nPitch** out-parameter is the size in bytes of a row of data in the texture.

The game code must inform LithTech when it is finished modifying the texture data. This is done by using the **ILTTexInterface::FlushTextureData** function described in the *Finish Modifying Texture Data* section.

Modify Palettized Texture Data

Palette data for a texture can be modified using the second version of the overloaded **ILTTexInterfaces::GetTextureData** function.

```

LTRESULT GetTextureData

(
    const HTEXTURE hTexture,

    bool bReadOnly,

    uint8* pData,

    uint8* pPalette,

    uint32 nPitch,

    uint32 nMipMap = 0

);

```

This function is the same as the first version except that it adds the **pPalette** out-parameter. This parameter is the address of the palette being used by the texture. This data can be modified or overwritten if the **bReadOnly** parameter is set to false.

The game code must inform LithTech when it is finished modifying the texture data. This is done by using the **ILTTexInterface::FlushTextureData** function. For more information about the **FlushTextureData** function, see [Finish Modifying Texture Data](#).

Add Detail to a Texture

Detail textures can be added to a texture using the **ILTTexInterface::SetDetailTexture** function.

```

LTRESULT SetDetailTexture

(
    HTEXTURE hTexture,

    const HTEXTURE hRefTex,

    ETexRefType eDetailTexType

);

```

The **hTexture** parameter sets the texture to which the detail will be added.

The **hRefTex** parameter is the detail texture to add to the base texture.

The **eDetailTexType** is an **ETexRefType** value setting the detail texture's type.

Use the **ILTTexInterface::GetDetailType** function to retrieve a texture's detail texture.

Finish Modifying Texture Data

While the game code is modifying the texture data (using either version of the overloaded **ILTTexInterface::GetTextureData** function), LithTech will not use that texture. Use the **ILTTexInterface::FlushTextureData** function to inform LithTech that the texture is no longer being modified.

```

virtual LTRESULT FlushTextureData

(
    const HTEXTURE hTexture,

```



```

    ETextureMod eChanged = TEXTURE_DATAANDPALETTECHANGED,

    uint32 nMipMap = 0

    )=0;

```

The modified texture is identified with the **hTexture** in-parameter.

The **eChanged** in-parameter is a **ETextureMod** enumerated type. It may be one of the following values:

- **TEXTURE_DATACHANGED**—Only the texture data (the **pData** parameter) was changed.
- **TEXTURE_PALETTECHANGED**—Only the texture's palette (the **pPalette** parameter) was changed.
- **TEXTURE_DATAANDPALETTECHANGED**—Both the texture data and palette data were changed.

The **nMipMap** parameter indicates individual mip map that is no longer being modified. A value of 0 indicates the base texture is no long being modified. This must match the **nMipMap** parameter value of the **GetTextureData** call used to modify the texture.

Automatically Populate Mip Maps

Mip maps can be populated with texture data using the **ILTTexInterface::AutoGenMipMaps** function. Storage sapce for the mip maps must already have been allocated using the **ILTTexInterface::CreateTextureFromName**, **CreateTextureFromData**, or **CreateEmptyTexture** functions.

```

LTRESULT AutoGenMipMaps

(

    const HTEXTURE hTexture,

    uint32 nAutoGenMipMaps = 1

);

```

The **hTexture** parameter identifies the texture for which the mip maps will be automatically populated.

The **nAutoGenMipMaps** parameter sets how many mip maps will be populated, starting with the mip map closest to the base texture. This number must be equal to or less than the number of mip maps that exist for the texture. If this parameter is set to 0, all existing mip maps for the texture will be populated.

Existing mip map data will be overwritten by this function.

Set Mip Map Range

At times, it may be desirable to confine the usage of a texture's mip maps to a subset of all mip maps available. This can be accomplished using the **ILTTexInterface::SetTextureMipMapRange** function.

```

LTRESULT SetTextureMipMapRange

(

    const HTEXTURE hTexture,

    uint32 nMaxMipMaps,

    uint32 nMinMipMaps

);

```

The **hTexture** parameter identifies the texture for which the mip map range is set.

The **nMinMipMaps** parameter sets the smallest mip map to use for the texture. The value is actually a number of mip maps from the base texture (which has a value of 0).For example, the third mip map for a texture has a value of 3.

The **nMaxMipMaps** parameter sets the largest mip map to use for the texture. The value is actually a number of mip maps from the base texture.

Only mip maps of sizes between (inclusive) the **nMipMaps** and **nMaxMipMaps** values will be used in the texture.

Release Texture

When the game code is done with a texture, the **HTEXTURE** should be released to free up system resources. Call the **ILTTexInterface::ReleaseTextureHandle** to free the **HTEXTURE** identified by the **hTexture** parameter.

```

bool ReleaseTextureHandle

(

    const HTEXTURE hTexture

);

```

If the **HTEXTURE** is still being used by LithTech, it will not be immediately freed. After LithTech is through with the **HTEXTURE**, it will be freed.

ILTTexInterface Review Functions

The **ILTTexInterface** class exposes several functions that can be used to review current information concerning a particular texture. Each of these functions takes an **HTEXTURE** parameter identifying the texture for which information is desired. The desired data is returned in one or more out-parameters.

GetDetailTexture—Retrieves a **HTEXTURE** (the **hRefTex** out-parameter) to the detail texture of the texture. The type (an **ETexRefType** enumerated type) of detail texture is also returned in the **eDetailTexType** out-parameter. The detail texture can be set using the **ILTTexInterface::SetDetailTexture** function.

GetTextureMipMapRange—Retrieves the minimum number of mip maps (the **nMinMipMaps** out-parameter) and the maximum number of mip maps (the **nMaxMipMaps** out-parameter) for the texture.

GetTextureNumMipMaps—Retrieves the number of mip maps (the nMipMaps out-parameter) for the texture.

GetTextureDims—Retrieves the current pixel width and pixel height (the nWidth and nHeight out-parameters) of the texture.

GetTextureFlags—Retrieves the flags (the TextureFlags out-parameter) set for the texture. The TextureFlags parameter can be a combination of one or more of the following values:

DTX_32BITSYSOCOPY—If there is a system copy of the texture don't convert it to a device specific format (keep it 32 bit).

DTX_FULLBRITE—Use fullbrite colors.

DTX_MIPALLOCED—TextureMipData texture data is allocated.

DTX_NOSYSOCACHE—Do not save the texture in the texture cache list.

DTX_PREFER16BIT—Use 16-bit, even if in 32-bit mode.

DTX_PREFER4444—If in 16-bit mode, use a 4444 texture.

DTX_PREFER5551—If in 16-bit mode, use 5551 texture.

DTX_SECTIONSFIXED—Set for fixed sections counts.

GetTextureType—This function retrieves the texture format type (the eTextureType out-parameter) of a texture. The possible types are defined by the ETextureType enumerated type and include:

TEXTURETYPE_INVALID—The HTEXTURE does not have an accepted type.

TEXTURETYPE_ARGB8888—Compatible with the PC platform only.

TEXTURETYPE_ABGR8888—Compatible with the PlayStation 2 platform only.

TEXTURETYPE_ARGB8888_P8—Compatible with the PC platform only.

TEXTURETYPE_ABGR8888_P8—Compatible with the PlayStation 2 platform only.

TEXTURETYPE_ARGB4444—Compatible with the PC platform only.

TEXTURETYPE_ARGB1555—Compatible with the PC platform only.

TEXTURETYPE_RGB565—Compatible with the PC platform only.

TEXTURETYPE_DXT1—Compatible with the PC platform only. DirectX texture compression format.

TEXTURETYPE_DXT3—Compatible with the PC platform only. DirectX texture compression format.

TEXTURETYPE_DXT5—Compatible with the PC platform only. DirectX texture compression format.

WasTextureDrawnLastFrame—Returns a Boolean value indicating whether or not the texture was rendered in the last frame. If it returns false, the texture is not in view and need not be updated.

For more information on these functions, see the API Reference.

[Top](#)

Canvases

The **ILTClient::SetCanvasFn** and **ILTClient::GetCanvasFn** functions are used to set and get callbacks at object render time (for sorted alpha rendering).

Rendering Objects By Hand

There are also situations where you might not be able to use the normal rendering methods, which require a world to be loaded. A good example might be if you wished to display one of the player models during a ‘character selection screen’ before allowing the person to start up the game. This is done by creating an array of **HLOCALOBJ** handles for the objects you want to render and placing them relative to a camera you create as a local client object. Then you render through them (again, in a **Start3D/End3D** block) like so:

```
GetClientLT()->ClearScreen(NULL,CLEARSCREEN_SCREEN);

GetClientLT()->Start3D();

GetClientLT()->RenderObjects(hCamera, pObjectList, 2);

GetClientLT()->End3D();

GetClientLT()->FlipScreen();
```

In this case, let us assume that the list of objects has two entries, one being a model (the player model) we are rendering for the character selection and the other being a light to give a nice dramatic lighting effect. Just as though we were rendering the world through a camera, model hooks can be called on manually rendered objects.

Rendering Hooks

While LithTech itself handles the majority of rendering, the game code has an opportunity to alter certain rendering functions, such as model rendering. You can set a ‘model hook function’ that can be used to alter lighting on a model in a way specific to that model, for example. This would be useful for creating a “pulsing glow” effect on a power-up in a first-person shooter.

A Model Hook Function is given a chance to alter the rendering options of a particular model each frame. This function will be called with two parameters: a

ModelHookData structure that contains information on what model is being rendered and allows changes to be made, and a void pointer defined by the user.

A good example might be to cause a “pulsing glow” as described above:

```
void DefaultModelHook (struct ModelHookData_t *pData, void *pUser)
{
    LTClientShell* pShell = (LTClientShell*) pUser;

    if (!pShell) return;

    unit32 nUserFlags = 0;

    g_pClientLT->GetObjectUserFlags (pData->m_hObject, &nUserFlags);

    if (nUserFlags & USRFLG_GLOW)
    {
        if (pData->m_LightAdd)
        {
            *pData->m_LightAdd = pShell->GetModelGlow();
        }
    }
}
```

The function to set the model rendering hook is

```
LTRESULT SetModelHook(ModelHookFn fn, void* pUser);
```

As the **pUser** variable is a pointer to whatever is defined by the user at the time they set the model hook, this particular example assumes that you passed a pointer to the ClientShell, which has a **GetModelGlow** function which will return the amount of light to add to the model as a **LTVector**. This would cause the model itself to be rendered as if there were an additional lightsource that affected only it.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Programming Guide



Render Styles

LithTech Render Styles provide a consistent mechanism to customize how models are rendered. The customization encompasses parts of both lighting and rasterization from the basic render states and texture passes to dynamic UV generation and lighting material properties. Render Styles also supports platform-specific rendering methods (such as Direct3D vertex shaders). Render Styles are representing in code by the **CRenderStyle** class.

This section contains the following render styles topics:

- [About Render Styles](#)
- [Using ILTRenderStyles](#)
- [Using CreateRenderStyle](#)
- [Setting Values in CRenderStyle](#)
- [Applying a Render Style to a Model](#)
- [Changing Render Styles Dynamically](#)
- [Using Pre-Made Render Styles](#)
 - [Default Render Styles](#)
 - [Additional Render Styles](#)

About Render Styles

The high-level process for applying Render Styles proceeds as follows:

1. Artists create a model in Maya or Max.
2. The model is exported to ModelEdit and customized therein.
3. Engineers create a Render Style to represent a given effect. This can be accomplished using the Render Styles Editor or by appropriate coding.
 4. Engineers assign a Render Style in the game code.
5. The artist or engineer sets the Render Style index in ModelEdit just like for Textures.

LithTech includes several pre-generated Render Styles for use in your application. These cover most of the basic effects that are used in games. However, you can create your own Render Styles for customized effects using the Render Styles Editor or programmatically using the **ILTRenderStyles** interface and **CRenderStyle** class.

Note: Creating a new Render Style requires a level of graphics and rendering knowledge. The Render Styles Editor automates some of

the Render Styles creation processes. This tool provides real-time feedback and convenient features, but is still fairly complex in nature. It is not our expectation that all members of a game design team will be able to build their own render styles.

When an object is created, you can set the Render Style using the **ObjectCreateStruct's m_RenderStyleName** member variable.

To programmatically change a Render Style on an existing model in your game code

1. Acquire a pointer to LithTech's **ILTRenderStyles** class instance using the Interface Manager.
2. Call the **ILTRenderStyles::CreateRenderStyle** function (or the **ILTModel::GetRenderStyle** function) to retrieve a pointer to a **CRenderStyle** instance.
 3. Set the values in the **CRenderStyle** instance as desired.
4. Apply the **CRenderStyle** instance to the model using the **ILTModel::SetRenderStyle** function.

The RenderStyles sample located in the `\samples\minimal\renderstyles\` directory of your LithTech installation provides examples of code implementing Render Styles.

[Top](#)

Using ILTRenderStyles

The **ILTRenderStyles** interface provides methods to create, load, duplicate and free Render Styles (**CRenderStyle** instances). LithTech exposes an **ILTRenderStyles** instance to the game code. You should not instantiate your own **ILTRenderStyles** instance. Use the **define_holder** macro of the Interface Manager to acquire a pointer to it:

```
static ILTRenderStyles *g_pLTRenderStyles;

define_holder(ILTRenderStyles, g_pLTRenderStyles);
```

After you have a pointer to the **ILTRenderStyles** instance, you can invoke several render style functions (**CreateRenderStyle**, **DuplicateRenderStyle**, **FreeRenderStyle**, and **LoadRenderStyle**).

[Top](#)

Using CreateRenderStyle

The **ILTRenderStyles::CreateRenderStyle** function returns a pointer to a **CRenderStyle** instance created by LithTech. This is the only way that a **CRenderStyle** instance should be instantiated.

```
CRenderStyle* CreateRenderStyle(bool bSetToDefault = true);
```

The function's **bSetToDefault** parameter determines whether or not the returned **CRenderStyle** instance will have its member variables set to default values. The return value is a pointer to a newly instantiated **CRenderStyle** instance. You can access this instance's member variables and functions to set values as desired.

[Top](#)

Setting Values in CRenderStyle

The **CRenderStyle** class defines a Render Style. **CRenderStyle** instances should only be created using one of the **ILTRenderStyles** functions (**CreateRenderStyle**, **DuplicateRenderStyle**, or **LoadRenderStyle**). You should not manually instantiate a **CRenderStyle** instance in your game code.

After you have a pointer to a **CRenderStyle** instance you can use its many Set and Get functions to manipulate the following render options:

- Clip mode (no clipping, fast clipping, full clipping)
- Lighting material (ambient, diffuse, emissive, specular, specular power)

In addition to the options available above, you must also set one or more render passes to be applied to the model using the various **CRenderStyles** render pass functions:

- **AddRenderPass**
- **RemoveRenderPass**
- **SetRenderPass**
- **GetRenderPass**
- **GetRenderPassCount**

Each render pass is defined by a **RenderPassOp** structure. You can set the following render pass options using the **RenderPassOp** member variables:

Alpha test mode (none, less than, less than or equal to, greater than, greater than or equal to, equal, not equal)

- Fill mode (wire, fill)
- Cull mode (no culling, clockwise culling, counter-clockwise culling)
 - Dynamic lighting (enable/disable)
 - Texture stages
 - Alpha blending mode
 - Z-buffer mode
 - Texture Factor
- Direct3D Bump environment mapping (enable/disable)

[Top](#)

Applying a Render Style to a Model

After you have set the **CRenderStyle** variables you can apply the render style to a piece of a model using the **ILTModel::SetRenderStyle** function:

```
LTRESULT SetRenderStyle(
    HOBJECT pObj,
    HMODELPIECE hPiece,
    uint32 iLOD,
    CRenderStyle* pRenderStyle);
```

The **pObj** parameter identifies the engine object on which the model piece is located. The **hPiece** is a handle to the specific model piece on which the render style will be applied. The **iLOD** parameter identifies the level of detail. The **pRenderStyle** parameter points to the **CRenderStyle** to apply to the model piece.

[Top](#)

Changing Render Styles Dynamically

Most Render Styles can be changed dynamically with little impact on performance. However, changing some of the parameters within the object is programmatically costly. A certain amount of pre-computation is done at pack time and at world load time. A render style is assigned to a model at model pack time. Some portions of the render style effect how it is packed and therefore cannot be changed at runtime.

[Top](#)

Using Pre-Made Render Styles

LithTech provides several pre-made Render Styles that you can use to do basic effects.

Use the **ILTRenderStyles::LoadRenderStyle** function to create an object with one of these styles, as shown in the following example:

```
m_CRenderStyle =
g_pLTCTRenderStyles->LoadRenderStyle("RenderStyles/rs_halo_vs.ltb");
```

[Top](#)

Default Render Styles

These standard Render Styles are compiled and placed in the engine.rez file which is loaded by default by LithTech. They are all prefaced with the **RenderStyles** directory.

AlphaBlend (with texalpha), rs_alphablend_texalpha.ltb—Alpha blend with alpha factor from the textures alpha.

AlphaBlend (with tfactoralpha), rs_alphablend_tfactoralpha_1tex.ltb—Alpha blend using the TextureFactor alpha as alpha.

AlphaTest (greater), rs_alphatest_greater.ltb—Alpha Test (also known as ChromaKey).

Sphereenvmap (Add), rs_sphereenvmap_add.ltb—Sphere environment map (add the environment map pass).

Sphereenvmap (modtexalpha), rs_sphereenvmap_modtexalpha.ltb—Sphere environment map (using the texture alpha to modulate on the env map).

Two Texture, rs_twotexture.ltb—Use two textures (both with same UV set).

Vertex Specular, rs_vertexspecular.ltb—Use vertex specular.

dot3bump bump mapping, rs_dot3bump_dir_vs.ltb.

Halo, rs_halo_vs.ltbI—Draw a white halo around the model on a second pass.

Cartoon render, rs_toon_vs.ltb—The first texture is the shade texture.

Fullbright, rs_fullbright.ltb—No dynamic lighting. Textures decal onto the models directly.

Fullbright (with texture masking), rs_fullbright_basetexmask.ltb—The model's base texture's alpha specifies where the model is fullbright and where it uses the regular dynamic lighting.

[Top](#)

Additional Render Styles

The following Render Styles are not compiled in the engine.rez file by default. To use these Render Styles you must explicitly add them to your game resources.

Anisotropic lighting, rs_aniso_3light_vs.lta—Simulates anisotropic lighting. Uses a second texture with an alpha mask as a lighting highlight with its UVs generated from $x = l \cdot n$, $y = h \cdot n$. Where l is the light vector, n is the normal, and h is the half angle vector.

Anisotropic lighting (with texture masking), rs_aniso_basetexmask_3light_vs.lta—Similar to **rs_aniso_3light_vs.lta**, except it uses the base texture's alpha mask to attenuate the anisotropic highlighting.

Blurry, rs_blurry_vs.lta—Simulate a blur (from field of view or other). Renders the model three times. The second and third renderings are offset in

screen space by a small amount to give the effect of a blurry object.

Eye dot Normal Highlighting, `rs_eyedotnormal_3light_vs.lta`—Uses a second texture pass to highlight the model with a texture map with its UVs generated from the eye vector dot the normal vector. The highlight texture can be used to emphasize the center or edges of the model (screen space center/edges).

Eye dot Normal Highlighting (with texture masking), `rs_eyedotnormal_basetexmask_3light_vs.lta`—Like `rs_eyedotnormal_3light_vs.lta`, but uses the base texture's alpha mask to attenuate the highlight texture.

Eye dot Normal Highlighting (with highlight texture alpha attenuation), `rs_eyedotnormal_highlight_3light_vs.lta`—Like `rs_eyedotnormal_3light_vs.lta`, but uses the alpha from the highlight texture in the effect.

Motion blur, `rs_motionblur_vs.lta`—Single pass motion blur. Transforms each vertex with the objects current and previous transform, takes the difference vector and dots it with the normal. Uses the result to choose the actual position of the vertex and its alpha. Note that this only works on rigid objects.

Motion fade, `rs_motionfade_vs.lta`—Renders two more copies of the objects at previous locations fading the object as it goes. Note that this only works on rigid objects.

Power shield 1, `rs_powershield1_vs.lta`—A power shield example. Expands the model by its normals, and uses the eye dot normal to set the transparency.

Power shield 2, `rs_powershield2_vs.lta`—A power shield example. Expands the model by its normals, uses the the eye dot normal for transparency, and generates a psuedo-random-rotation UV for a second texture with an alpha mask which is used for highlighting the shield.

Power shield 3, `rs_powershield2_vs.lta`—A power shield example. Very much like `rs_powershield2_vs.lta`, except that it inverts the e dot n for the UVs, expands the model even more. Creates a foggy looking shield with the highlight texture color and alpha coming through in the effect.

Cartoon render with base texture, `rs_toon_basetex_2pass_2texstage_vs.lta`—Very much like the original cartoon renderstyle, expect that it adds a base texture.

[Top](#)